Inhaltsverzeichnis

1. Motivation **2**

2. Verwandte Arbeiten **3**
2.1. Explainable AI . . . 3
2.2. MOOCs . . . 6
2.3. Interaktive webbasierte Lernplattformen . . . 7

3. Technische Grundlagen **8**
3.1. Skalierbare Vektorgrafiken . . . 8
3.2. Visualisierungsbibliothek D3.js . . . 15
3.3. JavaScript Frontend Bibliothek React . . . 22
3.3.1. React - Projektaufbau . . . 22
3.3.2. Funktionskomponenten . . . 23

4. Konzept **25**
4.1. Konzepte des maschinellen Lernens . . . 25
4.2. Modularisierung . . . 31
4.3. Interaktives "Scrollytelling" . . . 32
4.4. Interaktive Demonstrationen . . . 34

5. Implementation **37**
5.1. Projektstruktur . . . 37
5.1.1. Komponentenstruktur . . . 37
5.1.2. Routen . . . 39
5.2. Scrollytelling Gerüst . . . 41
5.2.1. React Scrollama . . . 42
5.2.2. Komponentenwiederverwendbarkeit . . . 45
5.2.3. Responsives Design . . . 46
5.3. Visualisierungen . . . 48
5.3.1. Präsentation der Visualisierung . . . 48
5.3.2. Plotten von Funktionen . . . 50
5.3.3. Plotten von 3D-Funktionen . . . 53
5.4. Interaktive Demos . . . 54
5.4.1. Interaktion mit Visualisierungsparametern . . . 54
5.4.2. Visualisierungsberechnungen mit `numeric.js` . . . 56
5.4.3. Handschrifterkennung im Web . . . 57

6. Erweiterungsmöglichkeiten und Ausblick **69**

Anhänge **70**

A. Bibliographie **71**

Abstract

Das Ziel der vorliegenden Arbeit ist es, eine interaktive und webbasierte Lernplattform für das Themengebiet des maschinellen Lernens zu entwickeln. Im Verlauf dieser Arbeit wird eine Methodik entwickelt, die nicht nur auf das maschinelle Lernen beschränkt ist, sondern auch auf andere Themenbereiche angewendet werden kann. Der Ansatz, komplexe Themen in kleinere, abgeschlossene Abschnitte zu unterteilen und diese schrittweise zu erweitern, wird in dieser Arbeit eine hervorgehobene Rolle einnehmen. Die Arbeit präsentiert daher Methoden und Techniken, die es ermöglichen, die dafür erforderlichen Strukturen, Visualisierungen und interaktiven Demonstrationen zu erstellen und in einer Art und Weise darzustellen, die den Lernenden durch das Thema führt. Das dabei zentrale Konzept des *Scrollytellings* wird hierbei als Hauptmethode zur Vermittlung der Lehrinhalte verwendet. Somit soll gezeigt werden, dass die Vermittlung von oftmals schwer empfundenen Konzepten, wie dem maschinellen Lernen durch Interaktivität und Visualisierung erleichtert werden kann. Für eine effektive und unkomplizierte Lehrvermittlung ist es dabei essenziell, Lehrkonzepte in ihrer Erzählstruktur zu hinterfragen und sich Zeit für eine klare Strukturierung der Inhalte und eine intuitive Navigation zu nehmen. Kombiniert mit thematisch passenden interaktiven Demonstrationen wird so eine konsistente Erzählung geschaffen, die den Lernenden während des Lernprozesses motiviert und unterstützt.

1. Motivation

Der Themenbereich des maschinellen Lernens (ML) hat in den letzten Jahren ein kaum zu vergleichendes Interesse in der Wissenschaft und Industrie geweckt. Dieser Bereich ist jedoch nicht nur auf professionelle Domänen wie beispielsweise der Medizin oder Robotik beschränkt, sondern findet auch im alltäglichen Leben immer mehr Anwendung. Der Aufstieg von "Large Language Models", die steigende Integration von ML-Algorithmen in Handys und Computern sowie die zunehmende Verbreitung von "Deep Learning" in der Bild- und Spracherkennung sind nur einige Beispiele, die dies bestätigen.
Die daher steigende Relevanz, grundlegende Konzepte des maschinellen Lernens zu verstehen, ist daher unumstritten. Diese Konzepte werden jedoch aufgrund ihrer eher mathematischen Natur oft als "Black Box"-Thema wahrgenommen und gelten als schwer verständlich. Die Tatsache, dass die Lehrvermittlung in diesem Bereich in den meisten Fällen universitär und/oder institutionell erfolgt, erschwert den Zugang zu diesen Konzepten zusätzlich, zumal die Lehrqualität und -methoden stark variieren.
Die vorliegende Arbeit soll daher eine Alternative zu etablierten Lehrmethoden und -plattformen anbieten und schlägt eine webbasierte und möglichst barrierearme Lernplattform [1] vor, die die Vermittlung von Konzepten des maschinellen Lernens durch Interaktivität und Visualisierung erleichtert. Die hier vorgestellten Konzepte zeigen Techniken und Methoden auf, die schwer verständliche Konzepte aufbrechen und in einer Art und Weise präsentieren, die den Lernenden durch das Thema führt. So sind die vermittelten Konzepte nicht nur auf den Bereich des maschinellen Lernens beschränkt und können als Anreiz dienen, ähnliche Lehrmethoden auch in anderen wissenschaftlichen und technischen Bereichen zu etablieren.
Aus diesem Grund beschäftigt sich diese Arbeit hauptsächlich mit der technischen Umsetzung einer interaktiven und webbasierten Lernplattform und ist nicht als vertiefte Einführung in die Thematik des maschinellen Lernens zu verstehen. Hierfür wird in Kapitel 2 genauer motiviert, warum eine solche Plattform sinnvoll ist, indem bestehende und etablierte webbasierte Lernplattformen sowie Techniken in ihrem Konzept und ihrer Umsetzung hinterfragt und Probleme, sowie Schwächen aufgezeigt werden. Die für die Umsetzung verwendeten und wesentlichen technischen Grundlagen werden in Kapitel 3 schrittweise und von Grund auf erklärt. Im dritten Kapitel, welches sich mit der Konzeption des Webprojekts beschäftigt, wird neben einer allgemeinen Beschreibung der im Webprojekt vermittelten Themen des maschinellen Lernens ebenfalls eine konzeptionelle Alternative zu etablierten webbasierten Lernplattformen vorgeschlagen. Das hierbei zentrale Konzept ist das sogenannte Scrollytelling, welches als interaktive und visuelle Geschichtserzählung verstanden werden kann. Zusammen mit interaktiven Demonstrationen von Algorithmen und Konzepten des maschinellen Lernens bilden diese die Grundlage für die interaktive und webbasierte Lernplattform. Diese Konzepte werden genauer in einem Implementationkapitel beschrieben und die dafür notwendigen Techniken und Methoden erläutert, darunter die Erstellung eines Scrollytelling-Gerüsts, die Implementierung von interaktiven Visualisierungen und Demonstrationen von Algorithmen des maschinellen Lernens sowie Beispiele für Echtzeit-Klassifikationsmodelle im Web.

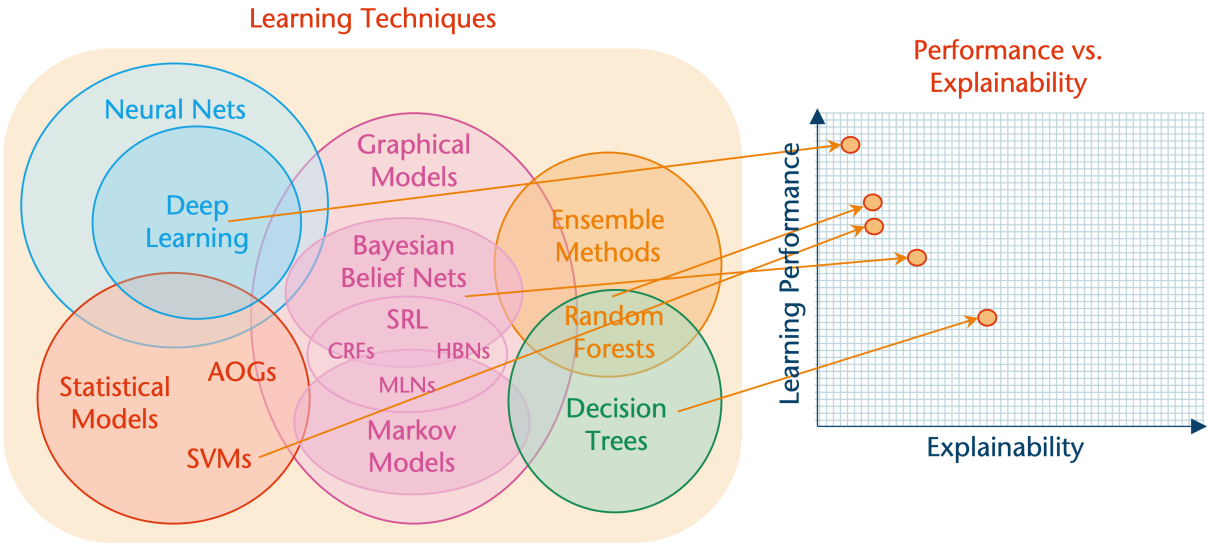
[1] Link zur Lernplattform: `https://scrollytelling-ml-medialab-e2608723069b047745ff21c2aab2b0bdc3f6.pages.rwth-aachen.de/`

2. Verwandte Arbeiten

Die Erfolge von maschinellen Lernverfahren und insbesondere von neuronalen Netzen sind unbestritten und finden in vielen Bereichen Anwendung. Dennoch ist nicht abzustreiten, dass immer komplexere Modelle und Algorithmen entwickelt werden, die nicht nur für Endnutzer, sondern auch für Entwickler schwer zu verstehen sind, man spricht hier von "Black Box"-Modellen. Ein Modell macht beispielsweise eine korrekte Klassifikation von einem Bild, aber der Nutzer selbst kann nicht nachvollziehen, warum das Modell diese Entscheidung getroffen hat.

2.1. Explainable AI

Der über die Jahre gewachsene Forschungszweig der *Explainable AI* (XAI) beschäftigt sich mit dem Öffnen dieser "Black Boxen" und versucht, die Entscheidungsfindung von Modellen nachvollziehbar und verständlich zu machen. So hat sich herausgestellt, dass mit steigender Komplexität von Modellen auch die Erklärbarkeit sinkt (vgl. [18]):

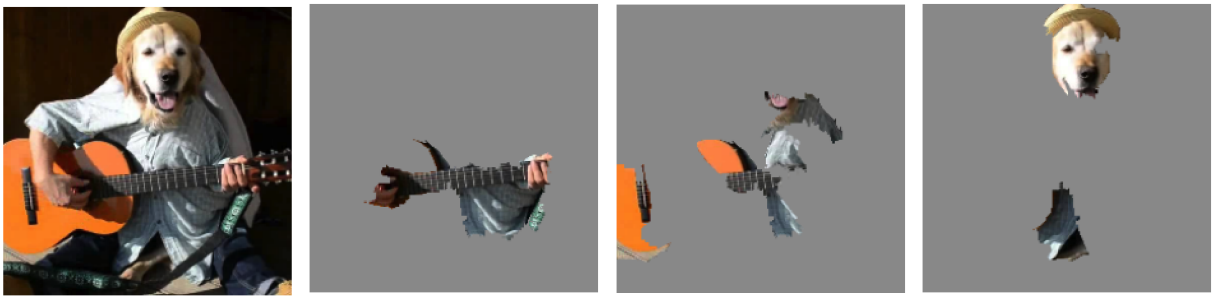


Erklärbarkeit von Modellen in Abhängigkeit von ihrer Komplexität (Grafik von [18])

Obige Grafik zeigt, dass einfache Modelle wie Entscheidungsbäume eine hohe Erklärbarkeit aufweisen, während Deep Learning Modelle wie neuronale Netze die geringste Erklärbarkeit haben. Der entscheidende Punkt ist hierbei, dass die am wenigsten erklärbaren Deep Learning Modelle, die höchste Vorhersagegenauigkeit aufweisen, was sie für viele Anwendungen unverzichtbar macht. In gewissen Anwendungsbereichen ist es jedoch unerlässlich, dass die Entscheidungen von Modellen nachvollziehbar und verständlich sind, um Fehlentscheidungen zu vermeiden und Vertrauen in die Modelle zu schaffen. So beispielsweise in der Medizin, hier können Fehlentscheidungen schwerwiegende Folgen haben und die Erklärbarkeit von Modellen ist daher von besonderer Bedeutung. Xu et al. berichten in ihrer Arbeit "Explainable AI: A Brief Survey on History, Research Areas, Approaches and Challenges" (vgl. [35]) von einem prominenten Fall, wo ein Modell welches zur Risikovorhersage für Pneumonie aufgrund Verzerrungen im Trainingsdatensatz völlig falsche Vorhersagen getroffen hat (vgl. [6]). Asthma-Patienten mit Herzproblemen schienen gemäß dem Modell ein geringeres Risiko für Pneumonie zu haben als gesunde Patienten,

Pneumonie: Fachbegriff für eine Lungenentzündung

was offensichtlich falsch ist (Asthma wirkt sich negativ auf die Lungenfunktion aus und erschwert die Erholung). Der Grund für diese Fehlentscheidung war eine systematische Verzerrung/ein Bias im Trainingsdatensatz. Im Gegensatz zu gesunden Patienten wurden die meisten Asthma-Patienten streng medizinisch überwacht, was im Modell zu einer geringeren Risikoeinstufung für Pneumonie führte. Hier ist zu betonen, dass das Modell an sich nicht das Problem darstellt, sondern die Verzerrung im Trainingsdatensatz, welche jedoch aufgrund einer Intransparenz/Komplexität des Modells unentdeckt blieb. Das obige Beispiel soll verdeutlichen, dass es von großer Bedeutung ist, Entscheidungen von Modellen nachvollziehen zu können, insbesondere für Modelle mit komplexen Deep-Learning-Architekturen welche nicht durchgängig erklärbar sind. Aus diesen Gründen wurden verschiedene Ansätze und Methoden entwickelt, um die Erklärbarkeit von Modellen zu verbessern. Eine Technik, die in diesem Zusammenhang oft genannt wird, ist die sogenannte LIME-Methodik (Local Interpretable Model-agnostic Explanations) (vgl. [30]). Diese Methode versucht komplexe Entscheidungen, (beispielsweise, wie sie in der Bildklassifikation getroffen werden) verständlich zu machen, indem sie die Entscheidung eines Modells auf eine lokale Ebene herunterbricht. Somit werden leicht verständliche Erklärungen für einzelne Vorhersagen getroffen.

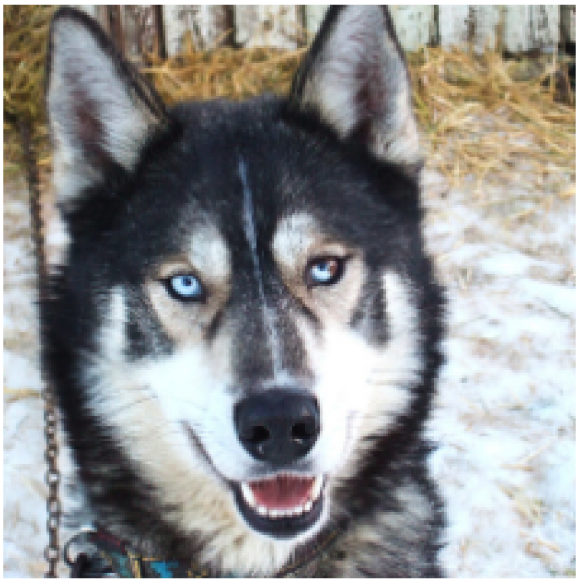


(1) Original Bild | (2) Erklärung für E-Gitarre ($p = 0.32$) | (3) Erklärung für Akustik-Gitarre ($p = 0.24$) | (4) Erklärung für Labrador ($p = 0.21$)

Grafiken von [30]

Dabei analysiert LIME, welche Teile der Eingabedaten (beispielsweise gewisse Bildbereiche) am wichtigsten für die Entscheidung des Modells sind und stellt diese lokal für eine bestimmte Vorhersage dar. Im obigen Beispiel wird die Entscheidung eines Bildklassifikationsmodells für ein künstlich generiertes Bild eines Labradors mit Gitarre erklärt. Die Erklärung zeigt, dass gewisse Bildbereiche unterschiedlich stark zur Entscheidung beitragen. So wird beispielsweise die Entscheidung für eine E-Gitarre durch die Saiten und den Körper der Gitarre beeinflusst, während die Entscheidung für einen Labrador durch den Kopf und den Körper des Hundes beeinflusst wird. Somit werden die Entscheidungen des zugrunde liegenden Modells selbst für Laien verständlich und nachvollziehbar gemacht.

Insbesondere können ebenfalls offensichtliche Fehlentscheidungen von Modellen aufgedeckt werden:



Ein Husky, welcher als Wolf fehlklassifiziert wird

Erklärung durch LIME

Grafiken von [30]

Die Ersteller der LIME-Technik beschreiben in ihrer Arbeit "'Why Should I Trust You?' Explaining the Predictions of Any Classifier" [30] mittels eines Experimentes, welches aufzeigt, wie ein fehlerhaftes Modell aufgrund von unerwünschten Korrelationen falsche Entscheidungen treffen kann. In dem Experiment wird ein Modell darauf trainiert, zwischen Huskies und Wölfen zu unterscheiden, wobei alle Wolf-Bilder Schnee im Hintergrund haben, während dies bei den Husky-Bildern nicht der Fall ist. Fälschlicherweise lernt das Modell dabei, den Schnee als Hauptmerkmal für einen Wolf zu identifizieren. Mittels der XAI-Technik LIME konnte jedoch gezeigt werden, dass das Modell hauptsächlich den Hintergrund (den Schnee) anstelle des eigentlichen Tieres für seine Entscheidung nutzt. XAI-Techniken wie LIME können somit dazu beitragen die Erklärbarkeit von Modellen zu verbessern und somit Vertrauen in die Modelle zu schaffen.
Jedoch ist das Hinterfragen der Entscheidungsfindung von Modellen nur ein Aspekt für ein besseres Verständnis und eine höhere Akzeptanz von Modellen. Ein weiterer Aspekt ist die Vermittlung von Konzepten und Grundlagen des maschinellen Lernens selbst. Konkret bedeutet dies, dass nicht nur das Verhalten oder die Entscheidungsfindung von Modellen als "Black Box" entlüftet werden sollte, sondern ebenfalls die zugrunde liegenden Struktur und Funktionsweise dieser Modelle, welche ebenfalls für viele als "Black Box"-Thema wahrgenommen wird. "Wie können neuronale Netze Vorhersagen über medizinische Diagnosen treffen?" oder "Wie können Autos autonom fahren?" sind Fragen, die nicht allein durch die Beschreibung bestehender Modelle beantwortet werden können. Mit der zunehmenden Präsenz von ML-Modellen im Alltag und ihrer Anwendung in sicherheitskritischen und medizinischen Bereichen betreffen diese Fragen zunehmend auch eine größere Öffentlichkeit, weshalb ein grundlegendes Verständnis von Modellen und Algorithmen des maschinellen Lernens für eine breite Masse von Bedeutung ist. Hier soll das zu vorzustellende Konzept einer interaktiven und webbasierten Lernplattform ansetzen, um die Vermittlung von Konzepten des maschinellen Lernens zu erleichtern.

2.2. MOOCs

Betrachtet man bestehende webbasierte Lernplattformen, die versuchen, den Lernprozess von Modellen oder allgemein das Konzept von Maschinen selbstständig Muster zu erkennen, um Vorhersagen oder Entscheidungen zu treffen, so sind vor allem MOOCs (Massive Open Online Courses) zu nennen. Diese Plattformen bieten Kurse zu verschiedensten Themen an, die in den meisten Fällen von Universitäten oder anderen Bildungseinrichtungen erstellt werden. So auch für das Themengebiet des maschinellen Lernens, welches in den letzten Jahren aufgrund steigender Bedeutung an Interesse gewonnen hat. Typische Beispiele für MOOCs sind Plattformen wie Coursera, edX oder Udacity. So bietet ebenfalls die RWTH mit ihrem Kurs "bridgingAI" eine videobasierte Einführung in die Grundlagen der künstlichen Intelligenz und damit insbesondere in das maschinelle Lernen an (vgl. [21]). Die offensichtlichen Vorteile von MOOCs sind die hohe Skalierbarkeit und die Möglichkeit, fachliche Expertise von weltweit führenden Universitäten und Experten zu erhalten, wodurch Bildung für eine breite Öffentlichkeit zugänglich wird (vgl. [7]). Das Konzept von MOOCs hat jedoch wesentliche Schwächen, die es zu berücksichtigen gilt:

MOOC	Students enrolled	Students completed	Percentage of Completion	Platform
Equine Nutrition	23322	8416	36.1%	Coursera
Astrobiology	39556	7707	19.5%	Coursera
Functional Programming Principles in Scala	50000	9593	19.2%	Coursera
Computing for Data Analysis	50899	6420	12.6%	Coursera
introduction to Machine Learning	104000	13000	12.5%	Coursera
Databases	60000	6500	10.8%	Coursera
Image and video processing - From Mars to Hollywood with a stop at the hospital	40000	4069	10.2%	Coursera
Internet History, Technology and Security	46000	4595	10.1%	Coursera
Gamification	81600	8280	10.1%	Coursera
Introduction to Philosophy	98128	9445	9.6%	Coursera
Critical Thinking	75884	6909	9.1%	Coursera
Mathematical Biostatistics Bootcamp	21916	2087	9.5%	Coursera

• • •

Abschlussraten von MOOCs (Grafik von [20])

Khalil et al. beschreiben in ihrer Arbeit "MOOCs Completion Rates and Possible Methods to Improve Retention - A Literature Review" [20] die geringen Abschlussraten von MOOCs. So liegen viele Abschlussraten von MOOCs bei unter 15%, was bedeutet, dass nur ein Bruchteil der Teilnehmer den Kurs erfolgreich abschließt. In der obigen Grafik ist beispielsweise zu sehen, dass die Abschlussrate des Kurses "Introduction to Machine Learning" von Coursera lediglich bei 12.5% liegt. Die Studie sieht die Ursachen für die geringe Abschlussrate in verschiedenen Faktoren, darunter die mangelnde Motivation, geringe Interaktivität, unzureichende Vorkenntnisse und versteckte Kosten. Berücksichtigt man, dass viele Teilnehmer von MOOCs freiwillig und ohne externe Motivation an den Kursen teilnehmen (vgl. [16]), jedoch nur ein Bruchteil der Teilnehmer den Kurs abschließen, so ist es offensichtlich, dass Motivation ein entscheidender Faktor für den Lernerfolg ist. Der in dieser Arbeit zu präsentierende interaktive Ansatz einer webbasierten Lernplattform soll diese Schwächen vermeiden, indem mangelnde Motivation und Interaktivität durch interaktive Demonstrationen und Visualisierungen ersetzt werden und ein klarer Lernpfad angeboten wird. Insbesondere sollen Konzepte die als schwer verständlich wahrgenommen werden, durch visuelle und interaktive Darstellungen aufgebrochen werden, um den Umfang an Vorwissen zu reduzieren.

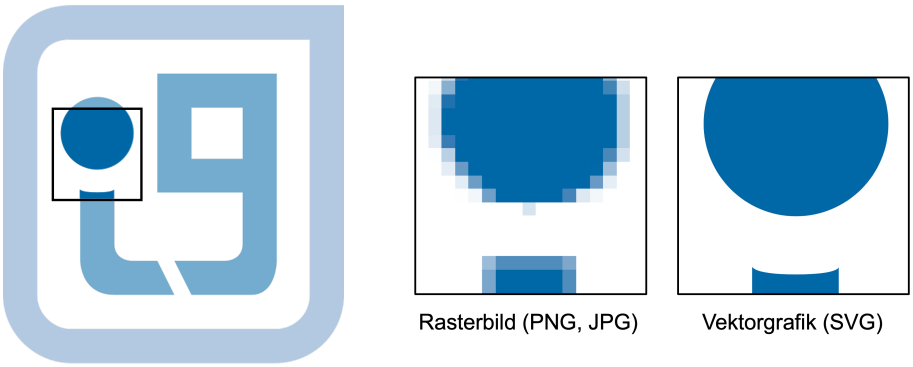
3. Technische Grundlagen

Um die wesentlichsten Bausteine des Webprojekts zu verstehen, soll in diesem Kapitel auf die dafür technischen Grundlagen eingegangen werden. Nicht jeder der folgenden Codeabschnitte wird in der vorliegenden Arbeit in exakt dieser Form wiederzufinden sein. Vielmehr soll ein Überblick und ein gewisser Grad an Verständnis für die zu vorstellende `svg`-Sprache vermittelt werden, die für die Projektarbeit mit `D3.js` essenziell ist. Für eine ausführlichere Beschreibung der behandelten Themen wird auf die jeweiligen Quellen verwiesen.

3.1. Skalierbare Vektorgrafiken

Das wichtigste Medium zur Darstellung von interaktiven Grafiken und interaktiven Demos im Bachelorprojekt sind skalierbare Vektorgrafiken (Scalable Vector Graphics - SVG). Sie sind ein Webstandard und dienen der Beschreibung von zweidimensionalen Vektorgrafiken, die sich in HTML-Dokumente einbetten lassen. Der dabei entscheidende Vorteil gegenüber Rastergrafiken wie JPG oder PNG ist, dass SVG-Grafiken nicht an Qualität verlieren, wenn sie vergrößert werden (vgl. [34]).

JPG: Joint Photographic Experts Group, PNG: Portable Network Graphics



Beispiel einer SVG-Grafik (Logo des Lehrstuhls)

Im Folgenden soll die `svg`-Sprache in größerem Detail erklärt werden, da sie die Grundlage für die im Projekt verwendete Visualisierungsbibliothek `D3.js` bildet.

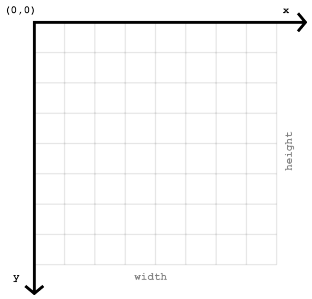
SVG-Positionierung

Die `svg`-Sprache benutzt dabei ein Koordinatensystem, welches den Ursprung in der oberen linken Ecke hat. Mittels `width` und `height`-Attributen kann die Größe des SVG-Elements definiert werden. Das Koordinatensystem wird jedoch nicht durch die Größe des SVG-Elements definiert, sondern durch das `viewbox`-Attribut (vgl. [26]). Die `viewbox` folgt dem Muster

`originX originY width height`

wobei `originX` und `originY` die Koordinaten des Ursprungs und `width` und `height` die Dimensionen des sichtbaren Bereichs festlegen.

```svg
<svg width="200" height="200" viewbox="0 0 100 100">
  ...
</svg>
```

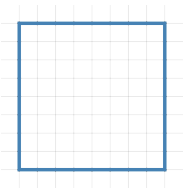


Im Beispiel heißt das, dass das SVG-Element eine tatsächliche Größe von 200×200 Pixeln hat, jedoch dieser Bereich durch ein Koordinatensystem von 100×100 Einheiten definiert wird. Dies erleichtert die Positionierung von Elementen innerhalb der SVG, da nun die Positionierung einer günstigen Konvention folgen kann.

Einfache SVG-Formen

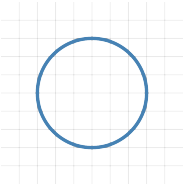
In einem SVG-Tag können verschiedene vordefinierte Formen wie Rechtecke, Kreise, Ellipsen, Linien, Polygone und Pfade definiert werden (vgl. [23]). Für ein Rechteck ist die Ursprungskoordinate `(x, y)` anzugeben, von der das Rechteck mit der Breite (`width`) und Höhe (`height`) gezeichnet wird. Die Ecken des Rechtecks können mit den Attributen `rx` und `ry` abgerundet werden.

```svg
<svg viewbox="0 0 100 100">
  <rect x="10" y="10" width="80" height="80"
        rx="0" ry="0"/>
</svg>
```



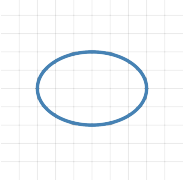
Ein Kreis ist mit Mittelpunkt `(cx, cy)` und Radius `r` anzugeben:

```svg
<svg viewbox="0 0 100 100">
  <circle cx="50" cy="50" r="30" />
</svg>
```



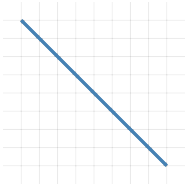
Für eine Ellipse sind dementsprechend die Koordinaten des Mittelpunkts `(cx, cy)` und die Radien `rx` und `ry` anzugeben:

```svg
<svg viewbox="0 0 100 100">
  <ellipse cx="50" cy="50" rx="30" ry="20" />
</svg>
```



Eine Linie wird durch die Koordinaten der Start- und Endpunkte `(x1, y1)` und `(x2, y2)` definiert:

```
<svg viewbox="0 0 100 100">
  <line x1="10" y1="10" x2="90" y2="90" />
</svg>
```

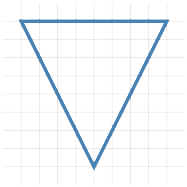


Eine etwas komplexere Form ist die sogenannte `polyline`, welche eine Gruppe an verbundenen Linien beschreibt. Die verbundenen Linien werden durch eine Liste `points` an Koordinatenpunkten definiert, wobei die Linien in der Reihenfolge der Koordinatenpunkte gezeichnet werden (im Beispiel: (10,10) → (50,90) → (50,50)).

```
<svg viewbox="0 0 100 100">
  <polyline points="10,10 50,90 50,50" />
</svg>
```

Eine `polygon` ist eine geschlossene Form einer `polyline`, wobei die letzte Koordinate mit der ersten Koordinate automatisch verbunden wird:

```
<svg viewbox="0 0 100 100">
  <polygon points="10,10 90,10 50,90" />
</svg>
```



SVG-path

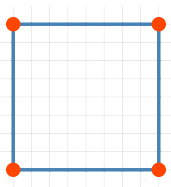
Die jedoch vielseitigste Form ist `<path>`, welche eine beliebige Form beschreiben kann, unter anderem die oben Genannten. Mittels des `d`-Attributs wird die Form durch eine Reihe von Befehlen definiert, welche Linien, Kurven und Bögen beschreiben können. In der Hinsicht ist `<path>` ähnlich zur `<polyline>`, jedoch müssen Kurven nicht durch viele kleinere Linien simuliert werden. Jeder Befehl besteht aus einem Buchstaben, welcher die Art der Aktion beschreibt und einer Liste von Koordinaten (vgl. [25]).
Mit dem Befehl `M` wird der Startpunkt definiert, dabei wird jedoch noch nichts gezeichnet (ähnlich wie einen Stift auf ein Blatt Papier zu setzen, bevor man anfängt zu zeichnen).

Aktuelle "Stift"-Position wird in den Beispielen mit einem Punkt illustriert

```
<svg viewbox="0 0 100 100">
  <path d="M 10 10" />
</svg>
```

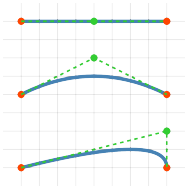

Mit dem Befehl `L` wird eine Linie von der aktuellen Position zu einer neuen Position `(x, y)` gezeichnet. Für vertikale und horizontale Linien gibt es die abkürzenden Befehle `V` und `H`, die nur eine Koordinate benötigen, da die jeweils andere Koordinate von der aktuellen Position übernommen wird. Ein weiterer abkürzender Befehl ist `Z`, welcher die Form schließt, das heißt eine Linie von der aktuellen Position zur Startposition zieht.

```
<svg viewbox="0 0 100 100">
  <path d="M 10 10 L 90 10 V 90 H 10 V 10" />
  <!-- oder: -->
  <path d="M 10 10 L 90 10 V 90 H 10 Z"/>
</svg>
```



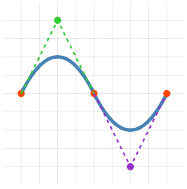
Um glatte Kurven zu zeichnen, gibt es im Wesentlichen drei Befehle. Die dabei einfachste Kurve ist die quadratische Bezierkurve, welche durch den Befehl `Q` definiert wird. Quadratische Bezierkurven benötigen einen Kontrollpunkt `(x1, y1)` und einen Endpunkt `(x, y)`. Dabei ist der Kontrollpunkt eine Art Anziehungspunkt für die Kurve, er lenkt die Kurve in seine Richtung.

```
<svg viewbox="0 0 100 100">
  <path d="M 10 10 Q 50 10, 90 10" />
  <path d="M 10 50 Q 50 30, 90 50" />
  <path d="M 10 90 Q 90 70, 90 90" />
</svg>
```



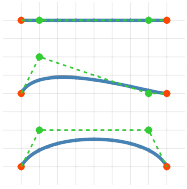
Mit dem Befehl `T` kann eine quadratische Bezierkurve nahtlos fortgesetzt werden, wobei der Kontrollpunkt der Fortsetzung automatisch berechnet wird. Dabei ist der Kontrollpunkt der Fortsetzung der Spiegelungspunkt des Kontrollpunktes der vorherigen Kurve, sodass lediglich der Endpunkt der Fortsetzung angegeben werden muss.

```
<svg viewbox="0 0 100 100">
  <path d="M 10 50 Q 30 10, 50 50 T 90 50" />
</svg>
```



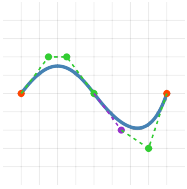
Die zweite Methode, um Kurven zu zeichnen ist die kubische Bezierkurve, welche durch den Befehl `C` definiert wird. Diese Art Kurven zu zeichnen ist ähnlich zur quadratischen Bezierkurve, jedoch benötigt die kubische Bezierkurve zwei Kontrollpunkte `(x1, y1)` und `(x2, y2)`. Die Kurve endet ebenfalls in einem Endpunkt `(x, y)`. Der extra Kontrollpunkt ermöglicht eine noch größere Kontrolle über die Form der Kurve.

```
<svg viewbox="0 0 100 100">
  <path d="M 10 10 C 20 10, 80 10, 90 10" />
  <path d="M 10 50 C 20 30, 80 50, 90 50" />
  <path d="M 10 90 C 20 70, 80 70, 90 90" />
</svg>
```



Ähnlich wie quadratische Bezierkurven mittels des Befehls `T` nahtlos fortgesetzt werden können, können kubische Bezierkurven mittels des Befehls `S` fortgesetzt werden. Der Unterschied ist, dass der Kontrollpunkt der Fortsetzung der Spiegelungspunkt des zweiten Kontrollpunktes der vorherigen Kurve ist. Hier muss lediglich der zweite Kontrollpunkt der Fortsetzung und der Endpunkt angegeben werden.

```
<svg viewbox="0 0 100 100">
  <path d="M 10 50 C 25 30, 35 30, 50 50 S 80 80, 90 50"/>
</svg>
```

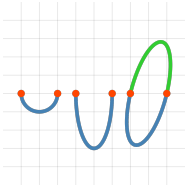


Die letzte Methode um Kurven zu zeichnen ist die elliptische Bogenkurve, welche durch den Befehl `A` definiert wird. Sie folgt dem Schema:

```
A rx ry x-axis-rotation large-arc-flag sweep-flag x y
```

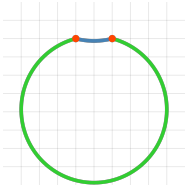
wobei `rx` und `ry` die Radien der Ellipse sind, `x-axis-rotation` die Rotation der Ellipse, `large-arc-flag` und `sweep-flag` Flags sind, die die Richtung der Ellipse bestimmen, und `x y` der Endpunkt der Kurve ist. Die Radien sind dabei eher als Verhältnis zu verstehen (wenn beispielsweise `rx = ry = 1` ist die Ellipse ein Kreis). Das `sweep-flag` gibt an, welcher Teil der Ellipse für den Bogen verwendet wird.

```
<svg viewbox="0 0 100 100">
  <path d="M 10 50 A 1 1, 0, 0 0, 30 50"/>

  <path d="M 40 50 A 1 3, 0, 0 0, 60 50"/>

  <path d="M 70 50 A 1 3, 15, 0 0, 90 50"/>
  <path d="M 70 50 A 1 3, 15, 0 1, 90 50"/>
</svg>
```



Falls man die Radien so wählt, dass sie größer sind als die Distanz zwischen Start- und Endpunkt, wird die Ellipse gestreckt und über diese Distanz hinweg gezeichnet. Im Beispiel unten wird die Ellipse gestreckt, da die Radien `rx = 40` und `ry = 40` größer sind als die Distanz zwischen Start- und Endpunkt. Ist das Flag `large-arc-flag` gesetzt, wird der größere Teil der Ellipse für den Bogen verwendet, ist es nicht gesetzt, wird der kleinere Teil verwendet.

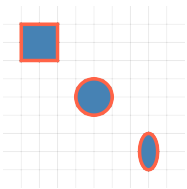
```
<svg viewbox="0 0 100 100">
  <path d="M 40 20 A 40 40, 0, 0 0, 60 20"/>
  <path d="M 40 20 A 40 40, 0, 1 0, 60 20"/>
</svg>
```



SVG-Styling

Einfache Färbungen können mittels des `fill`- und `stroke`-Attributs definiert werden. Das `fill`-Attribut definiert die Füllfarbe einer Form, während das `stroke`-Attribut die Farbe der Linie/Umrandung einer Form definiert. Mittels des `stroke-width`-Attributs kann die Dicke der Linie/Umrandung definiert werden. Manipulationen von mehreren SVG-Elementen wie beispielsweise Styleänderungen müssen nicht auf jedes Element einzeln angewendet werden, sondern können auf eine Gruppe von Elementen angewendet werden. Dazu wird ein `<g>`-Tag verwendet, welches eine Gruppe von Elementen definiert (vgl. [24]).

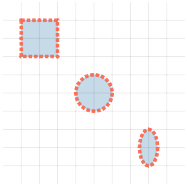
```
<svg viewbox="0 0 100 100">
  <g fill="steelblue" stroke="tomato" stroke-width="2">
    <rect x="10" y="10" width="20" height="20" />
    <circle cx="50" cy="50" r="10" />
    <ellipse cx="80" cy="80" rx="5" ry="10" />
  </g>
</svg>
```



Externes CSS-Styling ist ebenfalls möglich (wurde in den vorherigen Beispielen angewandt)

Weitere typische Styleoptionen sind die Transparenz der Füllfarbe (`fill-opacity`) und der Linienfarbe (`stroke-opacity`). Ferner kann mittels des `stroke-dasharray`-Attributs eine gestrichelte Linie definiert werden.

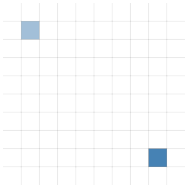
```
<svg viewbox="0 0 100 100">
  <g fill="steelblue" stroke="tomato" stroke-width="2"
     fill-opacity="0.3" stroke-opacity="0.9" stroke-
     dasharray="2,1">
    ...
  </g>
</svg>
```



SVG-Transformationen

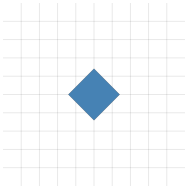
SVG-Elemente können mittels Transformationen in Größe, Position und Rotation verändert werden. Die Transformationen werden mittels des `transform`-Attributs definiert (vgl. [27]).
Die einfachste Transformation ist die Translation, welche ein Element in x- und y-Richtung verschiebt. Im Beispiel wird ein Rechteck von der Position (10, 10) um 70px in x- und y-Richtung verschoben, sodass es bei (80, 80) liegt

```
<svg viewbox="0 0 100 100">
  <rect x="10" y="10" width="10" height="10"
   transform="translate(70,70)"/>
</svg>
```



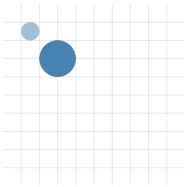
Mittels des `rotate`-Attributs kann ein Element um einen bestimmten Winkel an einem bestimmten Punkt rotiert werden. Im folgenden Beispiel wird ein Rechteck um 45 Grad an seinem Mittelpunkt (50, 50) rotiert.

```
<svg viewbox="0 0 100 100">
  <rect x="40" y="40" width="20" height="20"
   transform="rotate(45, 50, 50)"/>
</svg>
```



Das `scale`-Attribut skaliert ein Element in x- und y-Richtung um einen bestimmten Punkt. Beispielsweise wird ein Kreis mit Mittelpunkt `(15, 15)` und Radius `5` um den Faktor 2 in x- und y-Richtung skaliert, sodass der skalierte Kreis den Mittelpunkt `(30, 30)` mit neuem Radius `10` hat.

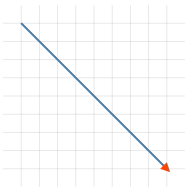
```
<svg viewbox="0 0 100 100">
  <circle cx="15" cy="15" r="5" transform="scale(2,2)"/>
</svg>
```



SVG-Marker

Mittels SVG-Marker können bestimmte Punkte eines Pfades, wie zum Beispiel Anfangs-, Mittel- oder Endpunkte, durch beliebige Formen ergänzt werden. Im Beispiel wird eine Pfeilspitze (`<polygon>`) als Marker definiert und an das Ende einer Linie angehängt (`marker-end`). Diese werden in einem `<defs>`-Tag definiert, und mittels `url(#marker-id)` referenziert. Um die Erstellung zu vereinfachen, wird mit `viewbox` ein passendes Koordinatensystem definiert. Der Referenzpunkt `(refX, refy)` bestimmt die Anhangstelle des Markers an den Pfad, das `orient`-Attribut die Ausrichtung und `markerWidth` und `markerHeight` die Größe des Markers. SVG-Marker vereinfachen die Darstellung von Pfeilen und anderen Formen an Pfaden und werden im Projekt oft für die Darstellung von Vektoren oder Achsen verwendet (vgl. [33]).

```svg
<svg viewbox="0 0 100 100">
  <defs>
    <marker id="x-arrow-head" viewbox="0 0 10 10"
            refX="5" refY="5" markerWidth="5"
            markerHeight="5" orient="auto">
      <polygon points="0,0 10,5 0,10"/>
    </marker>
  </defs>

  <line x1="10" y1="10" x2="90" y2="90"
        marker-end="url(#x-arrow-head)"/>
</svg>
```



3.2. Visualisierungsbibliothek D3.js

Für die interaktiven Visualisierungen in der Webanwendung wird die JavaScript-Bibliothek D3.js [3] verwendet. Viele webbasierte Visualisierungsbibliotheken verfolgen einen Ansatz von vorgefertigten Diagrammtypen, die vom Entwickler lediglich konfiguriert werden müssen. Oftmals abstrahieren diese Bibliotheken jedoch den Visualisierungsprozess zu stark und grenzen somit Flexibilität in der Erstellung von Visualisierungen ein, vor allem wenn die gewünschte Visualisierung nicht einer der Vorlagen entspricht. D3.js hingegen verfolgt einen grundlegenderen Ansatz, es wird sehr nah an den Webstandards HTML, SVG und CSS gearbeitet und löst von spezifischen Visualisierungsvorgaben, welches für die Erstellung von benutzerdefinierten Visualisierungen abseits von Standarddiagrammen sehr vorteilhaft ist (vgl. [22]). Als oft benannter Nachteil von D3.js wird die steile Lernkurve und der hohe Einarbeitungsaufwand genannt. Ersteres ist zumindest durch eine Einarbeitung in den Webstandard SVG zu mildern.

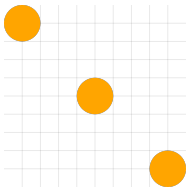
(Datenbasierte) Modifikation des DOM

Über D3 kann der DOM-Baum selektiert und manipuliert werden. Neben der Selektion von HTML-Elementen werden typischerweise SVG-Elemente selektiert und für eine entsprechende Visualisierung manipuliert. Mit `d3.select()` und `d3.selectAll()` können Elemente selektiert werden und mit

- `selection.attr()`
- `selection.style()`
- `selection.text()`
- `selection.html()`
- `selection.classed()`
- `selection.property()`

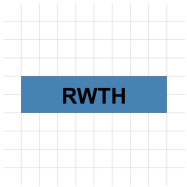
können die Attribute, Styles, Textinhalte, HTML-Inhalte, Klassen und Eigenschaften der selektierten Elemente verändert werden (vgl. [13]). Dabei nehmen `d3.select()` und `d3.selectAll()` als Argumente einen String in Form eines CSS-Selektors an:

```
<svg>
  <circle cx="10" cy="10" r="10" />
  <circle cx="50" cy="50" r="10" />
  <circle cx="90" cy="90" r="10" />
</svg>

<script>
  d3.selectAll("circle").attr("fill", "orange");
</script>
```



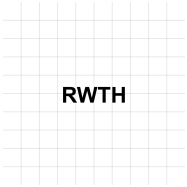
Neben Selektion und Manipulation von Elementen können auch neue Elemente erstellt und in den DOM-Baum eingefügt werden. Dies geschieht mittels `selection.append()` oder `selection.insert()`

```svg
<svg>
  <g class="item">
    <rect x="10" y="40" width="80" height="20"/>
  </g>
</svg>

<script>
  d3.select("g.item")
    .append("text")
    .text("RWTH")
    .attr("x", 50)
    .attr("y", 55)
    .attr("text-anchor", "middle");
</script>
```



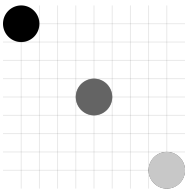
Dementsprechend können auch Elemente entfernt werden, dies geschieht mittels `selection.remove()`:

```svg
<script>
  ...
  d3.select("g.item").select("rect").remove();
</script>
```



Mittels `selection.data()` und `selection.join()` können Daten an Elemente gebunden werden, sodass eine Abhängigkeit zwischen den Daten und den selektierten Elementen hergestellt wird (vgl. [10]). Wird in `.attr` oder `.style` eine Funktion übergeben, ist es möglich, mit den Argumenten `d` welches den jeweiligen Datenpunkt repräsentiert und `i` welches den Index der Datenpunkte repräsentiert, die Attribute dynamisch und datenabhängig zu setzen. Im Folgenden bekommt jeder Datenpunkt ein Kreiselement im SVG-Element, wobei die Position und Größe des Kreises von den Daten abhängt:

```svg
<svg class="join-example">
</svg>

<script>
  const data = [
    { cx: 10, cy: 10, radius: 10 },
    { cx: 50, cy: 50, radius: 10 },
    { cx: 90, cy: 90, radius: 10 },
  ];

  d3.select(".join-example")
    .selectAll("circle")
    .data(data)
    .join("circle")
    .attr("cx", (d, i) => d.cx)
    .attr("cy", (d, _) => d.cy)
    .attr("r",  (d)    => d.radius)
    .attr("fill", (_, i) =>
     `rgb(${i * 100}, ${i * 100}, ${i * 100})`);
</script>
```



Im Beispiel werden ebenfalls verschiedene D3-Konventionen aufgezeigt, für die Verwendung der Argmuente `d` und `i`

Die Abfolge

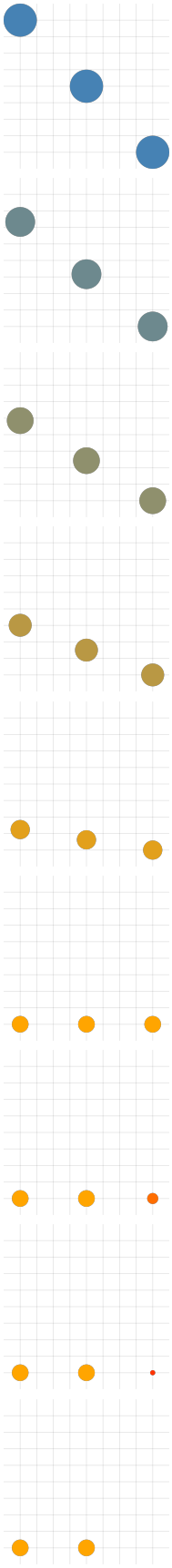
```
d3.select(".join-example")
  .selectAll("circle")
  .data(data)
  .join("circle");
```

scheint anfangs kontraintuitiv, da effektiv der Elementtyp zweimal angegeben wird, dies ist aber notwendig, um eine Auswahl zu erstellen, an die die Daten gebunden werden können. Auch wenn die Elemente noch nicht im DOM existieren, sagt `selectAll("circle")`, dass es sich um Kreise handelt, die später hinzugefügt oder aktualisiert werden sollen. Es erstellt quasi eine virtuelle Auswahl von Elementen, die noch nicht existieren. Mittels `.join("circle")` werden dann tatsächliche DOM-Elemente erstellt oder verwaltet. Es sorgt dafür, dass für jedes Datenobjekt ein entsprechendes Element vom Typ `"circle"` im DOM existiert. Wenn das Daten-Array größer ist als die aktuelle Anzahl der Kreise im DOM werden neue Kreise hinzugefügt. Wenn es kleiner ist, werden überschüssige Kreise entfernt.

Enter - Update - Exit

Das Verhalten der datengebundenen Funktion `.join()` kann ferner mittels der Methoden `.enter()`, `.update()` und `.exit()` genauer spezifiziert werden (vgl. [11]). Im folgenden Beispiel wird das Verhalten der Kreiselemente beim Hinzufügen, Aktualisieren und Entfernen von Datenpunkten spezifiziert. Im folgenden Beispiel werden für neue Datenpunkte, Kreise im `enter`-Abschnitt mit den angegebenen Attributen hinzugefügt und blau eingefärbt, für bestehende Datenpunkte werden die Kreise im `update`-Abschnitt zu ihrer neuen Position aktualisiert und orange eingefärbt und für entfernte Datenpunkte werden die Kreise in `exit`-Abschnitt entfernt:

```
<svg class="enter-update-exit-example"></svg>

<script>
  const dataState1 = [
    { cx: 10, cy: 10, radius: 10 },
    { cx: 50, cy: 50, radius: 10 },
    { cx: 90, cy: 90, radius: 10 },
  ];

  const dataState2 = [
    { cx: 10, cy: 90, radius: 5 },
    { cx: 50, cy: 90, radius: 5 },
    { cx: 90, cy: 90, radius: 5 },
  ];

  const dataState3 = [
    { cx: 10, cy: 90, radius: 5 },
    { cx: 50, cy: 90, radius: 5 },
  ];

  function update(data) {
    d3.select(".enter-update-exit-example")
      .selectAll("circle")
      .data(data)
      .join(
        (enter) => enter
            .append("circle")
            .attr("cx", (d) => d.cx)
            .attr("cy", (d) => d.cy)
            .attr("r", (d) => d.radius)
            .attr("fill", "steelblue"),
        (update) => update
            .transition()
            .duration(1000)
            .attr("cx", (d) => d.cx)
            .attr("cy", (d) => d.cy)
            .attr("r", (d) => d.radius)
            .attr("fill", "orange"),
        (exit) => exit
            .transition()
            .duration(1000)
            .attr("r", 0)
            .attr("fill", "red")
            .remove()
      );
  }

  setTimeout(() => update(dataState1), 0);
  setTimeout(() => update(dataState2), 2000);
  setTimeout(() => update(dataState3), 4000);
</script>
```

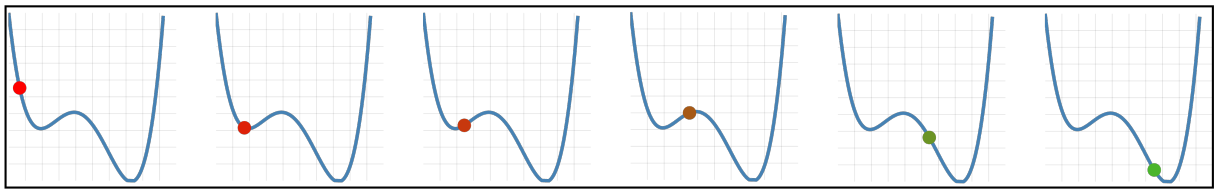


Übergänge

Im obigen Beispiel wird ebenfalls eine wichtige Stärke von D3.js demonstriert, die Möglichkeit, zwischen Datenzuständen zu transitionieren (vgl. [14]). Es werden zwischen Datenzuständen mittels `.transition()` weitere intermediäre Zustände erzeugt, die eine flüssige Animation zwischen den Zuständen ermöglichen. Dabei gibt `.duration()` die Dauer der Transition in Millisekunden an. Im obigen Beispiel wird die Position, Größe und Farbe von einem Datenzustand zum Nächsten mittels weitere intermediäre Zustände erweitert, sodass eine flüssige Animation zwischen den Zuständen entsteht. In vielen Situationen ist der Übergang von einem Zustand zum Nächsten mittels der Standard-Transition von D3 ausreichend und erlaubt viele Attribut- und Styleübergänge. Für komplexere Animationen und Übergänge können jedoch auch eigene Transitionen definiert werden. Soll beispielsweise ein Punkt von einer Koordinate
`state1 = [{ x: 1, y: f(1) }]` zu einer neuen Koordinate
`state2 = [{ x: 2, y: f(2) }]`
entlang eines Polynoms $f(x)$ bewegt werden, ist D3's `.attrTween()` Methode hilfreich. Diese Methode erwartet eine Funktion, die eine Interpolationsfunktion zurückgibt, die den Wert des Attributs für jeden Zeitpunkt $t \in [0, 1]$ zwischen dem Start- und Endwert interpoliert. Im Beispiel wird mittels `d3.interpolate()` eine Interpolationsfunktion erstellt, welche intermediäre Werte zwischen der Anfangskoordinate und der Endkoordinate erzeugt. Diese wird dann für jeden Zeitwert t aufgerufen und der Wert des Attributs entsprechend gesetzt:

`xScale`, `yScale` sind Skalenfunktionen, die die Koordinaten in SVG-Koordinaten umrechnen.

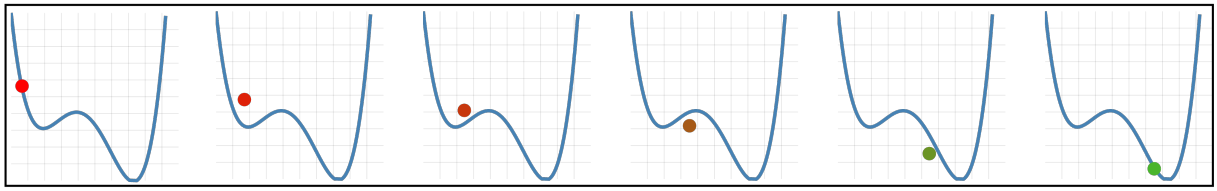
```
const dataPoint = svg
  .append("circle")
  .attr("cx", xScale(state1[0].x))
  .attr("cy", yScale(state1[0].y))
  .attr("r", 4)
  .attr("fill", "red");

dataPoint
  .transition()
  .duration(1000)
  .attrTween("cx", () => {
    const interpolateX = d3.interpolate(state1[0].x, state2[0].x);
    return function (t) {
      return xScale(interpolateX(t));
    };
  })
  .attrTween("cy", () => {
    const interpolateX = d3.interpolate(state1[0].x, state2[0].x);
    return function (t) {
      return yScale(f(interpolateX(t)));
    };
  })
  .attr("fill", "green");
```



Custom

Der Koordinatenpunkt würde mit der Standard-Transition von D3 von der Anfangs- zur Endkoordinate linear bewegt werden, anstatt entlang des Polynoms "zu rutschen":

```
const dataPoint = svg
  .append("circle")
  .attr("cx", xScale(state1[0].x))
  .attr("cy", yScale(state1[0].y))
  .attr("r", 4)
  .attr("fill", "red");

dataPoint
  .transition()
  .duration(1000)
  .attr("cx", xScale(state2[0].x))
  .attr("cy", yScale(state2[0].y))
  .attr("fill", "green");
```

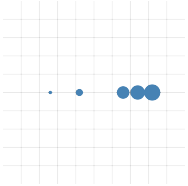


Standard

Skalen und Achsen

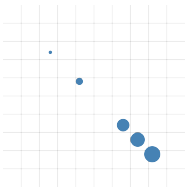
In vielen Situationen ist es vorteilhaft, seine Daten auf einen bestimmten Bereich zu skalieren. Typischerweise definiert man dafür eine Skala mit `d3.scaleLinear()` und gibt mittels `domain` den Wertebereich der Daten und mittels `range` die Positionen im SVG-Element an, auf die die Daten abgebildet werden sollen (vgl. [12]). Im folgenden Beispiel wird eine Skala erstellt, die einen Wertebereich von $[0, 10]$ definiert, welche im SVG-Element auf den Bereich $[10, 90]$ abgebildet wird. `xScale(0)` würde also 10 zurückgeben und `xScale(10)` würde 90 zurückgeben, sodass es als eine Art Umrechnungsfunktion zwischen Daten und Positionen im SVG-Element fungiert:

```
<svg class="scales-example">
</svg>

<script>
  const xScale = d3.scaleLinear()
                   .domain([0, 10]).range([10, 90]);

  d3.select(".scales-example")
    .selectAll("circle")
    .data([2, 4, 7, 8, 9])
    .join("circle")
    .attr("cx", (d) => xScale(d))
    .attr("cy", (d) => yScale(d))
    .attr("r", (d) => d / 2);
</script>
```



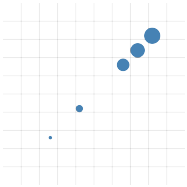
Auf die gleiche Weise können auch Skalen für die y-Achse erstellt werden:

```
<svg class="scales-example">
</svg>

<script>
  const xScale = d3.scaleLinear()
                   .domain([0, 10]).range([10, 90]);

  const yScale = d3.scaleLinear()
                    .domain([0, 10]).range([10, 90]);

  d3.select(".scales-example")
    .selectAll("circle")
    .data([2, 4, 7, 8, 9])
    .join("circle")
    .attr("cx", (d) => xScale(d))
    .attr("cy", 50)
    .attr("r", (d) => d / 2);
</script>
```



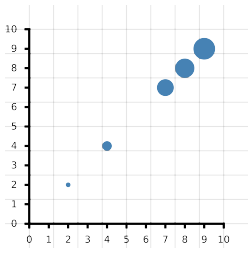
Hier ist allerdings zu beachten, dass die y-Skala in SVG-Elementen von oben nach unten verläuft, sodass die y-Skala von 10 (kleinster Wert) bis 90 (größter Wert) verläuft. Um die Skala von unten nach oben zu verlaufen, muss die Skala invertiert werden:

```
  ...
  const yScale = d3.scaleLinear()
                   .domain([0, 10]).range([90, 10]);
  ...
```



Ergänzend zu Skalen bietet D3 auch die Möglichkeit, Achsen zu erstellen, die die Skalen visualisieren. Typischerweise wird dabei die x-Achse mittels `d3.axisBottom()` und die y-Achse mittels `d3.axisLeft()` erstellt. Die Achsen können dann mittels `.call()` an das SVG-Element angehängt werden und mittels Funktionen wie `.tickSize()`, `.tickValues()`, `.tickFormat()`, `.tickPadding()` etc. konfiguriert werden.

```
<script>
  ...
  d3.select(".scales-example")
    .append("g")
    .attr("transform", "translate(0,90)")
    .call(xAxis);

  d3.select(".scales-example")
    .append("g")
    .attr("transform", "translate(10,0)")
    .call(yAxis);
</script>
```



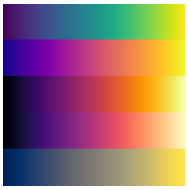
Weitere Funktionalitäten

Die obigen Beispiele zeigen nur einen kleinen Ausschnitt der Funktionalitäten von D3.js, die Bibliothek bietet eine Vielzahl von weiteren Funktionen und Methoden, die für die Erstellung von interaktiven Visualisierungen nützlich sind. In diesem Abschnitt sei zuletzt auf die Farbskalen von D3 hingewiesen, die im Webprojekt hauptsächlich in Verbindung mit Heatmaps und Konturplots als Ersatz für dreidimensionale Visualisierungen verwendet wurden (vgl. [4]). Dabei kommen

überwiegend sequentielle Farbskalen zum Einsatz, welche einen Zahlenwert $t \in \{0, 1\}$ auf einen Farbwert abbilden. Darunter die typischen wissenschaftlichen Farbskalen wie Viridis, Plasma, Inferno, Magma und Cividis:

abzubildene Datenwerte müssen auf den Bereich $[0, 1]$ normalisiert werden

```
d3.interpolateViridis(t)
d3.interpolatePlasma(t)
d3.interpolateInferno(t)
d3.interpolateMagma(t)
d3.interpolateCividis(t)
```



3.3. JavaScript Frontend Bibliothek React

Das Webprojekt basiert auf dem JavaScript-Framework React. Als komponentenbasiertes Framework ermöglicht React die Strukturierung von Benutzeroberflächen in wiederverwendbare Komponenten, die jeweils ihren eigenen Zustand verwalten können. Andere JavaScript-Frameworks wie Angular, Vue.js oder Svelte bieten ähnliche Funktionalitäten, sodass die Wahl des Frameworks eher eine Frage der Präferenz ist. Die im Projekt verwendenten Konzepten basieren zudem im Wesentlichen auf den Webstandards HTML, CSS, JavaScript und SVG, welche lediglich durch D3.js ergänzt werden. So sind beispielsweise die obigen Einführungen in D3.js und SVG mittels reinem JavaScript und HTML beschrieben.
Die Entscheidung für React basiert daher vor allem auf seiner Popularität und dem dadurch bedingten einfachen Zugang zu Ressourcen und einer großen Entwickler-Community sowie der einfachen Handhabung von Komponentenstrukturen.

3.3.1. React - Projektaufbau

Ein React-Projekt hat dabei immer eine bestimmte Struktur, die sich an bestimmte Konventionen hält. Dafür gibt es drei wesentliche Dateien, die für die Strukturierung und Funktionalität der Anwendung verantwortlich sind (`index.html`, `index.js` und `App.js`):

```
react-app
├── public
│   └── index.html
├── src
│   ├── App.js
│   └── index.js
└── package.json
```

Die `index.html` Datei bildet das Grundgerüst einer React-Anwendung. Hier wird ein einzelnes HTML-Element (`root` Element) als Container festgelegt, in den React die gesamte Applikation einbettet. Über die `index.js` wird die Hauptkomponente `App.js` gestartet und in diesen Container gerendert. Dabei dient die `App.js` als Ausgangspunkt für alle weiteren Komponenten und definiert die grundlegende Struktur der Benutzeroberfläche. React nutzt dabei das Konzept eines virtuellen DOM. Änderungen an der Benutzeroberfläche werden damit effizient berechnet, um nur die notwendigen Aktualisierungen im eigentlichen Browser-DOM vorzunehmen.

3.3.2. Funktionskomponenten

Die im Projekt verwendeten React-Komponenten sind sogenannte Funktionskomponenten, welche Funktionen sind, die ein React-Element zurückgeben. Üblicherweise wird eine Visualisierungskomponente definiert welche ein SVG-Element zurückgibt. Dieses SVG-Element wird dann innerhalb der Komponente mittels D3.js manipuliert.

```
// Importe
...
export default function VisualComponent() {
  const svgRef = useRef();
  ...
  // Funktionen zur Manipulation des SVG-Elements
  ...
  return (
      <svg ref={svgRef} viewBox="0 0 100 100" />
  );
}
```

Hooks

Für die Manipulation wird der `useRef`-Hook verwendet, um eine Referenz auf das SVG-Element zu erhalten um somit auch Zugriff. Mit den React Hooks `useState` und `useEffect` kann man den Zustand einer Komponente verwalten und auf Änderungen reagieren.
Der `useState`-Hook dient dazu, einen Zustand innerhalb der Komponente zu erstellen und zu verwalten. Man definiert eine Zustandsvariable und eine Funktion, um diesen Zustand zu aktualisieren. Diese werden von `useState` zurückgegeben und können mit einem Anfangswert initialisiert werden:

```
const [state, setState] = useState(defaultValue);
```

Der `useEffect`-Hook ermöglicht es wiederum, auf Änderungen des Zustands zu reagieren und entsprechende Aktionen auszuführen. Dafür wird eine Funktion als Argument entgegengenommen, die bei jeder Änderung des Zustands aufgerufen wird, wobei man im sogenannten "Dependency Array" festgelegt wird, auf welche Zustandsvariablen und dessen Änderungen die Funktion reagieren soll.

```
useEffect(() => { ...  }, [dependencies]);
```

Wenn keine Abhängigkeiten angegeben werden, wird die Funktion nur einmal beim ersten Rendern der Komponente ausgeführt. Soll beispielsweise die Farbe eines Rechtecks im SVG-Element auf Knopfdruck geändert werden könnte dies wie folgt implementiert werden:

```
import { useState, useRef, useEffect } from "react";
import * as d3 from "d3";

export default function VisualComponent() {
  const svgRef = useRef();
  const [color, setColor] = useState("steelblue");

  // Wenn die Komponente anfaenglich gerendert wird
  useEffect(() => {
    d3.select(svgRef.current)
      .append("rect")
      .attr("width", 100)
      .attr("height", 100)
      .attr("fill", color);
```

```
15    }, []);
16
17    // Bei Aenderung der Farbe wird die Farbe des Rechtecks geaendert
18    useEffect(() => {
19      d3.select(svgRef.current)
20        .select("rect")
21        .attr("fill", color);
22    }, [color]);
23
24    return (
25      <>
26        <svg ref={svgRef} viewBox="0 0 100 100" />
27        <button onClick={() => setColor(
28          `rgb(${Math.random() * 255},
29               ${Math.random() * 255},
30               ${Math.random() * 255})`
31          )}>
32          Klicke mich!
33        </button>
34      </>
35    );
36  }
```

Anfangs wird ein Rechteck mit der Farbe `color = "steelblue"` gerendert (Z. 9-15), das heißt, dass die Farbe durch den initialen Zustand in `color` definiert wird. Durch Klicken auf den Button wird die Farbe des Rechtecks zufällig geändert (Z. 18-22), da auf die Zuständsänderung von `color` (Z. 27-31) reagiert wird.

4. Konzept

4.1. Konzepte des maschinellen Lernens

Wenngleich der Fokus dieser schriftlichen Ausarbeitung auf der technischen Umsetzung und Vermittlung der Konzepte im Projekt liegt, soll dennoch im Folgenden ein kurzer Überblick über die vermittelten Kernkonzepte im Webprojekt gegeben werden. Die folgenden Abschnitte sind dabei als eine Art "Roter Faden" zu verstehen, der die zentralen Konzepte des maschinellen Lernens in seiner didaktischen Reihenfolge, wie sie im Webprojekt vermittelt werden aufzeigt und nicht als vollständige Einführung in das Thema verstanden werden sollte. Letzteres ist zumindest in Teilen Anspruch des Webprojekts, welches die Konzepte in einer interaktiven und visuellen Weise vermittelt.
Die vermittelten Konzepte umfassen neben einer Motivation des maschinellen Lernens die Teilgebiete der Wahrscheinlichkeitsdichteschätzung, Klassifikations- und Regressionsansätze sowie die Grundlagen des Deep Learnings. Als Referenz für die folgenden Abschnitte und im Webprojekt selbst wird das populäre Lehrbuch "Pattern Recognition and Machine Learning" von Christopher M. Bishop [2] verwendet, welches als eine der Standardreferenzen für das Thema des maschinellen Lernens gilt.

Motivation des maschinellen Lernens

Das Konzept des maschinellen Lernens wird damit motiviert, eine Aufgabe wie das Klassifizieren von handgeschriebenen Ziffern zu automatisieren. Eine Aufgabe, die für Menschen trivial ist, jedoch für Computer durch eine rein regelbasierte Implementierung, aufgrund der Vielzahl an möglichen Variationen und Schreibstilen, nicht zu bewältigen ist.

Dies begründet einen Ansatz des maschinellen Lernens, welcher darauf abzielt, auf Basis eines Trainingsdatensatzes

$$X = \{\vec{x}_1, \ldots, \vec{x}_N\} \text{ (im Beispiel: Vektoren von Pixelwerten)}$$

und im Fall eines überwachten Lernens zugehörigen Labels

$$T = \{t_1, \ldots, t_N\} \text{ (im Beispiel: Ziffern von 0-9)}$$

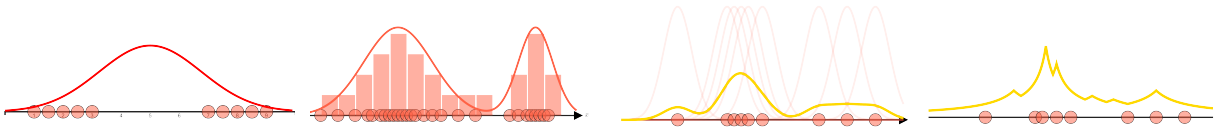
eine Modellfunktion $y(\vec{x})$ zu erlernen, welches die Labels t_n für neue Datenpunkte $\vec{x}_{test}$ vorhersagen kann. Wie diese Modellfunktion $y(\vec{x})$ im Trainingsprozess erlernt wird und dessen Fähigkeit auf neue unbekannte Daten zu generalisieren wird als Kernfrage im Webprojekt motiviert.

Wahrscheinlichkeitsdichteschätzung

Als dafür einführende Methode wird das Konzept der Klassifizierung mittels Bayes-Theorem gewählt:

$$y(\vec{x}) = p(\mathcal{C}_k \mid \vec{x}) = \frac{p(\vec{x} \mid \mathcal{C}_k)p(\mathcal{C}_k)}{p(\vec{x})} = \frac{p(\vec{x} \mid \mathcal{C}_k)p(\mathcal{C}_k)}{\sum_j p(x \mid \mathcal{C}_j)p(\mathcal{C}_j)}$$

Es wird die Frage untersucht, welcher Klasse $\mathcal{C}_k$ ein Datenpunkt $\vec{x}$ am wahrscheinlichsten zugehörig ist. Dies wird durch die a-posteriori-Wahrscheinlichkeit $p(\mathcal{C}_k \mid \vec{x})$ beschrieben, welche durch die Likelihood $p(\vec{x} \mid \mathcal{C}_k)$ und der a-priori-Wahrscheinlichkeit $p(\mathcal{C}_k)$ definiert ist. Die dafür benötigten Wahrscheinlichkeit $p(\vec{x} \mid \mathcal{C}_k)$ und $p(\mathcal{C}_k)$, sowie deren Berechnung werden im Webprojekt via parametrischen und nicht-parametrischen Methoden aufgezeigt.
Für die Erschätzung dieser Wahrscheinlichkeiten werden in den parametrischen Methoden gewisse Grundannahmen über die Verteilung getroffen. Basierend auf diesen können die Wahrscheinlichkeiten direkt mittels Methoden wie der Maximum-Likelihood-Schätzung berechnet werden können.
In den nicht-parametrischen Methoden wird hingegen keine Annahme über die Verteilung getroffen und die Wahrscheinlichkeiten direkt aus den Trainingsdaten mittels Techniken wie dem Histogramm-Schätzer, Kernel-Dichteschätzer oder Nächster-Nachbar-Schätzer berechnet:



`Maximum Likelihood` | `Histogramm-Schätzer` | `Kerneldichte-Schätzer` | `Nächster-Nachbar-Schätzer`

Klassifikations- und Regressionsansätze

Neben einem rein probabilistischen Ansatz begründet im Bayes-Theorem, wird ebenfalls auf ein Gerüst der linearen Klassifizierer/Diskriminierer eingegangen welches die Entscheidungsgrenze $y(\vec{x})$ ohne Wahrscheinlichkeiten direkt mittels einer linearen Funktion $y(\vec{x}) = \vec{w}^T\vec{x}$ definiert. Zentral ist hierbei eine sogenannte Fehlerfunktion, welche die Distanz zwischen den Modellvorhersagen $y(\vec{x_n})$ und dem tatsächlichen Label t_n misst. Dies passiert in der Regel mittels der quadratischen Fehlerfunktion, welche die Summe der quadrierten Differenzen zwischen Vorhersage beschreibt und minimiert:

$$\underset{\vec{w}}{\operatorname{argmin}}\, E(\vec{w}) = \underset{\vec{w}}{\operatorname{argmin}} \left(\frac{1}{2} \sum_{n=1}^{N} (y(\vec{x}_n) - t_n)^2 \right) = (X^\top X)^{-1} X^\top \vec{t} = X^\dagger \vec{t}$$

Die resultierende Lösung in geschlossener Form $\vec{w}^* = X^\dagger\vec{t}$, welche die Fehlerfunktion minimiert, wird als "Least-Squares" Lösung bezeichnet und führt zur linearen Entscheidungsgrenze $y(\vec{x}) = \vec{w}^T\vec{x} = \vec{t}^\top(X^\dagger)^\top\vec{x}$. Das resultierende Modell ($\rightarrow$ `(1)`) wird aufgrund Schwächen wie der Sensibilität gegenüber Ausreißern ($\rightarrow$ `(2)`) und der Unfähigkeit nichtlineare Entscheidungsgrenzen zu modellieren ($\rightarrow$ `(3)`), stückweise erweitert:

(1) $\underset{\vec{w}}{\text{argmin}} \left(\frac{1}{2} \sum_{n=1}^{N} (y(\vec{x}_n) - t_n)^2 \right)$ Least Square (Anfällig gegenüber Ausreißern)

(2) $\underset{\vec{w}}{\text{argmin}} \left(\frac{1}{2} \sum_{n=1}^{N} (g(y(\vec{x}_n)) - t_n)^2 \right)$ Least Square mit Aktivierungsfunktion $g(\cdot)$ (Robust gegenüber Ausreißern)

(3) $\underset{\vec{w}}{\text{argmin}} \left(\frac{1}{2} \sum_{n=1}^{N} (g(y(\vec{\phi}(\vec{x}_n))) - t_n)^2 \right)$ Least Square mit Basisfunktionen $\vec{\phi}(\cdot)$ (Ermöglicht nichtlineare Entscheidungsgrenzen)

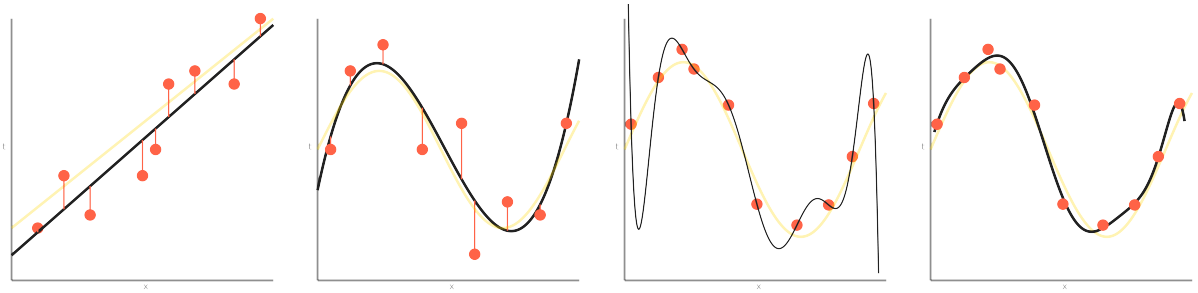
(1) (2) (3)

Neben dem grundlegenden Ansatz der Klassifikation mittels linearer Diskriminanten, bei dem versucht wird, Eingaben in vordefinierten Kategorien $\mathcal{C}_k$ einzuordnen, wird ebenfalls auf den Regressionsansatz eingegangen, der versucht, die Beziehung zwischen Eingabevariablen und Ausgabe zu modellieren, um Werte innerhalb eines kontinuierlichen Bereichs vorherzusagen. Das bedeutet, dass anstatt einer Entscheidungsgrenze nun eine Funktion an die Datenpunkte angepasst wird, sodass der Unterschied zwischen den vorhergesagten und den tatsächlichen Werten minimiert wird. Diese angepasste Funktion kann dann zur Vorhersage neuer Werte verwendet werden. Ebenso wie bei der Klassifikation wird das Modell mittels der quadratischen Fehlerfunktion $E(\vec{w}) = \frac{1}{2} \sum_{n=1}^{N} (y(\vec{x}_n) - t_n)^2$ motiviert, nur dass $t_n \in \mathbb{R}$ anstatt z.B. $t_n \in \{0, 1\}$. Ähnlich zum Klassifikationsansatz wird das Standardgerüst der linearen Regression stückweise erweitert um (→ (2)) ebenfalls nicht-lineare Beziehungen modellieren zu können. Daraus resultierende Phänomene (→ (3)) wie das "Overfitting" werden erklärt und in seiner Ursache hinterfragt, was letztlich zu geeigneten Techniken wie dem (→ (4)) Regularisieren der Modellkomplexität führt:

(1) $\underset{\vec{w}}{\text{argmin}} \left(\frac{1}{2} \sum_{n=1}^{N} (y(\vec{x}_n) - t_n)^2 \right)$ "Least Square"-Regression (Nur lineare Modelle)

(2), (3) $\underset{\vec{w}}{\text{argmin}} \left(\frac{1}{2} \sum_{n=1}^{N} (y(\vec{\phi}(\vec{x}_n)) - t_n)^2 \right)$ "Least Square"-Regression erweitert um Basisfunktionen $\vec{\phi}(\cdot)$ (Ermöglicht nichtlineare Beziehungen, kann jedoch zu Overfitting führen)

(4) $\underset{\vec{w}}{\text{argmin}} \left(\frac{1}{2} \sum_{n=1}^{N} (y(\vec{\phi}(\vec{x}_n)) - t_n)^2 + \lambda \frac{1}{2} \|\vec{w}\|^2 \right)$ "Ridge"-Regression (Verhindert Overfitting durch das "Bestrafen" zu großer Werte von w_i)



(1) (2) (3) (4)

Als letzte Klassifikationsmethode wird die logistische Regression eingeführt, welche den probabilistischen Ansatz des Bayes-Theorems mit dem Modell der linearen Klassifizierer verbindet. Dies resultiert in einer neuen Fehlerfunktion, dem sogenannten "Binary-Cross-Entropy" Fehler:

$$E(\vec{w}) = -\sum_{n=1}^{N} \left(t_n \ln(y(\vec{x}_n)) + (1 - t_n) \ln(1 - y(\vec{x}_n))\right)$$

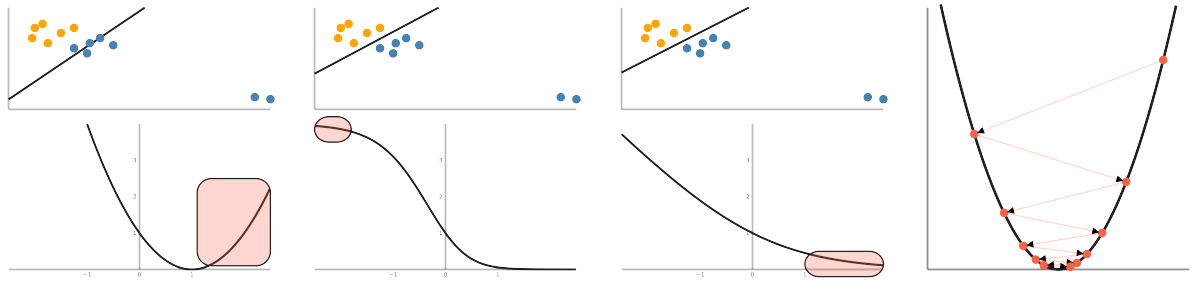
Neben der Motivation und Herleitung dieser Fehlerfunktion wird diese insbesondere in den Vergleich mit den bisher eingeführten Klassifikationsmethoden gestellt, indem Eigenschaften der verschiedenen Fehlerfunktionen wie beispielsweise Robustheit gegenüber Ausreißern oder "Optimierungsfreundlichkeit" diskutiert werden.
Ebenso wird auf die Optimierung der Fehlerfunktion mittels iterativer Gradientenabstiegsverfahren eingegangen (→ (4)), sowie die Notwendigkeit solcher Verfahren. Neben dem Gradientenabstiegsverfahren erster Ordnung wird ebenfalls auf Gradientenabstiegsverfahren zweiter Ordnung eingegangen und diese im Sinne von Begriffen wie Konvergenz und Berechnungsaufwand miteinander verglichen.

(1) $E(\vec{w}) = \frac{1}{2}\sum_{n=1}^{N}(y(\vec{x}_n) - t_n)^2$ "Least Square"

(2) $E(\vec{w}) = \frac{1}{2}\sum_{n=1}^{N}(g(y(\vec{x}_n)) - t_n)^2$ "Least Square" mit Aktivierungsfunktion

(3) $E(\vec{w}) = -\sum_{n=1}^{N} \left(t_n \ln(y(\vec{x}_n)) + (1 - t_n) \ln(1 - y(\vec{x}_n))\right)$ "Binary-Cross-Entropy"

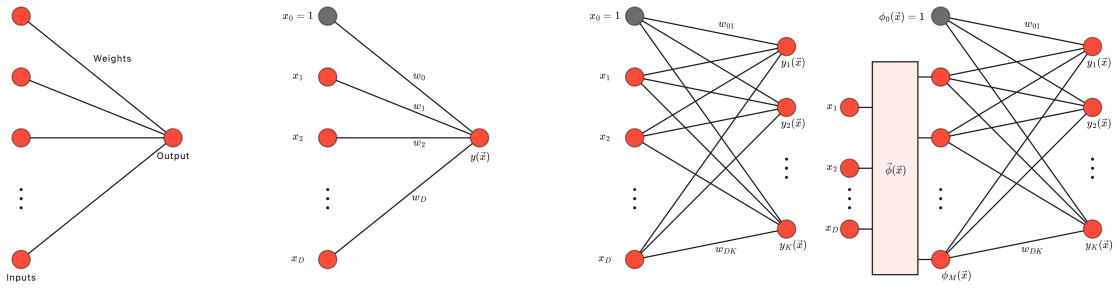
(4) $w^{(\tau+1)} = w^{(\tau)} + \eta \nabla E(\vec{w})$ Gradientenabstiegsverfahren (erster Ordnung)



(1) (2) (3) (4)

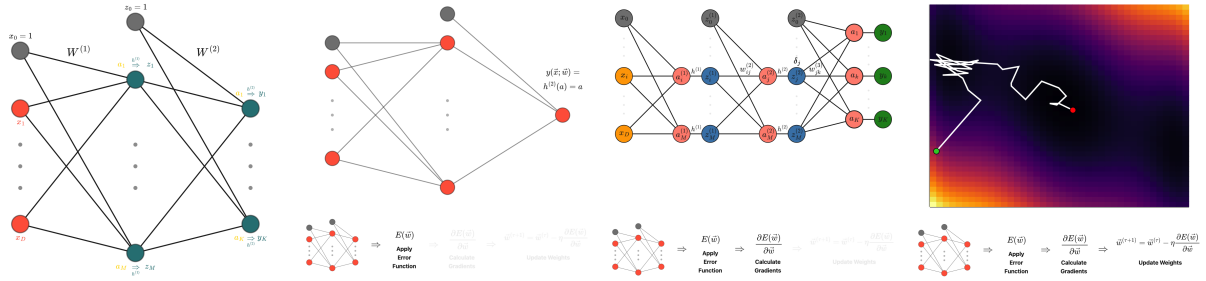
Deep Learning

Im letzten Abschnitt des Webprojekts wird auf die Grundlagen des Deep Learnings eingegangen. Dabei wird in einem ersten Schritt der Aufbau von Perzeptronen eingeführt, welche als ein-schichtige neuronale Netzwerke verstanden werden können. Die Struktur von Eingabe- und Ausgabeschicht sowie die Verbindungsgewichte zwischen den Neuronen werden als zentrale Bestandteile eines Perzeptrons eingeführt (→ (1)). Insbesondere wird auf den Zusammenhang zwischen Perzeptronen und dem Gerüst der "(generalized) linear discriminants" eingegangen und erkenntlich gemacht, dass bisher eingeführte Methoden wie beispielsweise Regression (→ (2)) oder mehrklassige Klassifikation (→ (3)) sowie deren Erweiterungen durch Basisfunktionen (→ (4)) alle durch Perzeptronen modelliert werden können.



(1) (2) (3) (4)

Die daraus resultierende Frage warum dann überhaupt Deep Learning benötigt wird, wird durch die Einführung von mehrschichtigen neuronalen Netzwerken motiviert. Diese können nichtlineare Beziehungen zwischen Eingabe- und Ausgabedaten modellieren, ohne dass der Modelldesigner spezifische Basisfunktionen $\vec{\phi}$ sowie deren Form vorgeben muss. Dies wird durch "Hidden Layers" erreicht, die den Prozess der fixen Feature-Modellierung flexibilisieren, indem sie die Basisfunktionen mit erlernbaren Gewichten parametrisieren und in das Modell integrieren ($\rightarrow$ (1)). Darüber hinaus wird diskutiert wie die Struktur des neuronalen Netzwerks aufzubauen ist, welche eng mit der zu lösenden Aufgabe verbunden ist. So wird beispielsweise aufgezeigt, dass für ein einfaches Regressionsproblem die Ausgabeschicht nur ein Neuron benötigt ($\rightarrow$ (2)), während für ein K-klassiges Klassifikationsproblem K Neuronen in der Ausgabeschicht benötigt werden. Aufgrund der erhöhten Komplexität und Anzahl an Parametern in mehrschichtigen neuronalen Netzwerken wird in zwei letzten Schritten auf die Notwendigkeit von speziellen Algorithmen wie dem "Backpropagation"-Algorithmus eingegangen, welcher die Ableitungen $\frac{\partial E(\vec{w})}{\partial \vec{w}}$ der Fehlerfunktion nach den Gewichten des neuronalen Netzwerks berechnet ($\rightarrow$ (3)), welche dann zur Aktualisierung der Gewichte in einem modifizierten Gradientenabstiegsverfahren $\vec{w}^{(\tau+1)} = \vec{w}^{(\tau)} + \eta \frac{\partial E(\vec{w})}{\partial \vec{w}}$ verwendet wird ($\rightarrow$ (4)). Diese letzten zwei Schritte bilden den Kern des Deep Learnings und sind essenziell für das Training von mehrschichtigen neuronalen Netzwerken. Im Gegensatz zu den bisher behandelten Themen erfordern sie jedoch trotz entsprechender Visualisierungen und schrittweiser Erklärungen eine erhöhte Aufmerksamkeit und Geduld, um ein tiefgehendes Verständnis zu erlangen. Aus diesem Grund wird im Webprojekt auf die Komplexität dieser Schritte hingewiesen.



(1) (2) (3) (4)

Auswahl der Themen

Die Wahl der Themen und die Reihenfolge, in der sie im Webprojekt präsentiert werden, sind nicht zufällig, sondern folgen einem didaktischen Ansatz. Dieser zielt darauf ab, verschiedene Arten und Ansätze darzustellen, wie Maschinen/Modelle auf Basis eines zugrunde liegenden Datensatzes lernen können, eigenständig Entscheidungen zu treffen. Das Modul der Wahrscheinlichkeitsdichteschätzung dient dabei als Einführung in einen reinen probabilistischen Ansatz. Entscheidungen werden hier auf Basis von Wahrscheinlichkeiten getroffen, die aus den Trainingsdaten (mittels parametrischer oder nicht-parametrischer Methoden) geschätzt werden. Das

Gerüst der linearen Klassifikatoren dient als Gegenstück, da hier Entscheidungen auf Basis einer linearen Funktion getroffen werden, ein rein analytischer Ansatz. Der Lernende soll hierbei erkennen, dass ein Modell aufgrund von Schwächen (wie der Sensibilität gegenüber Ausreißern oder der Unfähigkeit, nichtlineare Entscheidungsgrenzen zu modellieren) erweitert werden kann. Die dafür notwendigen Erweiterungen, wie Basisfunktionen oder Aktivierungsfunktionen, sind grundlegende Werkzeuge, die auch in anderen Modellen wie der (logistischen) Regression oder im Deep Learning verwendet werden. Im Regressionsansatz, bei dem die zugrunde liegende Aufgabe von der Klassifikation zur Vorhersage kontinuierlicher Werte wechselt, werden diese Werkzeuge ebenfalls genutzt, um nichtlineare Beziehungen zu modellieren. Ferner dient die logistische Regression als Mischung aus probabilistischem und linearem Ansatz, sodass der Lernende bereits vier verschiedene Ansätze kennengelernt hat, bevor er sich dem Deep Learning zuwendet. Der Themenbereich des Deep Learnings wird zuletzt behandelt, da er auf den vorherigen Themen aufbaut. Würde der Lernende zu früh mit dem Thema konfrontiert, könnten keine Verbindungen zu linearen Modellen und deren Erweiterungen hergestellt werden. Zudem wurden bereits zwischen Aufgabentypen wie Klassifikation und Regression unterschieden, interaktive Optimierungsmethoden wie das Gradientenabstiegsverfahren behandelt und die Notwendigkeit von Basisfunktionen sowie Aktivierungsfunktionen erklärt. Der Lernende soll erkennen, dass Deep Learning lediglich eine Erweiterung der bisherigen Themen darstellt und somit der Anschein eines "Black-Box"-Themas genommen wird. Die entscheidende Fähigkeit des mehrschichtigen neuronalen Netzwerks, geeignete Basisfunktionen selbst zu erlernen, wird dabei als Schlüsselkonzept hervorgehoben. Mehrmals zuvor im Kurs wird der Lernende dazu aufgefordert, Modellentscheidungen wie die Wahl einer passenden Basisfunktion in interaktiven Demos selbstständig zu treffen. So wird im Deep-Learning-Themenbereich verdeutlicht, dass solche Designentscheidungen nicht mehr vom Modellierer, sondern vom Modell selbst erlernt werden können. Zusammenfassend wird der Lernende mit verschiedenen Ansätzen vertraut gemacht, die auf unterschiedliche Weise Entscheidungen treffen und Muster in Daten erkennen können, sei es über Wahrscheinlichkeiten, lineare Funktionen oder einer Kombination aus beidem. Im Deep Learning finden viele der darin vorgestellten Konzepte Anwendung, sodass der Lernende die bisher erlernten Konzepte in einem neuen stark praxisrelevanten Kontext wiederfindet.

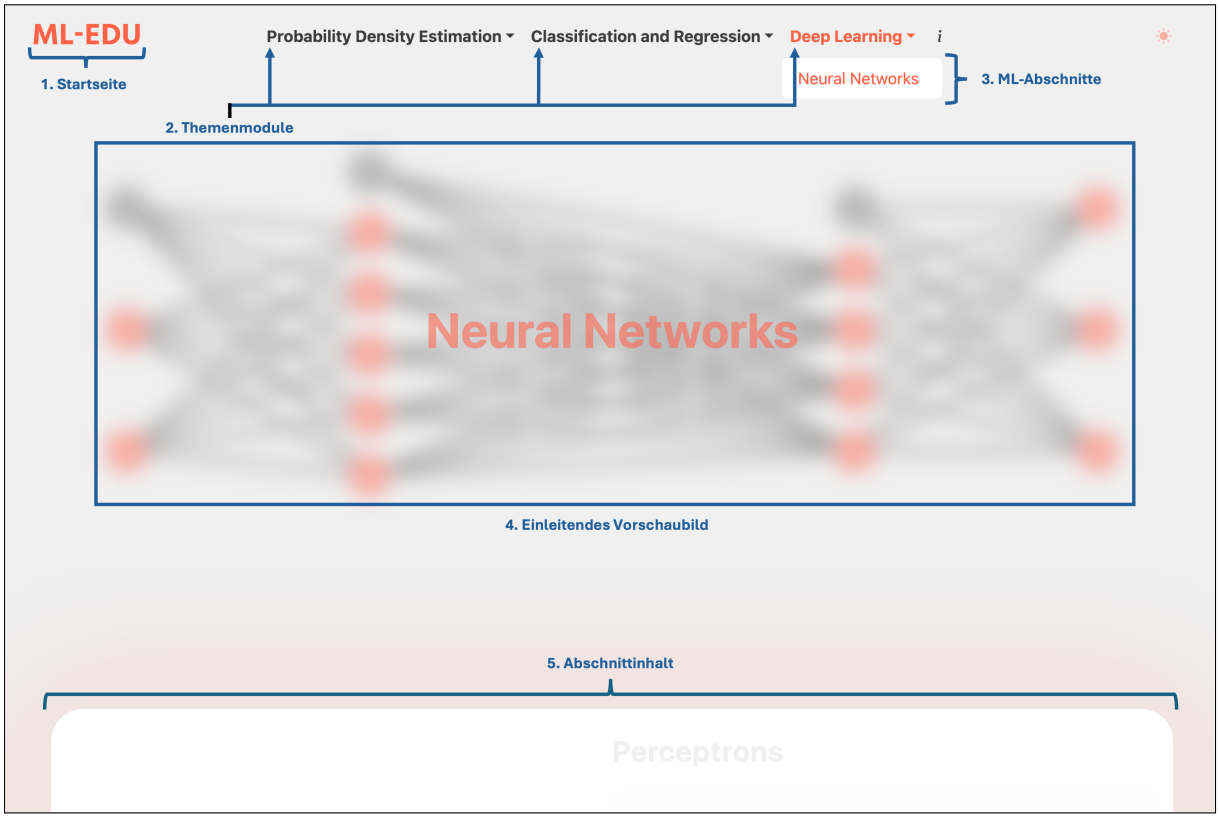
4.2. Modularisierung

Das Webprojekt mit dem Namen `ML-EDU` unterteilt die zu vermittelnden Themenbereiche in passende Module. Die Startseite bietet eine konzeptionelle Einführung und Motivation in das maschinelle Lernen und bereitet auf die Hauptmodule "Probability Density Estimation", "Classification and Regression" und "Deep Learning" vor. Diese Hauptmodule sind wiederum in Unterkapitel unterteilt, die die einzelnen Lehrkonzepte vermitteln. Die Module sind gemäß dem Abschnitt Konzepte des maschinellen Lernens in die folgenden Themenbereiche unterteilt:

- `ML-EDU` (Startseite - Einführung und Motivation)
- `Probability Density Estimation`
  - `Bayes Decision Theory`
  - `Parametric Methods`
  - `Non-Parametric Methods`
- `Classifcation and Regression`
  - `Linear Discriminants`
  - `Linear Regression`
  - `Logistic Regression`
- `Deep Learning`
  - `Neural Networks`

Seitenaufbau

Die folgende Abbildung beschreibt den generellen Aufbau der Webseitenstruktur:

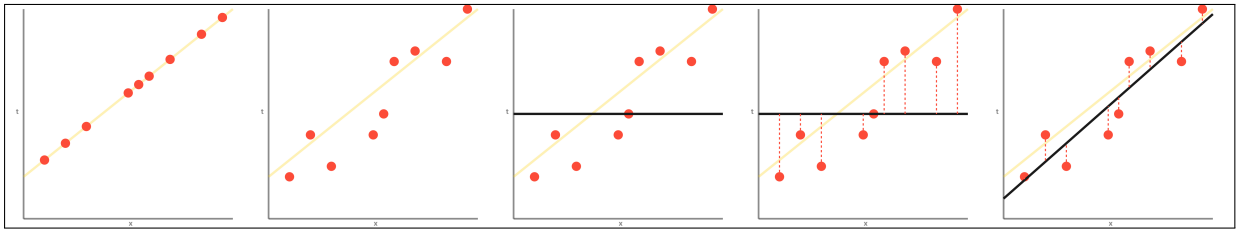


Ausschnitt von `ML-EDU` für den Abschnitt "Neural Networks"

Eine Navigationsleiste am oberen Bildschirmrand ermöglicht den Wechsel zwischen den Themenmodulen (2) und ihren Unterkapiteln (3). Die Startseite (1), welche die Einführung und Motivation enthält, ist über das Logo erreichbar. Jeder Abschnitt enthält neben dem eigentlichen Inhalt auch eine dynamische Einführungsvisualisierung (4), passend zum jeweiligen Thema. Der eigentliche Inhalt (5) der Webanwendung ML-EDU wird in Form von interaktiven "Scrollytelling"-Abschnitten und interaktiven Demonstrationen präsentiert, welche nun im Folgenden genauer beschrieben werden.

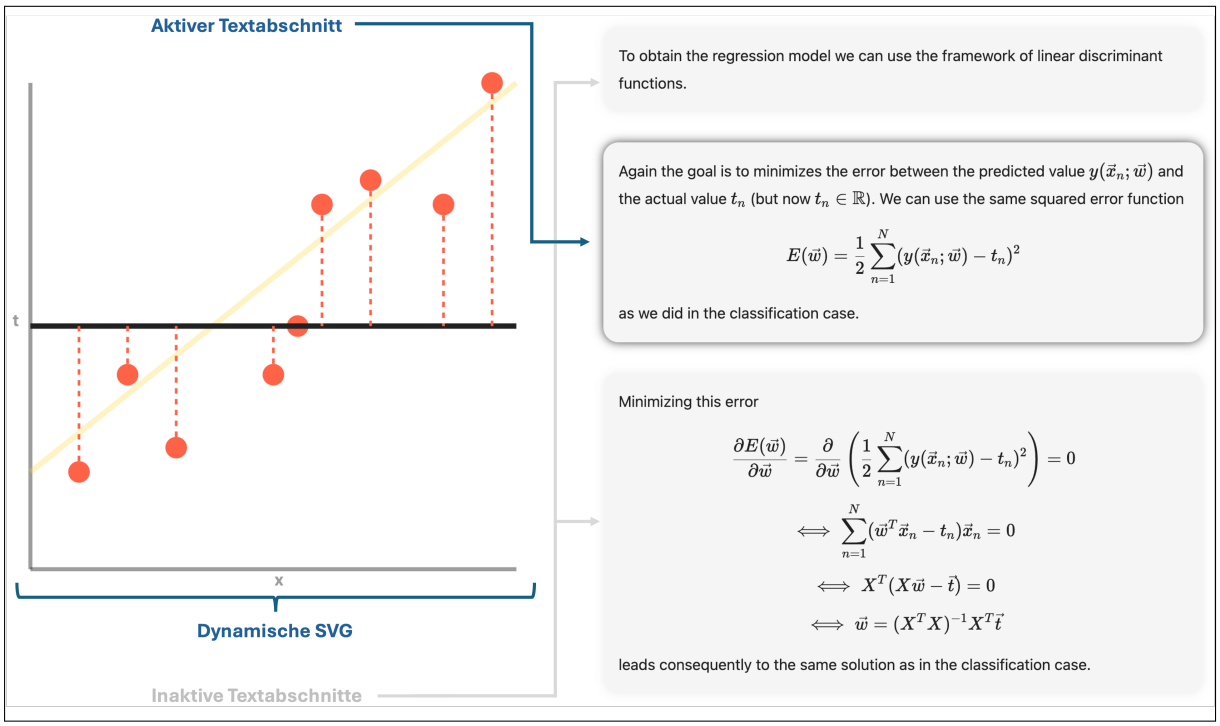
4.3. Interaktives "Scrollytelling"

Die wohl wichtigste Kernfrage des Bachelorprojekts beschäftigt sich damit, oftmals als trocken und schwer verständlich geltende Themenbereiche, wie sie unter anderem im maschinellen Lernen vorkommen, interaktiv und visuell ansprechend zu vermitteln. Häufig erfordert dies, die Themenbereiche auf einen Grundgedanken herunterzubrechen, der dann schrittweise erweitert wird, bis ein vollständiges Konzept entsteht, das vom Nutzer verstanden werden kann. Ein Ansatz, um dies zu erreichen, ist das sogenannte "Scrollytelling". Dabei handelt es sich um eine Art interaktive Geschichtserzählung, die durch das Scrollen der Seite durch den Nutzer gesteuert wird. Im Projekt wird dabei ein visuelles Element in Form einer SVG stückweise manipuliert:



Beispiel aus dem Themenbereich der linearen Regression

Jeder visueller Schritt wird dabei durch einen passenden thematischen Textabschnitt begleitet:

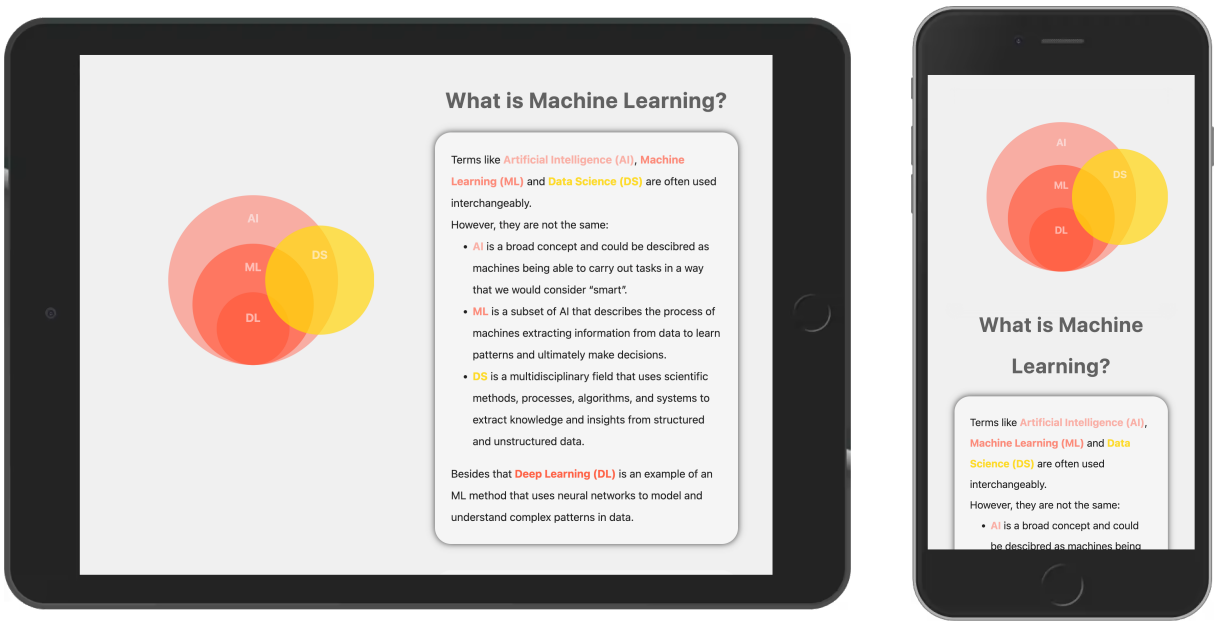


Das heißt, dass durch das Scrollen ein jeweiliger Abschnitt getriggert und aktiv wird. Dies

bewirkt wiederum eine entsprechende Änderung in der Visualisierung, sodass einer textuellen Erklärung immer eine visuelle Komponente gegenübersteht. Dem Nutzer soll damit nicht nur ein visueller Anker zum Thema gegeben werden, durch das Aufbauen einer visuellen Geschichte können komplexe Themenbereiche aufgebrochen und schrittweise vermittelt werden. Essenziell für eine effektive Umsetzung sind dabei eine klare Strukturierung der Inhalte und eine intuitive Navigation, um den Nutzer durch die Themenbereiche zu führen. Gut strukturierte und geplante Inhalte können dabei mittels animierter Übergänge den Geschichtsfluss unterstützen, da einzelne Themenabschnitte verknüpft werden und so eine konsistente Erzählung entsteht.

Platzierung von Text und Visualisierung

Für die Platzierung von Text und Visualisierung sind zwei verschiedene Varianten vertreten, welche sich nach der Displaybreite des Endgeräts richten. Für breite Displays, wie sie beispielsweise auf einem Desktop-Computer oder einem Tablet vorkommen, wird der Text neben der Visualisierung (→ (1)) platziert. Wenngleich dies dem wahrscheinlichsten Anwendungsfall entspricht, wurde auch eine alternative Platzierung für schmalere Displays berücksichtigt. Aufgrund der geringeren Displaybreite wird der Text unterhalb der Visualisierung platziert (→ (2)). Dies erlaubt eine bessere Lesbarkeit und verhindert einen überladenen Bildschirm.



(1) (2)

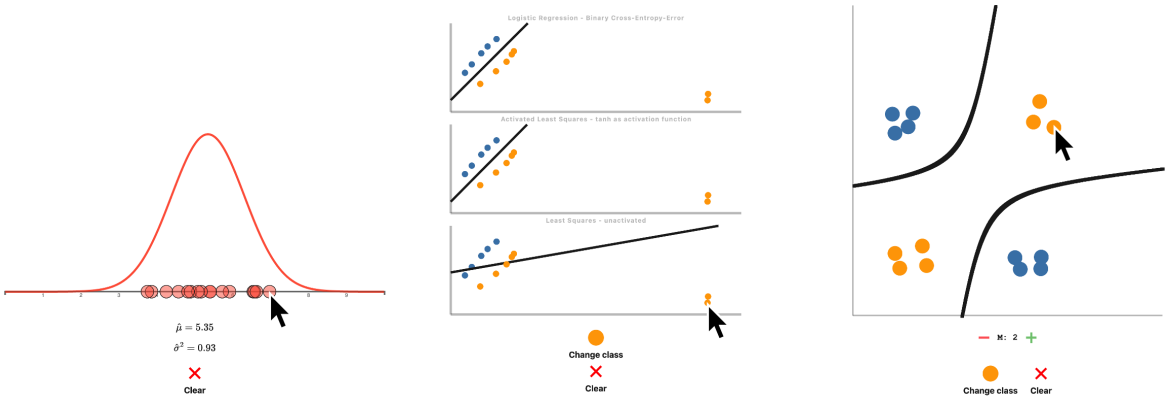
Im Allgemeinen wird bei Veränderung der Displaybreite seitens des Nutzers die Webseitenstruktur responsiv angepasst, das heißt, das Visualisierungselemente bei Anpassung der Displaybreite dynamisch skaliert werden.

4.4. Interaktive Demonstrationen

Ein "Scrollytelling"-Abschnitt wird in den meisten Fällen mit einer interaktiven Demonstration der dort vermittelten Themen abgeschlossen. Dort können Nutzer die zuvor erklärten Konzepte selbst ausprobieren und so ein besseres Verständnis für die Thematik entwickeln. Die Art der Demonstration kann dabei variieren, von interaktiven Visualisierungen von typischen ML-Algorithmen bis hin zu trainierten Modellen, die in Echtzeit auf Nutzereingaben reagieren. Im Folgenden soll eine Übersicht der verschiedenen Arten von interaktiven Demonstrationen, die im Projekt vertreten sind, gegeben werden:

Interaktive Visualisierungen mittels Klicken und Schieberegler

Die meisten interaktiven Demonstrationen basieren auf der Interaktion durch Klicken und Ziehen eines Schiebereglers. Klickbasierte Demonstrationen im Projekt dienen in erster Linie dazu, einen vorgestellten Algorithmus in Aktion zu zeigen. Typischerweise gibt der Nutzer dabei einen Datenpunkt auf die SVG-Fläche, um diesen zu einem Datensatz hinzuzufügen. Exemplarisch wird in den folgenden drei Abbildungen gezeigt, welche Auswirkungen das Hinzufügen eines Datenpunkts auf die Visualisierung hat.



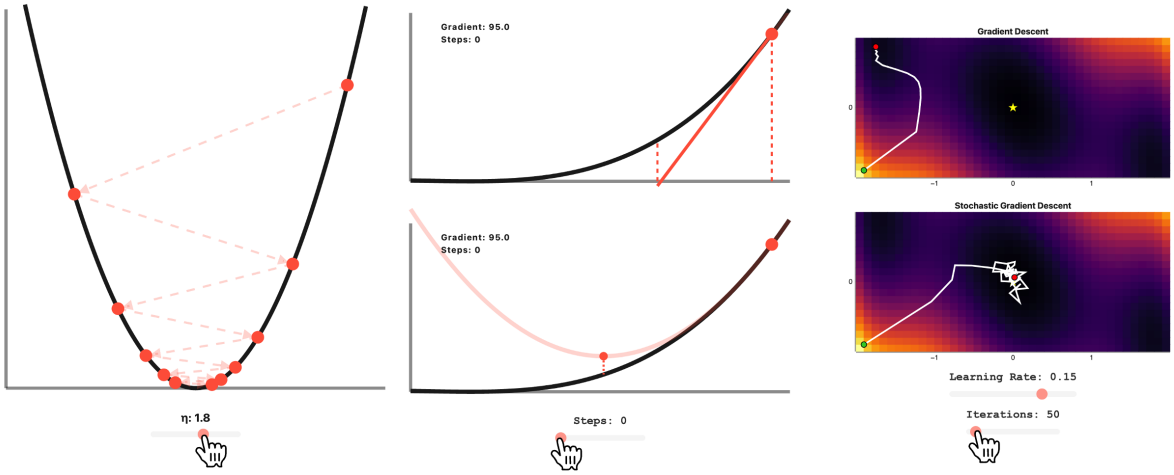
(1) (2) (3)

(1) **Maximum Likelihood Schätzung**
Das Hinzufügen von Datenpunkten bewirkt eine Schätzung von Mittelwert und Varianz der Datenpunkte gemäß der Maximum Likelihood Methode.

(2) **Vergleich von Klassifikationsalgorithmen**
Das Hinzufügen von klassenzugehörigen Datenpunkten bewirkt eine Berechnung einer linearen Entscheidungsgrenze zwischen den Klassen. Der Nutzer steuert neben der Position der Datenpunkte auch deren Klassenzugehörigkeit.

(3) **Klassifikationsalgorithmus mit nicht-linearen Entscheidungsgrenzen**
Das Hinzufügen von Datenpunkten bewirkt eine Berechnung einer nichtlinearen Entscheidungsgrenze zwischen den Klassen. Der Nutzer steuert neben der Position der Datenpunkte und deren Klassenzugehörigkeit ebenfalls Modellparameter, die die Form der Entscheidungsgrenze beeinflussen.

Ein weiteres Mittel zur Interaktion sind Schieberegler. Interaktive Demonstrationen dieser Art werden vorrangig dazu genutzt, gewisse Eigenschaften eines Algorithmus oder Modells vorzustellen.



(1) (2) (3)

(1) **Gradientabstiegverfahren erster Ordnung**
Das Verschieben des Schiebereglers bewirkt eine Anpassung der Lernrate des Gradientenabstiegsverfahrens, welche einen direkten Einfluss auf die Konvergenz des Optimierungsalgorithmus hat.

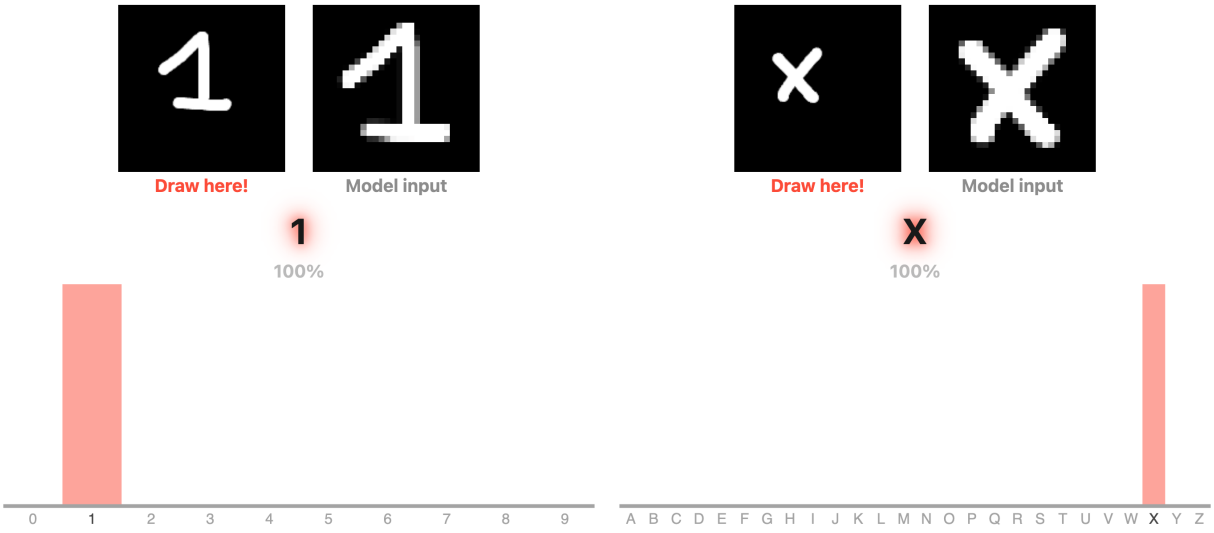
(2) **Vergleich der Gradientabstiegverfahren erster und zweiter Ordnung**
Das Verschieben des Schiebereglers bewirkt eine Anpassung der Anzahl an Iterationen, die benötigt werden, um ein lokales Minimum zu finden, und ermöglicht so den Vergleich der Konvergenzgeschwindigkeit der beiden Verfahren.

(3) **Vergleich von Gradientabstiegsverfahren mit stochastischen Gradientabstiegsverfahren**
Das Verschieben des Schiebereglers bewirkt eine Anpassung der Anzahl an Iterationen und der Lernrate beider Verfahren. Der Vergleich zeigt neben dem Abstiegsverhalten unter anderem auch die Fähigkeit des stochastischen Gradientenabstiegsverfahrens, lokalen Minima zu entkommen.

Interaktive Visualisierungen für praktische Anwendungen

Neben einer Vielzahl an interaktiven Demonstrationen von ML-Algorithmen, welche auf Klick- und Schiebereglerinteraktionen basieren, bietet das Projekt auch zwei interaktive Demonstrationen, bei denen Nutzer bisher vorgestellte Konzepte in einer praktischen Anwendung ausprobieren können. Konkret handelt es sich dabei um eine interaktive Zeichenanwendung für die Echtzeitklassifikation von handgezeichneten Zahlen (→ `(1)`) und Buchstaben (→ `(2)`). Die Anwendung, welche als Abschluss aller bisher vorgestellten Konzepte dienen soll, dient dazu, die Funktionsweise von neuronalen Netzen anhand eines einfachen Beispiels zu veranschaulichen und zu konkretisieren:

- Datenpunkte $\vec{x}_n$ und Klassen $\mathcal{C}_k$ werden direkt mit praktischen Werten assoziiert (Vektoren an Pixelwerten, Buchstaben und Zahlen als Klassen).
- Die Modellvorhersage $y(\vec{x}_n)$ wird vom Nutzer durch das Zeichnen eines Bildes bestimmt und in Form eines Buchstabens oder einer Zahl konkretisiert.
- Die praktische Qualität eines Modells kann durch die Genauigkeit der Klassifikation von handgezeichneten Zahlen und Buchstaben vom Nutzer selbst bewertet werden.



`(1)` `(2)`

5. Implementation

Die folgenden Abschnitte dienen dazu, die grundsätzliche Struktur der Webanwendung, Visualisierungstechniken sowie die Echtzeit-Klassifikationsalgorithmen im Webprojekt genauer zu beschreiben. An manchen Stellen wird auf eine zu detaillierte Code-Darstellung oder Dateibeschreibungen verzichtet, um den Lesefluss nicht zu unterbrechen. Die folgenden Abschnitte dienen vielmehr dazu, die wesentlichen Implementierungen und Techniken zu verstehen.

5.1. Projektstruktur

5.1.1. Komponentenstruktur

Statische Webanwendung: Keine serverseitige Logik oder Datenbankanbindung

Das Webprojekt ist eine rein statische Webanwendung und als React-Projekt umgesetzt. Wie bei React-Projekten üblich, besteht die Projektstruktur aus den Ordnern `public` und `src` sowie einigen Konfigurationsdateien. Im `public`-Ordner befindet sich die index.html-Datei, die als Einstiegspunkt für die Webanwendung dient und einen Container `root` bereitstellt, in dem die gesamte Anwendung gerendert wird. Außerdem gibt es einen Ordner `models`, in dem die trainierten Modelle, sowohl die Gewichte als auch die Modellarchitektur, gespeichert werden. Diese Modelle werden für die Deep-Learning-Interaktion zur Erkennung von handschriftlichen Zahlen und Buchstaben verwendet. Der `src`-Ordner enthält die eigentliche Webanwendung, welche in die Unterordner `assets`, `components`, `pages` und `utils` unterteilt ist. Der `assets`-Ordner enthält statische Dateien wie das Logo sowie Pixelwerte für handgeschriebene Zahlen und Buchstaben, die im Projekt für erklärende Visualisierungen verwendet werden. Der `components`-Ordner enthält die wiederverwendbaren Komponenten, wie beispielsweise die Navigationsleiste oder das Scrollytelling-Gerüst. Im `utils`-Ordner befinden sich Hilfsfunktionen, die in mehreren Komponenten verwendet werden. Beispiele für Funktionen in diesem Ordner sind solche, die LaTeX in Textkomponenten oder SVG integrieren sowie eine Funktion zum Abfragen und Wechseln des Helligkeitsmodus.

Helligkeitsmodus: "Dark Mode" und "Light Mode"

```
public
    models
    index.html
src
    assets
    components
    pages
    utils
    App.js
    App.scss
    index.js
    index.scss
package.json
package-lock.json
.gitignore
```

Der `pages`-Ordner enthält die Hauptseiten der Webanwendung, die in die Unterordner `home`, `probability-density-estimation`, `classification-and-regression` und `deep-learning` unterteilt sind. Diese beschreiben den thematischen modularen Aufbau der Webanwendung und enthalten die jeweiligen Unterkapitel der Themenbereiche.

Um dies an einem Beispiel zu verdeutlichen: Das Themenmodul "Probability Density Estimation" enthält die Unterkapitel `bayes-decision-theory`, `parametric-methods` und `non-parametric-methods`.
Für `non-parametric-methods` wird dieser Themenbereich in einzelne Abschnitte wie `histogram-density-estimation`, `kernel-density-estimation` und `knn-density-estimation` unterteilt und schließlich in einer Datei `non-parametric-methods.js` zusammengefasst. Das heißt, dass `non-parametric-methods.js` alle Unterabschnitte eines Themenbereichs zusammenfasst und schließlich die finale Seite für den Nutzer darstellt.

```
pages
├── home
├── probability-density-estimation
│   ├── bayes-decision-theory
│   ├── parametric-methods
│   └── non-parametric-methods
│       ├── histogram-density-estimation
│       ├── kernel-density-estimation
│       ├── knn-density-estimation
│       └── non-paremtric-methods.js
├── classification-and-regression
├── deep-learning
└── about
```

Jeder dieser Unterabschnitte (hier am Beispiel von `kernel-density-estimation`) beinhaltet die Ordner `textual` und `visual`, die die textuellen und visuellen Erklärungen gemäß dem Konzept des interaktiven Scrollytellings enthalten. Diese werden in der `kernel-density-estimation-scrollytelling.js`-Datei zusammengeführt, welche schließlich Text und Visualisierung synchronisiert darstellt. Falls thematisch passend, gibt es auch eine `interactive`-Komponente, die interaktive Demonstrationen zum jeweiligen Thema enthält.

```
kernel-density-estimation
├── interactive
│   ├── interactive.js
│   └── interactive.scss
├── textual
│   ├── text-1.js
│   ├── text-2.js
│   ├── text-3.js
│   └── ...
├── visual
│   ├── visual.js
│   └── visual.scss
└── kernel-density-estimation-scrollytelling.js
```

Seitenaufbau

Im Folgenden wird anhand der `non-parametric-methods.js`-Datei ein Beispiel für die Struktur einer Seite demonstriert.

```
...
import KernelDensityEstimationScrollytelling from
"./kernel-density-estimation/kernel-density-estimation-scrollytelling";
...
export default function NonParametricMethods() {
  return (
    <>
      <HistogramDensityEstimationScrollytelling/>
      <HistogramDensityEstimationInteractive/>

      <KernelDensityEstimationScrollytelling/>
      <KernelDensityEstimationInteractive/>

      <KNNDensityEstimationScrollytelling/>
      <KNNDensityEstimationInteractive/>
    </>
  );
}
```

Diese Datei importiert die entsprechenden `Scrollytelling`- und `Interactive`-Komponenten für die Abschnitte `Histogram Density Estimation`, `Kernel Density Estimation` und `KNN Density Estimation` und gibt diese in der `NonParametricMethods`-Komponente zurück.

5.1.2. Routen

Komponenten wie beispielsweise `NonParametricMethods` werden schließlich in der `App.js`-Datei eingebunden und über die `react-router-dom`-Bibliothek geroutet.

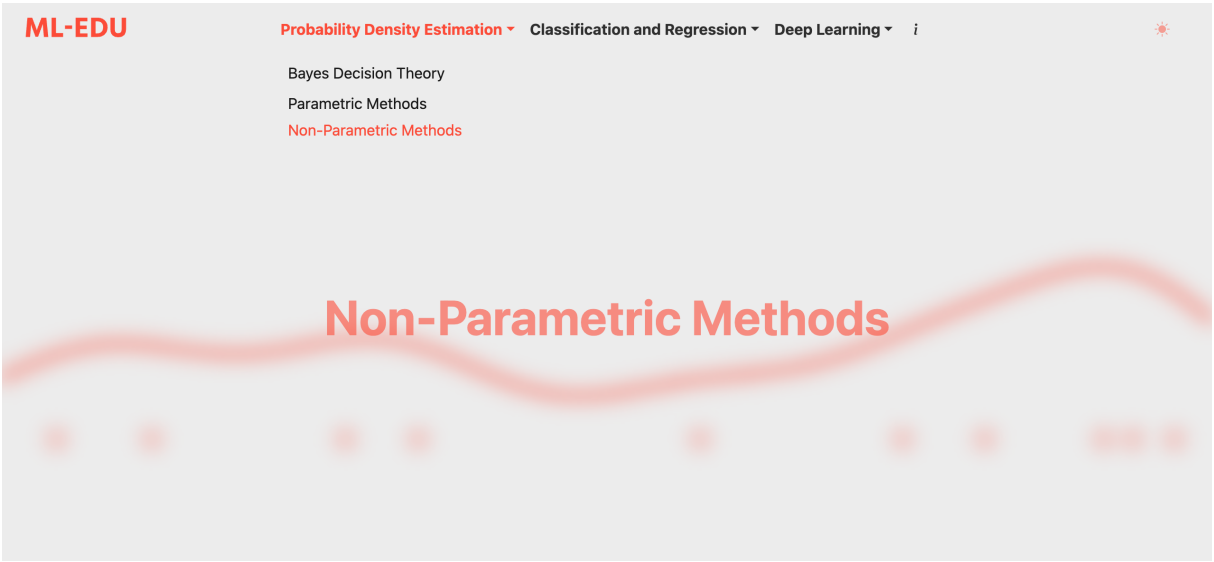
```
import { Routes, Route } from "react-router-dom";
// ...Restliche Imports

export default function App() {
  return (
    <>
      <Navbar/>
      <Routes>
        <Route path="/" element={<Home/>}/>
        ...
        <Route path="/probability-density-estimation">
          <Route path="bayes-decision-theory"
                 element={<BayesDecisionTheory/>}/>
          <Route path="parametric-methods"
                 element={<ParametricMethodsPage/>}/>
          <Route path="non-parametric-methods"
                 element={<NonParametricMethods/>}/>
        </Route>
        ...
      </Routes>
    </>
  );
}
```

Das Routing wird durch das `Routes`-Element gesteuert, das eine Sammlung von `Route`-Komponenten enthält. Jede dieser `Route`-Komponenten definiert einen bestimmten URL-Pfad und die zugehörige React-Komponente, die bei Aufruf dieses Pfades gerendert wird. Beispiel-

sweise wird die `NonParametricMethods`-Komponente auf der Route `"/probability-density-estimation/non-parametric-methods"` eingebunden.

```
...
import { NavLink } from "react-router-dom";
...
export default function Navbar() {
  return (
    <nav>
        ...
      <ul>
        ...
        <li>
          <NavLink
           to="/probability-density-estimation/non-parametric-methods"
           className={({ isActive })
           => (isActive ? "dropdown-item-active" : "dropdown-item")}
           >
            Non-Parametric Methods
          </NavLink>
        </li>
      </ul>
      ...
    </nav>
  );
}
```

Schließlich kann über die Navigationsleiste am oberen Rand der Webanwendung die verschiedenen Seiten aufgerufen werden. Die `NavLink`-Komponente aus der `react-router-dom`-Bibliothek ermöglicht es dabei, die URL-Pfade der verschiedenen Seiten zu definieren und auf diese zu verlinken. Zusätzlich erlaubt `NavLink` das automatische Hinzufügen einer speziellen CSS-Klasse für den aktuell aktiven Link, wodurch dem Benutzer die aktive Seite visuell hervorgehoben wird.

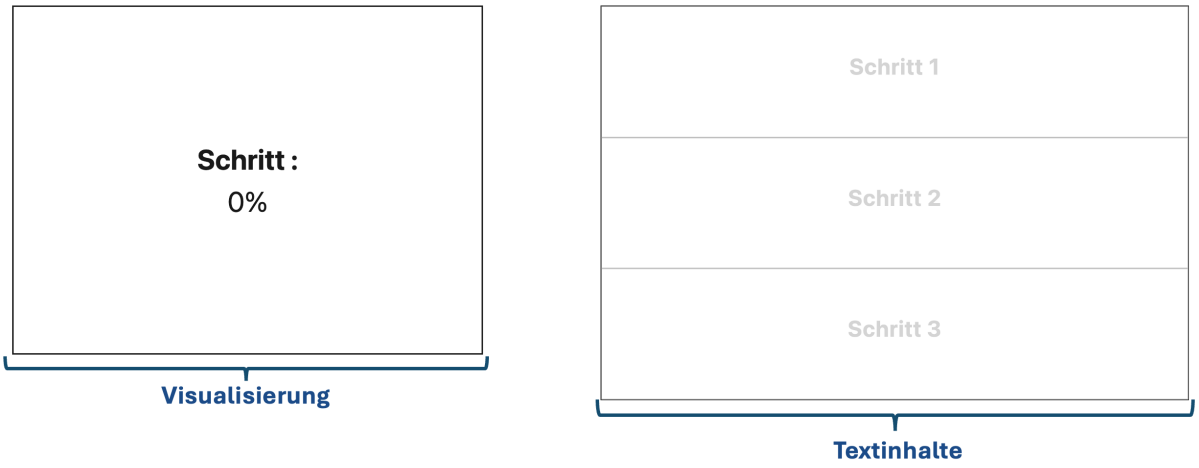


5.2. Scrollytelling Gerüst

Das Konzept des Scrollytellings ist eine essenzielle Komponente im Webprojekt und ist als grundlegend wiederverwendbare Komponente konzipiert. Im Folgenden wird diese stückweise beschrieben. Das Scrollytelling-Gerüst besteht aus zwei Hauptelementen, der Visualisierung und einer bestimmten Anzahl an Textinhalten (im folgenden Beispiel drei). Diese werden in einer `scrollytelling.js`-Datei zusammengeführt und innerhalb eines Containers mit der Klasse `scrollytelling-container` dargestellt.

```
export default function Scrollytelling() {
  const steps = [1, 2, 3];

  return (
    <div className="scrollytelling-container">
      <div className="visuals">
        {/* Visualisierung */}
      </div>
      <div className="textuals">
        {steps.map((value) => (
          {/* Textinhalte */}
        ))}
      </div>
    </div>
  );
}
```

scrollytelling.js



5.2.1. React Scrollama

Mittels der `react-scrollama`-Bibliothek [19] wird die scrollbasierte Interaktion ermöglicht. Die `Scrollama`-Komponente wird um die `Step`-Komponente herumgewickelt, die jeweils einen Textinhalt repräsentieren. Die `onStepEnter`-, `onStepExit`- und `onStepProgress`-Funktionen werden genutzt, um die aktuelle Schrittanzahl, den Scrollfortschritt sowie den Ein- und Austritt eines Schrittes zu verfolgen.

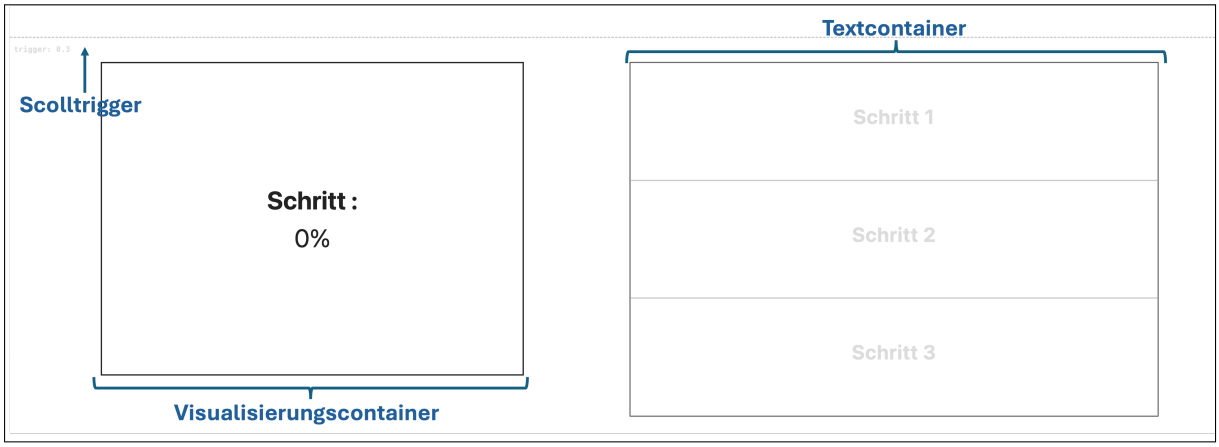
```
import { useState } from "react";
import { Scrollama, Step } from "react-scrollama";
import "./scrollytelling.scss";

export default function Scrollytelling() {
  const [currentStep, setCurrentStep] = useState(null);
  const [scrollProgress, setScrollProgress] = useState(0);
  const steps = [1, 2, 3];

  const onStepEnter = ({ data }) => setCurrentStep(data);
  const onStepProgress = ({ progress }) => setScrollProgress(progress);
  const onStepExit = ({ data, direction }) => {
    if ((direction === "down" && data === steps[steps.length - 1]) ||
        (direction === "up" && data === steps[0])) {
      setCurrentStep(null);
    }
  };

  return (
    <div className="scrollytelling-container">
      <div className="visuals">
        {/* Visualisierung */}
      </div>

      <div className="textuals">
        <Scrollama
        onStepEnter={onStepEnter}
        onStepExit={onStepExit}
        onStepProgress={onStepProgress}
        progress
        offset={0.3}
        debug={true}>
          {steps.map((value) => (
            <Step data={value}>
              {/* Textinhalte */}
            </Step>
          ))}
        </Scrollama>
      </div>
    </div>
  );
}
```

scrollytelling.js

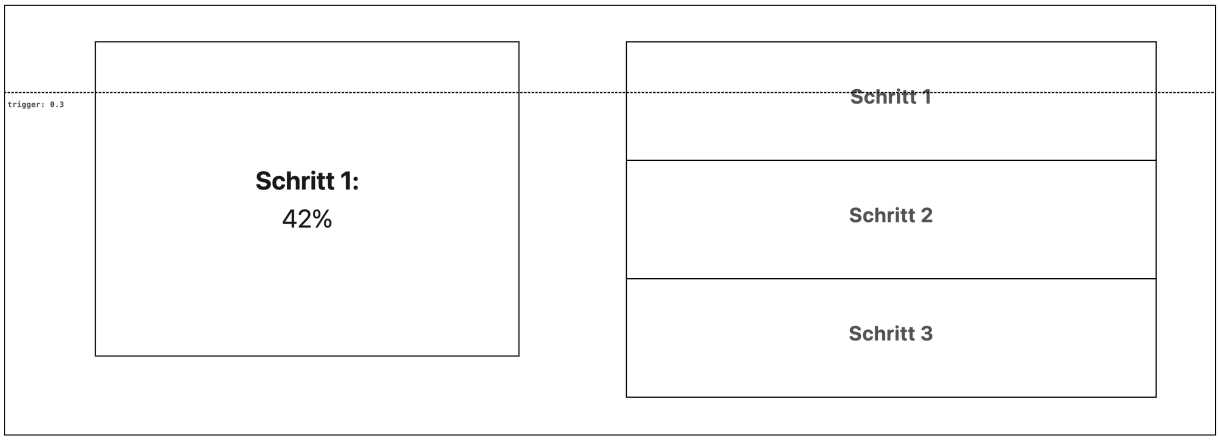
Um das Verhalten zu verdeutlichen und die Funktionsweise der einzelnen Schritte zu zeigen, soll folgendes Beispiel dienen:

Initial kreuzt sich der Auslöser (Trigger) noch nicht mit den sichtbaren Scrollytelling-Elementen. Der Trigger ist dabei durch den `offset`-Wert definiert, welcher angibt, wie viel Prozent von der Top-Position des Viewports der Trigger entfernt sein muss, um einen Schritt `Step` zu aktivieren:

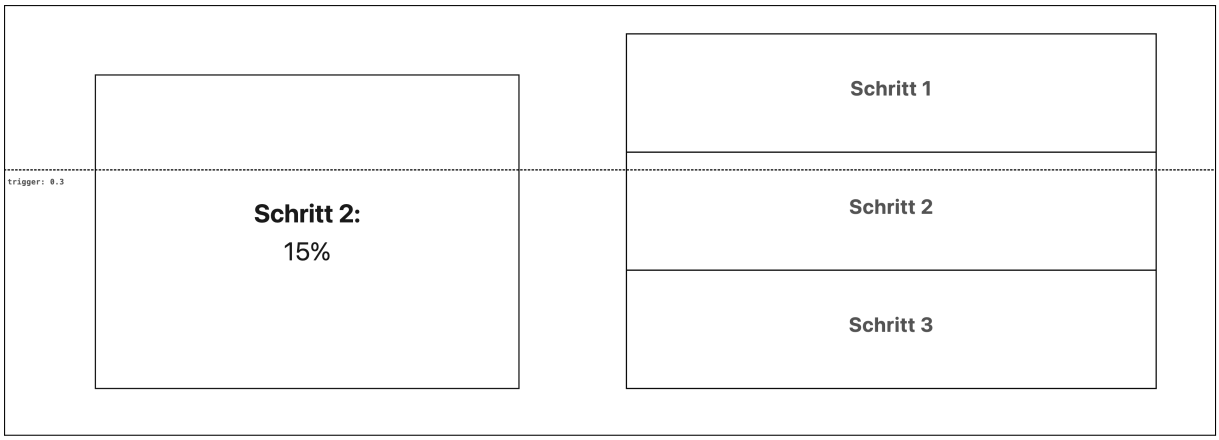
`debug=true` ermöglicht es, den Trigger sichtbar zu machen.



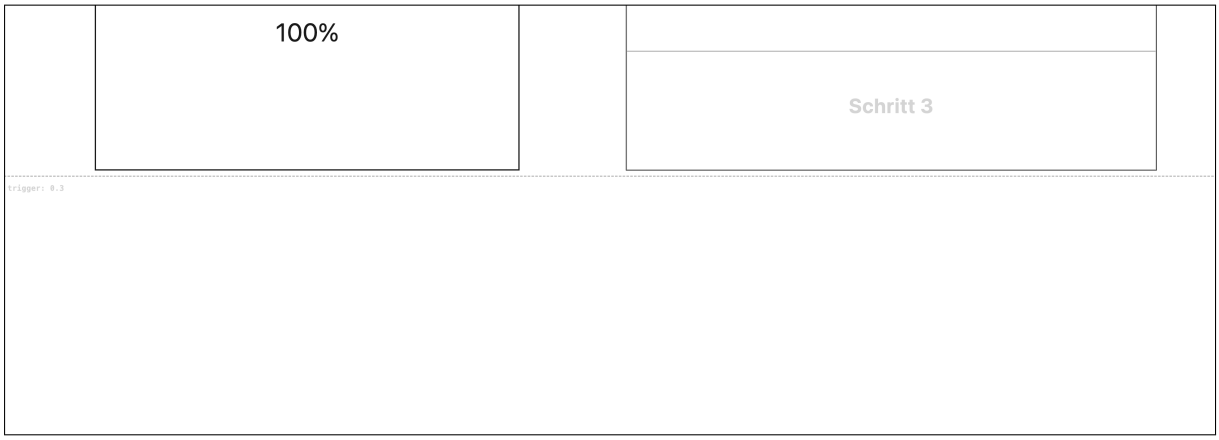
Sobald der obere oder untere Rand eines Schrittes `Step` die `offset`-Grenze erreicht, wird die `onStepEnter`-Funktion aufgerufen und der Schritt wird aktiviert. Dies ist der Moment, in dem ein bestimmter Inhalt für den Benutzer im Viewport sichtbar wird und der Fokus auf diesen Inhalt gelegt werden soll. Die Funktion `onStepEnter` erhält dabei ein Objekt mit den Daten des aktuellen Schrittes, wie in diesem Fall die Schrittnummer `data`. Schließlich wird der aktuelle Schritt `currentStep` auf den Wert des aktuellen Schrittes gesetzt. Ein weiteres, aber nicht zwingend notwendiges Feature ist die `onStepProgress`-Funktion, die den aktuellen Scrollfortschritt in Prozent zurückgibt. Damit lassen sich zusätzliche dynamische Interaktionen innerhalb eines Schrittes implementieren, die vom Scrollfortschritt abhängig sind:



Die aktuell gesetzte und scrollabhängige Information von `currentStep` kann dann an die Visualisierungskomponenten weitergegeben werden, um diese entsprechend zu aktualisieren. Beispielsweise wird im oberen Bild der aktuelle Schritt `currentStep = 1` an die Visualisierungskomponente übergeben, welche die Visualisierung für den ersten Schritt darstellt. Scrollt der Benutzer nun zur nächsten Schrittnummer, wird der aktuelle Schritt auf den Wert des nächsten Schrittes `currentStep = 2` gesetzt und die Visualisierungskomponente aktualisiert:



Verlässt der Benutzer einen Schritt `Step`, wird die `onStepExit`-Funktion aufgerufen. Im Webprojekt wird diese Funktion so modifiziert, dass wenn der erste Schritt nach oben oder der letzte Schritt nach unten verlassen wird, der aktuelle Schritt `currentStep` auf `null` gesetzt wird. Dies ist notwendig, um die Visualisierungskomponente zurückzusetzen und dem Benutzer zu signalisieren, dass er sich nicht mehr in einem Scrollytelling-Abschnitt befindet:



Damit die Visualisierungskomponente während des Scrollens immer sichtbar bleibt, wird der Container, in der sich die Visualisierungskomponente befindet mittels der `position: sticky` CSS-Eigenschaft fixiert. Das heißt, dass der Container sich relativ zum Viewport verhält und beim Scrollen immer sichtbar bleibt:

```
.visuals {
  position: sticky;
  ...
}
```

`scrollytelling.scss`

Weitere CSS-Stylings, wie beispielsweise die Hervorhebung des aktuellen Schrittes oder eine responsive Platzierung von Text und Visualisierung, sind ebenfalls sinnvoll, sollen hier aber nicht weiter vertieft werden.

5.2.2. Komponentenwiederverwendbarkeit

Anstatt das Scrollytelling-Gerüst isoliert zu lassen, wird im Webprojekt mittels der `props`-Funktion die Wiederverwendbarkeit der Komponente ermöglicht. Hier wird die `Scrollytelling`-Komponente um die `props`: `steps`, `textComponents`, `Visual`, und `heading` erweitert, welche die Anzahl der Schritte, die Textinhalte, die Visualisierungskomponente und die Überschrift eines Lehrabschnitts definieren. Der `Visual`-Komponente wird dabei der aktuelle Schritt und der Scrollfortschritt übergeben, um im Visualisierungsteil dynamische Interaktionen zu ermöglichen:

```
export default function
Scrollytelling({ steps, textComponents, Visual, heading = "" }) {
  ...
  return (
    <div className="scrollytelling-container">
      <div className="visuals">
        <Visual currentStep={currentStep}
                scrollProgress={scrollProgress} />
      </div>

      <div className="textuals">
        <p>{heading}</p>
        ...
          {steps.map((value) => (
            <Step data={value}>
              {textComponents[value - 1]}
            </Step>
          ))}
        </Scrollama>
      </div>
    </div>
  );
}
```

scrollytelling.js

Dieses Gerüst kann dann in den jeweiligen Lehrabschnitten wiederverwendet werden: Für einen Lehrabschnitt wie beispielsweise `Kernel Density Estimation` bedeutet dies Folgendes:

```
components
├── ...
└── scrollytelling
    ├── scrollytelling.js
    └── scrollytelling.scss
pages
├── ...
└── probability-density-estimation
    ├── ...
    └── non-parametric-methods
        ├── ...
        └── kernel-density-estimation
            ├── interactive
            ├── textual
            ├── visual
            └── kernel-density-estimation-scrollytelling.js
```

Das Scrollytelling-Gerüst, die Visualisierung und die Textinhalte werden in der `kernel-density-estimation-scrollytelling.js` importiert und als eine Scrollytelling-Einheit zusammengeführt:

```
import Scrollytelling from
"../../../../components/scrollytelling/scrollytelling";
import Visual from "./visual/visual";
import Text1 from "./textual/text-1";
...
import Text12 from "./textual/text-12";

const steps = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12];
const textComponents = [
  <Text1 />,
  ...
  <Text12 />,
];

const KernelDensityEstimationScrollytelling = () => (
  <Scrollytelling
    steps={steps}
    textComponents={textComponents}
    Visual={Visual}
    heading="Kernel Density Estimation"
  />
);

export default KernelDensityEstimationScrollytelling;
```

kernel-density-estimation-scrollytelling.js

Als solche Einheit wird diese dann exportiert, sodass in diesem Fall dieser Lehrabschnitt in die Seite für nicht-parametrische Methoden (`non-parametric-methods`) eingebunden werden kann.

5.2.3. Responsives Design

Wie in "Platzierung von Text und Visualisierung" beschrieben, gibt es zwei verschiedene Varianten für die Platzierung von Text und Visualisierung, die sich nach der Displaybreite des Endgeräts richten. Im Scrollytelling-Gerüst werden dafür zwei verschiedene `offset`-Werte definiert sowie ein Schwellenwert für die Displaybreite, ab dem die Platzierung der Textinhalte geändert wird.
Beim ersten Rendern der Komponente wird der `offset`-Zustand initial auf Basis der aktuellen Fensterbreite gesetzt:

```
const MOBILE_WIDTH = 768;
const OFFSET_MOBILE = 0.7;
const OFFSET_DESKTOP = 0.55;

export default function
Scrollytelling({ steps, textComponents, Visual, heading = "" }) {
  ...
  const calculateOffset = () => window.innerWidth < MOBILE_WIDTH ?
                                OFFSET_MOBILE : OFFSET_DESKTOP;
  const [offset, setOffset] = useState(calculateOffset);
}
```

scrollytelling.js

Wenn die Fensterbreite kleiner als `MOBILE_WIDTH` (768px) ist, wird der offset-Wert auf `OFFSET_MOBILE` (0.7) gesetzt, andernfalls auf `OFFSET_DESKTOP` (0.55). Dies sind lediglich Erfahrungswerte,

die sich im Entwicklungsprozess und durch Tests mit verschiedenen Endgeräten ergeben haben und können je nach Anforderung angepasst werden. Schließlich wird mit der folgenden `useEffect`-Funktion der `offset`-Wert bei einer Fenstergrößenänderung aktualisiert:

```
useEffect(() => {
  const handleResize = () => {
    setOffset(calculateOffset);
  };
  window.addEventListener("resize", handleResize);
  return () => window.removeEventListener("resize", handleResize);
}, []);
```

scrollytelling.js

Die `handleResize`-Funktion prüft auf die Fensterbreite und aktualisiert den `offset`-Wert dementsprechend. Ein `resize`-Event-Listener wird hinzugefügt, um die `handleResize`-Funktion jedes Mal auszuführen, wenn die Fenstergröße geändert wird. Schließlich wird der Event-Listener wieder entfernt, wenn die Komponente abmontiert wird.

5.3. Visualisierungen

5.3.1. Präsentation der Visualisierung

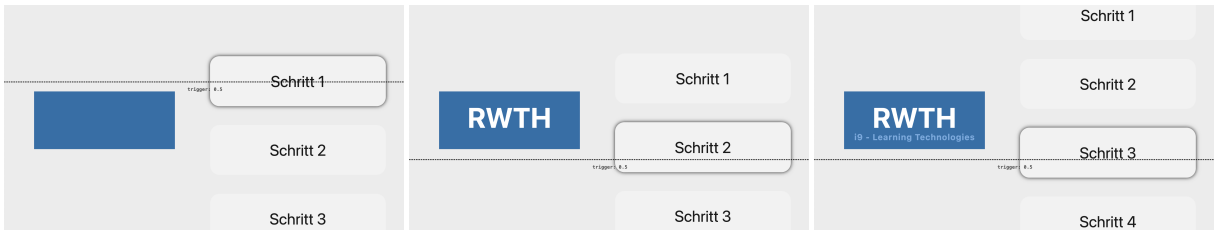
Die Visualisierungen in den Dateien `visual.js`, die in das Scrollytelling-Gerüst eingebunden sind, sind SVGs

```
<svg ref={svgRef} viewBox="0 0 1000 1000" />;
```

die mittels der Visualisierungsbibliothek D3.js über eine gewisse Anzahl an `steps` hinweg manipuliert werden. Um den Manipulationsprozess über mehrere Schritte hinweg zu steuern und den Entwicklungsprozess dabei zu erleichtern, ist folgende Struktur vorgesehen:

```
const actions = useRef([
  {
    start: 1,
    end: 3,
    action: () => {
      drawRectangle();
    },
    remove: () => {},
    called: false,
  },
  {
    start: 2,
    end: 3,
    action: () => {
      drawText("RWTH");
    },
    remove: () => {
      d3.selectAll(".text").remove();
    },
    called: false,
  },
  {
    start: 3,
    end: 3,
    action: () => {
      drawSubText("i9 - Learning Technologies");
    },
    remove: () => {
      d3.selectAll(".subtext").remove();
    },
    called: false,
  },
  ...
]).current;
```

visual.js



(1) (2) (3)

Das Array `actions` enthält eine Liste von Aktionen, die die Manipulationen der SVG über die Schritte hinweg steuern. Jede Aktion enthält eine Funktion (`action`), die ausgeführt wird, wenn der aktuelle Schritt (`currentStep`) in den definierten Bereich von `start` bis `end` fällt, und eine Funktion (`remove`), die ausgeführt wird, wenn der Schritt den Bereich verlässt. Im Beispiel oben wird ein Rechteck für die Schritte 1 bis 3 gezeichnet (→ `(1)`), der Text `RWTH` für die Schritte 2 bis 3 (→ `(2)`) und der Untertext `i9 - Learning Technologies` (→ `(3)`) für den Schritt 3. Verlässt der Nutzer den Schritt 3, wird der Untertext entfernt, ebenso wie der Text `RWTH`, wenn der Nutzer das Schrittintervall 2 bis 3 verlässt. Aktionen werden in ihrem Intervall nur einmal ausgeführt, was durch das Attribut `called` überwacht wird.
Die folgende `useEffect`-Logik überprüft hierbei bei jeder Veränderung von `currentStep`, ob eine Aktion ausgeführt oder entfernt werden muss, und setzt den Status der Aktionen entsprechend (Zeile 15–24).

```
1  export default function Visual({ currentStep, scrollProgress }) {
2    const svgRef = useRef();
3    ...
4    const actions = useRef([
5      ...
6    ]).current;
7
8    useEffect(() => {
9      if (currentStep === null) {
10       d3.select(svgRef.current).selectAll("*").remove();
11       actions.forEach((action) => {
12         action.called = false;
13       });
14     }
15     actions.forEach((action) => {
16       const inRange = currentStep >= action.start &&
17                       currentStep <= action.end;
18       if (inRange && !action.called) {
19         action.action();
20         action.called = true;
21       } else if (!inRange && action.called) {
22         action.remove();
23         action.called = false;
24       }
25     });
26   }, [currentStep]);
27
28   return <svg ref={svgRef} viewBox="0 0 1000 1000" />;
29 }
```

visual.js

Wird ein Scrollytelling-Abschnitt verlassen (`currentStep === null`), werden alle Elemente aus der SVG entfernt und der Status der Aktionen zurückgesetzt (Z. 9–14). Im oberen Code-Ausschnitt ist ebenso zu sehen, dass neben der Schrittinformation vom Scrollytelling-Gerüst auch die `scrollProgress`-Information übergeben wird (Z. 1), welche den prozentualen Fortschritt des aktuellen Schrittes angibt. Wenngleich diese in der Webanwendung nur sehr selten genutzt wird, kann sie in der Visualisierung verwendet werden, um einen schrittspezifischen Effekt zu erzielen, wie beispielsweise das Einblenden eines Elements bei einem bestimmten prozentualen Fortschritt des Schrittes. Die dafür notwendige Logik kann in der `useEffect`-Funktion ergänzt werden, wird jedoch im aktuellen Zustand der Webanwendung nicht genutzt.

5.3.2. Plotten von Funktionen

Eine typische Visualisierungsaufgabe im Webprojekt ist das Plotten von Funktionen. Das Plotten wird dabei in einer Funktion ausgelagert, um sie anschließend im `actions`-Array zu verwenden, was die Lesbarkeit und Struktur der in `actions` definierten Aktionen verbessert.
Für das Plotten der Funktionen wird erneut die Bibliothek D3.js verwendet. Soll beispielsweise die Funktion $\sin(x)$ geplottet werden, kann dies wie folgt realisiert werden:

Definieren der Skalierungsfunktionen

Um die Funktion auf der SVG zu plotten, ist es sinnvoll, passende Skalierungsfunktionen zu definieren, die die Funktionswerte in Pixelwerte umrechnen. Der Domänenbereich der Skalierungsfunktionen wird dabei so gewählt, dass die Funktion auf der SVG gut sichtbar ist, und ist dementsprechend von der Art der Funktion und der Größe der SVG abhängig. Für die Funktion $\sin(x)$ könnte der Domänenbereich $[0, 2\pi]$ und der Wertebereich $[-1.5, 1.5]$ gewählt werden. Dieser Wertebereich sollte ferner auf der SVG so platziert werden, dass ein kleiner Rand um die Funktion entsteht und die Funktion nicht an den Rändern der SVG anliegt. Für eine SVG mit den Maßen von 1000×1000 Pixeln sollte der Wertebereich nicht auf $[0, 1000]$ gesetzt werden, da die Funktion an den Rändern der SVG anliegen würde. Ein Rand von 50 Pixeln als Abstand zu den SVG-Rändern ist hierbei sinnvoll:

```
// 1. Definieren der Skalierungsfunktionen
const xScale = d3
  .scaleLinear()
  .domain([0, 2 * Math.PI])
  .range([50, 950]);
const yScale = d3
  .scaleLinear()
  .domain([-1.5, 1.5])
  .range([950, 50]);
```

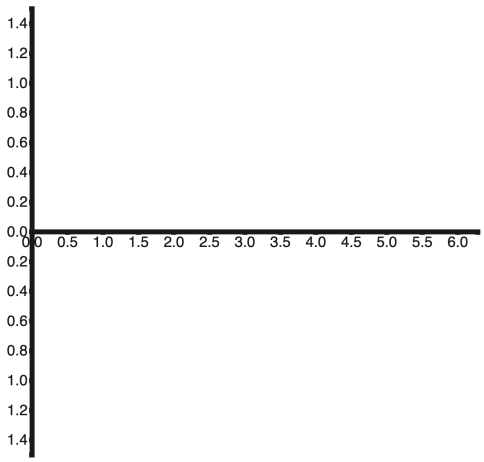
visual.js

Achsenfunktionen

Um die definierten Domänen als Achsen sichtbar zu machen, eignet es sich, eine Funktion zu definieren, die im Projekt meistens als `drawCoordSystem` bezeichnet wird.

```
// 2. Plotten der x- und y-Achse
function drawCoordSystem() {
  const xAxis = d3.axisBottom(xScale);
  const yAxis = d3.axisLeft(yScale);

  d3.select(svgRef.current)
    .append("g")
    .attr("transform", "translate(0, 500)")
    .call(xAxis)
  d3.select(svgRef.current)
    .append("g")
    .attr("transform", "translate(50, 0)")
    .call(yAxis)
}
```

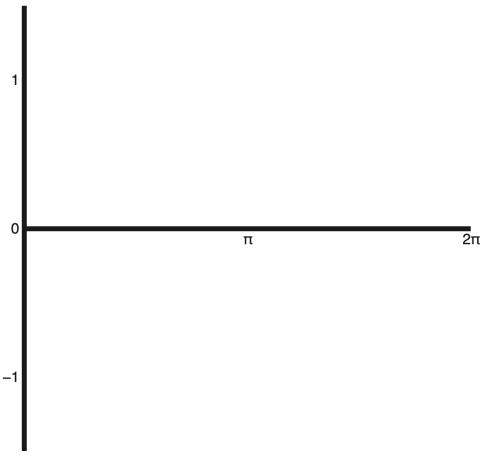
visual.js



Das obere Grundgerüst kann, wie im Abschnitt "Skalen und Achsen" erwähnt, um Eigenschaften wie Achsenbeschriftungen, Abstände der Achsenbeschriftungen, Achsendicke und viele weitere

Eigenschaften erweitert werden. Für trigonometrische Funktionen wie $\sin(x)$ kann beispielsweise folgende Achsengestaltung sinnvoll sein:

```
...
.call(
  xAxis
    .tickSize(0)
    .tickPadding(10)
    .tickValues([Math.PI, 2 * Math.PI])
    .tickFormat((d) => (
      d === Math.PI ?
      "\u03C0" :
      d === 2 * Math.PI ?
      "2\u03C0" : d))
)
...
.call(
  yAxis
    .tickSize(0)
    .tickPadding(10)
    .tickValues([-1, 0, 1])
    .tickFormat((d) => d3.format("d")(d))
)
...
```



visual.js

Plottfunktionen

Um letztlich eine Funktion wie $\sin(x)$ zu plotten, sollte eine wiederverwendbare und dynamische Funktion definiert werden, welche ebenfalls auf Änderungen der zu plottenden Funktion reagieren kann:

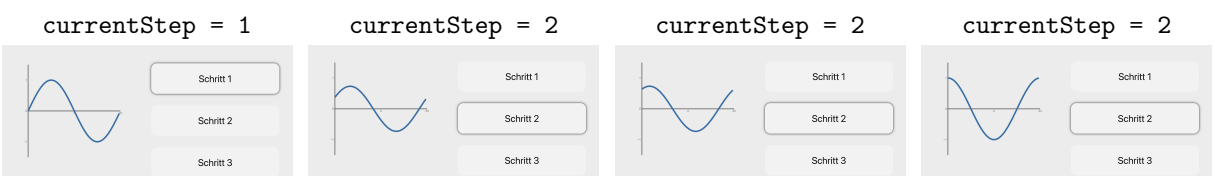
```
// 3. Plotten der Funktion
function plotFunction(funToPlot, className) {
  const line = d3
    .line()
    .x((d) => xScale(d.x))
    .y((d) => yScale(d.y));

  const xValues = d3.range(xScale.domain()[0], xScale.domain()[1], 0.1);
  const data = xValues.map((x) => ({ x: x, y: funToPlot(x) }));

  svg.current
    .selectAll(`.${className}`)
    .data([data])
    .join(
      (enter) =>
        enter
          .append("path")
          .attr("class", className)
          .attr("d", line),
      (update) => update.transition().duration(400).attr("d", line),
      (exit) => exit.remove()
    );
}
```

visual.js

Essenziell ist dabei D3's `line`-Funktion, die einen Generator darstellt, der die x- und y-Werte der Datenpunkte in Pixelwerte umrechnet und diese Daten als einen SVG-Pfad-String zurückgeben kann (Z. 3-6). Diese Daten müssen für die zu plottende Funktion jedoch erst generiert werden (Z. 8-9). Schließlich kann mit dem üblichen D3-Enter-Update-Exit-Pattern die Funktion geplottet werden (Z. 11-22). Dabei werden die von der zu plottenden Funktion generierten Koordinatendaten an die `line`-Funktion übergeben und als Pfad auf der SVG gezeichnet. Sollte die Funktion, die durch `className` gekennzeichnet ist, bereits geplottet sein, wird diese bei erneutem Aufruf der Funktion mittels `update.transition().duration(400).attr("d", line)` dynamisch aktualisiert. Falls passend, kann hier auch eine selbstdefinierte Transition hinzugefügt werden. Ein typischer Visualisierungsablauf sieht im Projekt dabei wie folgt aus:

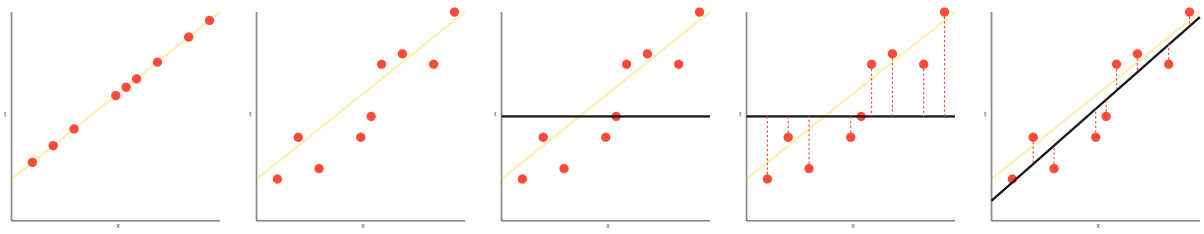
```
1  export default function Visual({ currentStep, scrollProgress }) {
2    const svgRef = useRef();
3    // 1. Definieren der Skalierungsfunktionen
4    ...
5    const actions = useRef([
6      {
7        start: 1, end: 2,
8        action: () => {
9          drawCoordSystem();
10       },
11       remove: () => {}, called: false,
12     },
13     {
14       start: 1, end: 1,
15       action: () => {
16         plotFunction((x) => Math.sin(x), "trigonometric-function");
17       },
18       remove: () => {}, called: false,
19     },
20     {
21       start: 2, end: 2,
22       action: () => {
23         plotFunction((x) => Math.cos(x), "trigonometric-function");
24       },
25       remove: () => {}, called: false,
26     },
27   ]).current;
28   // 2. Plotten der x- und y-Achse
29   function drawCoordSystem() {...}
30   // 3. Plotten der Funktion
31   function plotFunction() {...}
32
33   useEffect(() => {...}, [currentStep]);
34
35   return <svg ref={svgRef} viewBox="0 0 1000 1000" />
36 }
```

visual.js



$\sin(x)$ | In Transition | In Transition | $\cos(x)$

Das Koordinatensystem wird für die Schritte 1 bis 2 gezeichnet, und im ersten Schritt wird die Funktion $\sin(x)$ geplottet. Falls der Nutzer Schritt 2 erreicht, wird die Funktion $\sin(x)$ durch die Funktion $\cos(x)$ dynamisch mittels einer Transition ersetzt. Das hier vorgestellte Prinzip der schrittweisen und dynamischen Manipulation von Funktionen wird im Projekt als Hauptmethode für die Visualisierung von Funktionen sowie Datenpunkten und anderen Visualisierungselementen verwendet. Als Beispiel aus vielen im Projekt, die dieses Prinzip verfolgen, sei hier die Einführungsvisualisierung in das Thema der linearen Regression genannt:



Hier wird neben der stückweisen Anpassung der Regressionsfunktion ebenfalls die Visualisierung von Datenpunkte und Linien schrittweise angepasst. Die dafür benötigte Funktionalität ist der obigen sehr ähnlich, das D3-Enter-Update-Exit-Pattern ist dabei die zentrale Methode um die Visualisierung schrittweise zu manipulieren.

5.3.3. Plotten von 3D-Funktionen

Das native Plotten von 3D-Funktionen ist in D3.js nicht möglich. Jedoch ist es möglich, die zusätzliche Dimension durch die Verwendung von Farben zu repräsentieren. Der strukturelle Aufbau der Plot-Funktion ist dabei ähnlich wie zuvor, jedoch wird ein Gitter von Rechtecken anstatt einer Linie gezeichnet. Die Farbe der Rechtecke wird dabei durch die Funktionswerte bestimmt, sodass der Funktionsplot einer Heatmap entspricht.

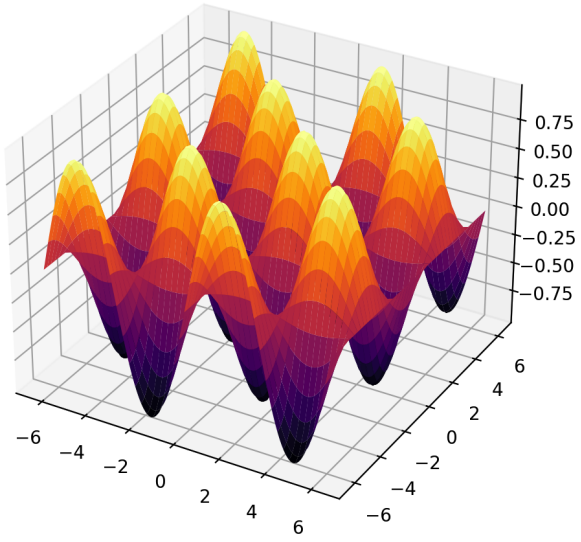
```
1  function plot3DFunction(funToPlot, className) {
2    const stepSize = 0.05;
3    const xValues = d3.range(xScale.domain()[0], xScale.domain()[1],
       stepSize);
4    const yValues = d3.range(yScale.domain()[0], yScale.domain()[1],
       stepSize);
5
6    const data =
7     xValues.map((x) =>
8     yValues.map((y) =>
9      ({ x: x, y: y, z: funToPlot(x, y) }))).flat();
10
11   const colorScale = d3.scaleSequential(d3.interpolateInferno).domain
       ([-1, 1]);
12
13   svg.current
14     .selectAll(`.${className}`)
15     .data(data)
16     .join(
17       (enter) =>
18         enter
19           .append("rect")
20           .attr("class", className)
21           .attr("width", 10)
22           .attr("height", 10)
23           .attr("x", (d) => xScale(d.x))
24           .attr("y", (d) => yScale(d.y))
25           .attr("fill", (d) => colorScale(d.z)),
```

```
26        (update) =>
27          update
28            .transition()
29            .duration(500)
30            .attr("x", (d) => xScale(d.x))
31            .attr("y", (d) => yScale(d.y))
32            .attr("fill", (d) => colorScale(d.z)),
33        (exit) => exit.remove()
34      );
35  }
```

visual.js

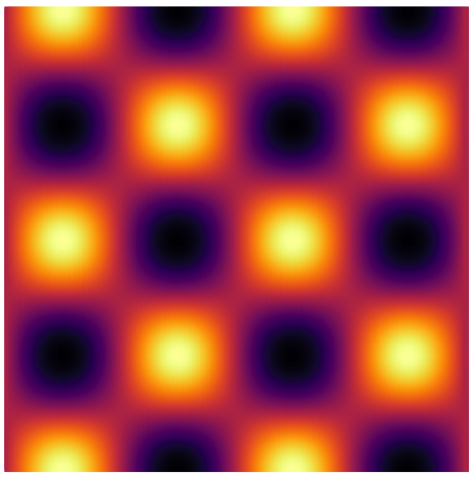
Die Auflösung des Gitters ist durch die Schrittweite `stepSize` definiert und sollte mit Bedacht gewählt werden, da eine zu hohe Auflösung zu einer Überlastung des Browsers führen kann.

Plot mit matplotlib.pyplot



$f(x, y) = \sin(x) \cdot \cos(y)$

Plot mit D3.js (plot3DFunction)



$f(x, y) = \sin(x) \cdot \cos(y)$

5.4. Interaktive Demos

Die interaktiven Scrollytelling-Einheiten werden, falls passend, mit einer interaktiven Demo ergänzt, die die Funktionsweise des jeweiligen Themas veranschaulicht. Als Basis für die interaktiven Demos wird erneut eine SVG verwendet, die mit D3.js manipuliert wird.

5.4.1. Interaktion mit Visualisierungsparametern

Die interaktiven Demos beschreiben in den meisten Fällen eine Änderung von Parametern oder Datenzuständen, die eine Veränderung der Visualisierung zur Folge haben. Soll beispielsweise eine interaktive Demo für eine Funktion wie der univariaten Gaußverteilung $\mathcal{N}(\mu, \sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$ erstellt werden, so ist folgendes Schema üblich:
Die zu verändernden Parameter werden als Zustandsvariablen in der jeweiligen React-Komponente festgehalten:

```
1  const [mean, setMean] = useState(0);
2  const [variance, setVariance] = useState(1);
```

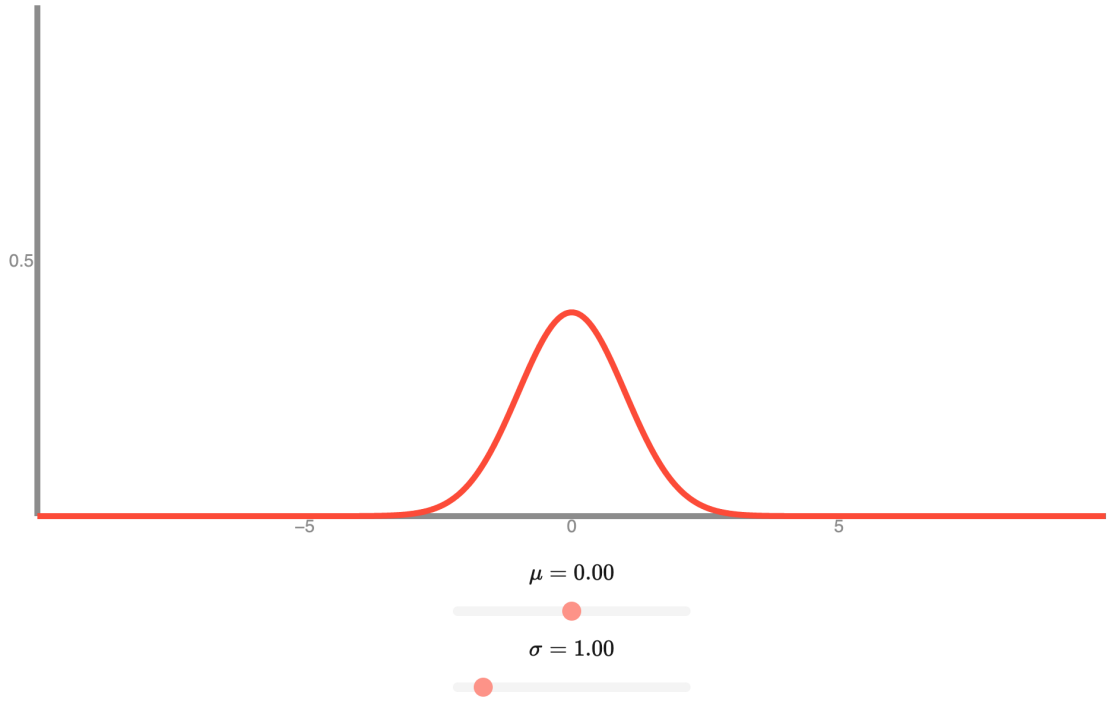
Mittels einer geeigneten `useEffect`-Logik wird die Visualisierung bei einer Änderung der Zustandsvariable neu gezeichnet:

```
useEffect(() => {
  plotUnivariateGaussian(mean, variance);
}, [mean, variance]);
```

Die Funktion, welche die Visualisierung zeichnet, sollte dabei auf die Zustandsvariable zugreifen und die Visualisierung mittels des üblichen D3-Enter-Update-Exit-Pattern zeichnen:

```
function plotUnivariateGaussian(mean, variance) {
  const line = d3
    .line()
    .x((d) => xScale(d.x))
    .y((d) => yScale(d.y));

  const xValues = d3.range(xScale.domain()[0], xScale.domain()[1], .01);
  const data = xValues.map((x) =>
              ({ x: x, y: univariateGaussian(x, mean, variance) }));

  svg.current
    .selectAll(".univariate-gaussian-line")
    .data([data])
    .join(
      (enter) => enter
        .append("path")
        .attr("class", "univariate-gaussian-line")
        .attr("d", line),
      (update) => update.
        attr("d", line),
      (exit) => exit.remove()
    );
}
```

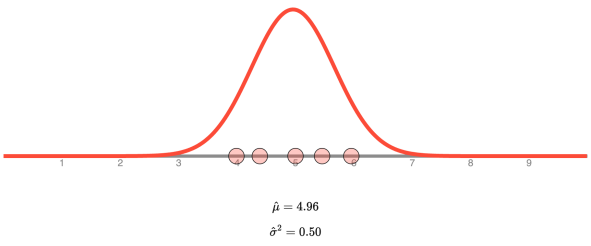
Wie die Parameter verändert werden sollen, hängt vom jeweiligen Anwendungsfall ab. Für eine einfache Visualisierungsaufgabe wie die obige können beispielsweise Slider verwendet werden, die die Parameter verändern:



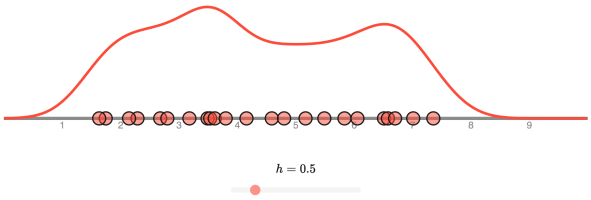
Ein weiterer üblicher Anwendungsfall, um die Visualisierung zu verändern, ist das Hinzufügen von Datenpunkten via Click-Events. Hierbei wird ein Click-Event auf das SVG-Element registriert und die Datenpunkte werden bei jedem Click-Event aktualisiert:

```
1  const [dataPoints, setDataPoints] = useState([]);
2
3  useEffect(() => {
4    clickToAddPoints();
5    plotPoints();
6    yourVisualizationFunction();
7  }, [dataPoints]);
8
9  function clickToAddPoints() {
10   const svg = d3.select(svgRef.current);
11   svg.on("click", (event) => {
12     const [clickX, clickY] = d3.pointer(event, svg.node());
13     const xValue = xScale.invert(clickX);
14     const yValue = yScale.invert(clickY);
15     setDataPoints((prev) => [...prev, { x: xValue, y: yValue }]);
16   });
17 }
```

Der Klick auf das SVG-Element wird dabei mittels der `d3.pointer`-Funktion in die Koordinaten des SVG-Elements umgerechnet (vgl. Z. 12), welche dann in den entsprechenden Domänenbereich zurückgerechnet werden (vgl. Z. 13-14) um die Datenpunkte in den für die Visualisierung passenden Koordinatenraum zu transformieren. Schließlich wird der neu hinzugefügte Datenpunkt in einer entsprechenden State-Variable `dataPoints` gespeichert (vgl. Z. 15). Der aktualisierte Zustand der Datenpunkte, sowie die von den Datenpunkten abhängige Visualisierung wird dann in der `useEffect`-Funktion erneut gezeichnet (vgl. Z. 3-7). Im Webprojekt wird eine interaktive Demo typischerweise nicht nur durch Klick-Events realisiert (→ `(1)`), sondern häufig auch durch eine Kombination von Klick-Events und Slidern (→ `(2)`), um die Eigenschaften von Funktionen oder Algorithmen anschaulicher darzustellen.



(1)



(2)

5.4.2. Visualisierungsberechnungen mit `numeric.js`

Während sich einige Visualisierungen durch einfache mathematische Funktionen wie die univariate Gaußverteilung beschreiben lassen, erfordern andere Visualisierungen, insbesondere im Bereich des maschinellen Lernens, komplexere Berechnungen. Entscheidungsgrenzen in der Klassifikation oder Regressionskurven bedürfen oftmals numerisch aufwendige Berechnungen wie Matrixmultiplikationen, Matrixinversionen, dem Lösen von linearen Gleichungssystemen oder dem iterativen Lösen von Optimierungsproblemen. Um solche Berechnungen in JavaScript durchzuführen, wird die Bibliothek `numeric.js` (vgl. [28]) verwendet, welche eine Vielzahl von numerischen Funktionen bereitstellt. Die wohl üblichste Abfolge von Matrixoperationen im Webprojekt ist die geschlossene Lösung des Optimierungsproblems der quadratischen Fehlerfunktion:

$$\underset{\vec{w}}{\operatorname{argmin}} \frac{1}{2} \sum_{n=1}^{N} (y(\vec{x}_n; \vec{w}) - t_n)^2 = (X^\top X)^{-1} X^\top \vec{t}$$

welche sich mit `numeric.js` in der folgenden einfachen Form darstellen lässt:

```
// Vorausgesetzt, dass die Datenpunkte zwei-dimensionale Koordinaten sind
const X = dataPoints.map((d) => [d.x, d.y, 1]);
const t = dataPoints.map((d) => [d.t]);

const XT = numeric.transpose(X);
const XTX = numeric.dot(XT, X);
const XTX_inv = numeric.inv(XTX);
const pseudoInverse = numeric.dot(XTX_inv, XT);

const w = numeric.dot(pseudoInverse, t);
```

Berechnungen sowie Variationen dieser Art werden benutzt, um die Visualisierungen für die Themenbereiche "Linear Discriminant", "Linear Regression" und "Logistic Regression" zu erstellen, stellen jedoch fachspezifische Gleichungen dar, die nicht der Fokus dieser Arbeit sind. Es soll lediglich darauf hingewiesen werden, dass im Entwicklungsprozess sich gezeigt hat, dass die Invertierung von Matrizen mittels `numeric.inv` zu numerischen Instabilitäten führen kann. Daher sollte in der Praxis auf numerische Stabilität geachtet werden, indem beispielsweise die zu invertierende Matrix mit einem kleinen Wert ϵ regularisiert wird $(X^\top X + \epsilon I)$ oder die Pseudoinverse mit einem numerisch stabileren Verfahren wie der Singulärwertzerlegung berechnet wird.
Der bei den Verfahren resultierende Wert $\vec{w}$ wird dann wiederum dazu verwendet, um übliche Visualisierungen wie die Darstellung von Linien, Pfaden oder höherdimensionalen Entscheidungsgrenzen oder Regressionskurven in Form eines Gitters zu erstellen. Visualisierungsaufgaben, die in dieser Arbeit bereits vertieft beschrieben sind.

5.4.3. Handschrifterkennung im Web

Während somit die ML-Visualisierungen im Projekt auf der Visualisierungsbibliothek D3.js, dem Webstandard von SVGs und numerischen Berechnungen basieren, stellt die letzte interaktive Demo im Projekt eine Ausnahme dar. Für die Handschrifterkennung von Ziffern und Buchstaben ist neben dem Trainieren von Modellen ebenfalls eine Zeichenfläche für die Eingabe von Zeichen erforderlich. Darüber hinaus ist es notwendig, Eingaben auf dieser Zeichenfläche so vorzuverarbeiten, dass sie möglichst ähnlich zu den Trainingsdaten im Datensatz sind, welcher zum Trainieren des Modells verwendet wird. Letztlich muss das trainierte Modell in ein webfähiges Format konvertiert werden, um es im Browser verwenden zu können. Der Entwicklungsprozess ist somit in vier wesentliche Schritte unterteilbar:

1. Entwicklung einer Zeichenfläche (für die Eingabe von Ziffern und Buchstaben)
2. Vorverarbeitung der Eingaben (Extrahieren der Zeichnung, Skalierung, Zentrierung)
3. Trainieren eines Modells (Datensatzaufbereitung, Modelltraining, Modellvalidierung)
4. Konvertierung des Modells (in ein webfähiges Format)

Im Folgenden wird auf die einzelnen Schritte eingegangen:

Entwicklung einer Zeichenfläche

Die Zeichenfläche wird mittels HTML-Canvas erstellt, welches die Zeichnung des Nutzers aufnimmt. Die Zeichenfunktionalität ist dabei in einer React-Hook-Komponente ausgelagert, welche die Zeichnung des Nutzers in Form von Pfaden aufnimmt und an die übergeordnete Komponente weitergibt:

```
  export default function useDrawingCanvas(canvasRef, setStrokes) {
  const isDrawing = useRef(false);
  const lastPosition = useRef({ x: 0, y: 0 });

  useEffect(() => {
    const canvas = canvasRef.current;
    const context = canvas.getContext("2d");
    ...
  }, []);
}
```

canvas.js

Die für die Zeichnung notwendigen Event-Listener werden dabei in einer `useEffect`-Hook registriert und Maus- oder Touch-Events an die entsprechenden Handler-Funktionen weitergeleitet:

```
useEffect(() => {
  const canvas = canvasRef.current;
  const context = canvas.getContext("2d");
  ...
  canvas.addEventListener("mousedown", handleMouseDown);
  canvas.addEventListener("mousemove", handleMouseMove);
  canvas.addEventListener("mouseup", handleMouseUp);
  canvas.addEventListener("mouseleave", handleMouseLeave);
  canvas.addEventListener("mouseenter", handleMouseUp);
  canvas.addEventListener("touchstart", handleTouchStart);
  canvas.addEventListener("touchmove", handleTouchMove);
  canvas.addEventListener("touchend", handleTouchEnd);
}, []);
```

Die für die `handle`-Funktionen relevanten Funktionen sind dabei `startDrawing`:

```
function startDrawing(x, y) {
  isDrawing.current = true;
  lastPosition.current = { x, y };
  // Logik um strokes zu aktualisieren
}
```

welche die Zeichnung startet, `draw`:

```
function draw(x, y) {
  if (!isDrawing.current) return;

  context.beginPath();
  context.moveTo(lastPosition.current.x, lastPosition.current.y);
  context.lineTo(x, y);
  context.stroke();

  lastPosition.current = { x, y };
  // Logik um strokes zu aktualisieren
}
```

welche die Zeichnung fortsetzt, indem eine Linie zwischen der letzten Position und der aktuellen Position gezeichnet wird, und `stopDrawing`:

```
function stopDrawing() {
  isDrawing.current = false;
}
```

welche die Zeichnung beendet. Ebenso wird mit `setStrokes`, welche von der übergeordneten Komponente übergeben wird, die Zeichnung in Form von Pfaden gespeichert. Mittels der Funktion `getOffsetPosition` wird die Position des Mauszeigers relativ zur Zeichenfläche ermittelt:

```
function getOffsetPosition(e) {
  const rect = canvas.getBoundingClientRect();
  return {
    x: e.clientX - rect.left,
    y: e.clientY - rect.top,
  };
}
```

Mittels dieser Funktionen können die `handle`-Funktionen die Maus- oder Touch-Events an die entsprechenden `startDrawing`, `draw` und `stopDrawing`-Funktionen weiterleiten und die Zeichnung des Nutzers aufnehmen. Exemplarisch für Mausbewegungen sieht dies wie folgt aus:

Die Logik für Touchevents ist ähnlich

```
function handleMouseDown(e) {
  if (e.button === 2) return; // Ignoriere Rechtsklick
  const { x, y } = getOffsetPosition(e);
  startDrawing(x, y);
}

function handleMouseMove(e) {
  if (!isDrawing.current) return; // Ignoriere Mausbewegungen ohne Klick
  const { x, y } = getOffsetPosition(e);
  draw(x, y);
}

function handleMouseUp(e) {
  if (e.button === 2) return; // Ignoriere Rechtsklick
  stopDrawing();
}

function handleMouseLeave() {
  stopDrawing();
}
```

`handleMouseDown` registriert den Start der Zeichnung, indem die Position des Mauszeigers relativ zur Zeichenfläche ermittelt wird und `startDrawing` aufgerufen wird, welche wiederum die Zeichnung startet und die letzte Position (hier die Startposition) speichert. `handleMouseMove` zeichnet die Linie zwischen der letzten Position und der aktuellen Position (welche ebenfalls relativ zur Zeichenfläche ermittelt wird), und `handleMouseUp` beendet die Zeichnung. `handleMouseLeave` beendet die Zeichnung, wenn der Mauszeiger die Zeichenfläche verlässt. Für die Ziffer- und Buchstabenerkennung ist auf gewisse Konfigurationen, wie die Farbe der Zeichnungen und die Linienstärke, zu achten, um die Zeichnung möglichst ähnlich zu den Trainingsdaten zu halten. Diese Konfigurationen werden ebenfalls in der `useDrawingCanvas`-Komponente vorgenommen:

```
useEffect(() => {
  ...
  const context = canvas.getContext("2d");
  context.lineWidth = 10;
  context.lineCap = "round";
  context.strokeStyle = "white";
  ...
}, []);
```

Vorverarbeitung der Eingaben (Preprocessing)

Die Datensätze, die für die Handschrifterkennung verwendet werden, sind der MNIST [36] (Ziffern) und EMNIST [9] (Ziffern- und Buchstaben) Datensatz, wobei das Format der Datenexemplare des EMNIST-Datensatzes dem Format der Datenexemplare des MNIST-Datensatzes entspricht. Die Ersteller des MNIST-Datensatzes beschreiben das Format der Daten wie folgt:

"The original black and white (bilevel) images from NIST were size normalized to fit in a 20x20 pixel box while preserving their aspect ratio. The resulting images contain grey levels as a result of the anti-aliasing technique used by the normalization algorithm. The images were centered in a 28x28 image by computing the center of mass of the pixels, and translating the image so as to position this point at the center of the 28x28 field." [36]

Um eine möglichst gute Klassifikationsqualität zu erreichen, ist es daher notwendig, die Eingaben der Zeichenfläche zu extrahieren und so zu transformieren, dass sie dem Format der Trainingsdaten des zu trainierenden Modells möglichst ähnlich sind. Dafür ist die Datei `handwriting-preprocessing.js` zuständig, welche die einzelnen Schritte der Vorverarbeitung implementiert. Der erste Schritt ist die Berechnung einer sogenannten Bounding Box, die die Zeichnung des Nutzers umschließt:

```
function drawBoundingBox() {
  // Filtern der Koordinatenwerte aus den Strokes
  const strokesFlatten = strokes.flat();
  const xValues = strokesFlatten.map((stroke) => stroke.x);
  const yValues = strokesFlatten.map((stroke) => stroke.y);

  // Kleinste und groesste Koordinatenwerte fuer Bounding Box
  const minX = Math.min(...xValues) - lineWidth / 2;
  const maxX = Math.max(...xValues) + lineWidth / 2;
  const minY = Math.min(...yValues) - lineWidth / 2;
  const maxY = Math.max(...yValues) + lineWidth / 2;

  // Bounding Box als minimalstes Quadrat um die Zeichnung
  const boundingBoxHeight = maxY - minY;
  const boundingBoxWidth = maxX - minX;
  const maxSide = Math.max(boundingBoxHeight, boundingBoxWidth);

  // Kopieren des Zeichnungs-Canvas auf das Bounding Box Canvas
  const boundingBoxCanvas = boundingBoxCanvasRef.current;
  const ctx = boundingBoxCanvas.getContext("2d");

  ctx.clearRect(0,0,boundingBoxCanvas.width,boundingBoxCanvas.height);
  ctx.drawImage(
    canvasRef.current,
    // Canvas vom dem die Zeichnung kopiert wird
    minX,
    minY,
    maxSide,
    maxSide,
    // Canvas auf das die Zeichnung kopiert wird
    0,
    0,
    boundingBoxCanvas.width,
    boundingBoxCanvas.height
  );
}
```

handwriting-preprocessing.js

Zeichencanvas Bounding Box

Die Bounding Box wird dabei als das kleinste Quadrat definiert, welches die Zeichnung umschließt. Die Koordinatenwerte der Zeichnung werden dafür aus den gespeicherten Pfaden extrahiert, die in der `useDrawingCanvas`-Komponente gespeichert werden. Schließlich bestimmen die minimalen und maximalen Koordinatenwerte die Größe der Bounding Box. Diese wird dann auf ein neues Canvas-Element `boundingBoxCanvas` mittels der `drawImage`-Funktion kopiert. Das zu kopierende Element vom Quellcanvas (Z. 24) ist dabei durch das Quadrat definiert, das seine obere linke Ecke bei `(minX, minY)` (Z. 26-27) hat und seine Seiten mit Länge `maxSide` aufspannt (Z. 28-29). Das kopierte Element wird dann auf den Zielcanvas `ctx` kopiert (Z. 31-34), welcher die gesamte Größe des Zielcanvas einnimmt (Z. 33-34).
Der nächste Schritt ist die Berechnung des Pixelschwerpunkts der Zeichnung, welcher für die Zentrierung der Zeichnung benötigt wird:

```
1  function drawCenteredImage() {
2    const boundingBoxCanvas = boundingBoxCanvasRef.current;
3    const boundingBoxCtx = boundingBoxCanvas.getContext("2d");
4
5    // Array an Pixelinformationen des Bounding Box Canvas
6    const boundingBoxImageData = boundingBoxCtx.getImageData(
7      0,
8      0,
9      boundingBoxCanvas.width,
10     boundingBoxCanvas.height
11   ).data;
12
13   // Berechnung des Schwerpunkts der Zeichnung
14   let sumX = 0;
15   let sumY = 0;
16   let pixelCount = 0;
17
18   for (let x = 0; x < boundingBoxCanvas.width; x++) {
19     for (let y = 0; y < boundingBoxCanvas.height; y++) {
20       const pixelIndex = (y * boundingBoxCanvas.width + x) * 4;
21       const isFilledPixel = boundingBoxImageData[pixelIndex + 3] > 0;
22       if (isFilledPixel) {
23         sumX += x;
24         sumY += y;
25         pixelCount++;
26       }
27     }
28   }
29   const xCenterOfMass = sumX / pixelCount;
30   const yCenterOfMass = sumY / pixelCount;
31   ...
32 }
```

handwriting-preprocessing.js

Zeichencanvas | Bounding Box mit Pixelschwerpunkt

Dafür wird das Bild des Bounding Box Canvas in Form von Pixelinformationen mittels `getImageData` extrahiert (Z. 6-11), und der Pixelschwerpunkt der Zeichnung als Mittelwert der Koordinaten der gefüllten Pixel berechnet (Z. 13-21). Hierbei dient der Alpha-Wert des Pixels als Indikator, ob der Pixel gefüllt ist (Z. 21). Hier ist zu beachten, dass die bisherigen Canvas-Elemente keinen Hintergrund besitzen und somit nur die gezeichneten Linien gefüllte Pixel haben (und somit der Alpha-Kanal als zulässiger Indikator dient). Die obigen Abbildungen zeigen zwar einen schwarzen Hintergrund, diese sind jedoch nur ein CSS-Styling und nicht tatsächlich gefüllte Pixel.
Laut der Beschreibung der Bilder des MNIST-Datensatzes passt jede Ziffer in ein 20×20 Pixel großes Quadrat, welches anschließend in ein 28×28 Pixel großes Quadrat zentriert wird, wobei der Anhangspunkt des 20×20-Quadrats der Pixelschwerpunkt der Zeichnung ist. Dieser Prozess wird auch hier simuliert:

```
1  function drawCenteredImage() {
2    ...
3    const xCenterOfMass = sumX / pixelCount;
4    const yCenterOfMass = sumY / pixelCount;
5
6    const centeredCanvas = centeredImageCanvasRef.current;
7    const centeredCtx = centeredCanvas.getContext("2d");
8
9    // MNIST Skalierungsfaktor fuer die Zentrierung
10   const mnistScaleFactor = 1 - 8 / 28;
11
12   // Offeset Berechnung um Schwerpunkt in die Mitte des Centered Canvas
        zu verschieben
13   const scaledXCenter = xCenterOfMass * mnistScaleFactor;
14   const scaledYCenter = yCenterOfMass * mnistScaleFactor;
15   const offsetX = centeredCanvas.width / 2 - scaledXCenter;
16   const offsetY = centeredCanvas.height / 2 - scaledYCenter;
17
18   centeredCtx.fillRect(0,0,centeredCanvas.width,centeredCanvas.height);
19
20   // Kopieren des Bounding Box Canvas auf das Centered Canvas
21   centeredCtx.drawImage(
22     boundingBoxCanvas,
23     offsetX,
24     offsetY,
25     boundingBoxCanvas.width * mnistScaleFactor,
26     boundingBoxCanvas.height * mnistScaleFactor
27   );
28 }
```

handwriting-preprocessing.js

Es wird ein neues, drittes Canvas-Element `centeredImageCanvas` benutzt, welches die Zentrierung der Zeichnung darstellt. Der Skalierungsfaktor `mnistScaleFactor` sorgt dabei dafür, dass die Zeichnung im Bounding Box Canvas auf $1 - \frac{8}{28} \approx 71\%$ reduziert wird (analog zum 20×20-Quadrat im MNIST-Datensatz). Die Verschiebung des Pixelschwerpunkts in die Mitte des `centeredImageCanvas` wird durch die Berechnung des Offsets `offsetX` und `offsetY` erreicht (Z. 15-16), sodass letztlich der Pixelschwerpunkt des skalierten Bounding Box Canvas in der Mitte des `centeredImageCanvas` liegt (Z. 20-26). Der dritte Canvas hat nun tatsächlich einen schwarzen Hintergrund, welcher durch die `fillRect`- Funktion erzeugt wird (Z. 18).

Wird kein `fillStyle` gesetzt, füllt `fillRect` mit der Standard-farbe schwarz

Zeichencanvas | Bounding Box | Zentriertes Bild (am Pixelschwerpunkt)

Der letzte Schritt ist die Umwandlung des zentrierten Bildes in ein 28×28 Pixel großes Graustufenbild, dessen Pixelwerte extrahiert und in ein Array umgewandelt werden, welches als Eingabe für das Modell dient.

```
1  ...
2  function drawPixelImage() {
3    const pixelCanvasCtx = pixelCanvasRef.current.getContext("2d");
4    const centeredCanvas = centeredImageCanvasRef.current;
5    pixelCanvasCtx.drawImage(centeredCanvas, 0, 0, 28, 28);
6
7    // Array an Pixelinformationen des Pixel Canvas
8    const pixelData = pixelCanvasCtx.getImageData(0, 0, 28, 28).data;
9    const pixels = [];
10
11   // Berechnung der Grauwerte der Pixel
12   for (let i = 0; i < pixelData.length; i += 4) {
13     const pixel = [pixelData[i], pixelData[i + 1], pixelData[i + 2]];
14     pixels.push(pixel);
15   }
16   const grayscaleValues = pixels.map(([r, g, b]) => (r + g + b) / 3);
17   setPixelImage(grayscaleValues);
18 }
19 ...
20 return(
21   ...
22   <canvas ref={canvasRef}/>
23   <canvas ref={boundingBoxCanvasRef}/>
24   <canvas ref={centeredImageCanvasRef}/>
25   <canvas
26     ref={pixelCanvasRef}
27     width={28}
28     height={28}
29   />
30   ...
31 )
```

handwriting-preprocessing.js

Dies wird erreicht, indem das zentrierte Bild auf ein 28×28-Pixel großen Pixel-Canvas kopiert wird (Z. 5, Z. 25-29) und dessen Pixelinformationen mittels `getImageData` extrahiert werden (Z. 8). Diese Pixelinformationen werden dann in ein Array umgewandelt, wobei die Grauwerte der Pixel als Durchschnitt der RGB-Werte berechnet werden (Z. 9-16). Das Array wird dann als Zustand `pixelImage` gespeichert und kann als Eingabe für ein Modell verwendet werden.

Trainieren eines Modells

Das Trainieren der Modelle für die Ziffern- und Buchstabenerkennung erfolgt in einem separaten Python-Skript mittels der Bibliothek TensorFlow [1]. Im Folgenden soll lediglich am Beispiel des Ziffernmodells gezeigt werden, wie das Modell trainiert wird, wobei die Schritte für das Buchstabenmodell ähnlich sind:

```
1 import tensorflow as tf
2 import tensorflowjs as tfjs
```

mnist-digits.ipynb

Zunächst werden die notwendigen Bibliotheken importiert, wobei `tensorflow` für das Modelltraining und `tensorflowjs` für die Konvertierung des Modells in ein webfähiges Format benötigt wird.

```
1 # Laden und Strukturieren des MNIST Datensatzes
2 mnist = tf.keras.datasets.mnist
3 (x_train, y_train), (x_test, y_test) = mnist.load_data()
```

mnist-digits.ipynb

Der MNIST-Datensatz ist in gewisser Weise ein Standarddatensatz für das Trainieren von Modellen zur Handschrifterkennung und ist einer der wenigen Datensätze, die bereits in TensorFlow integriert sind, sodass die sonst mühsame Datenaufbereitung entfällt. Der Datensatz wird in Trainings- und Testdaten aufgeteilt und ist wie folgt strukturiert:

- `x_train`: 60.000 Bilder von handgeschriebenen Ziffern mit einer Größe von 28×28 Pixel
- `y_train`: 60.000 Labels der handgeschriebenen Ziffern (Zuweisungen der Bildern zu den Ziffern 0-9)
- `x_test`: 10.000 Bilder von handgeschriebenen Ziffern mit einer Größe von 28×28 Pixel
- `y_test`: 10.000 Labels der handgeschriebenen Ziffern (Zuweisungen der Bildern zu den Ziffern 0-9)

Die Testdaten werden später dazu verwendet, um die Qualität des trainierten Modells zu evaluieren, das heißt die Fähigkeit des Modells, auf neue, noch nicht gesehene Daten zu generalisieren.
Bevor das Modell definiert und trainiert wird, werden die Pixelwerte der Bilder normalisiert:

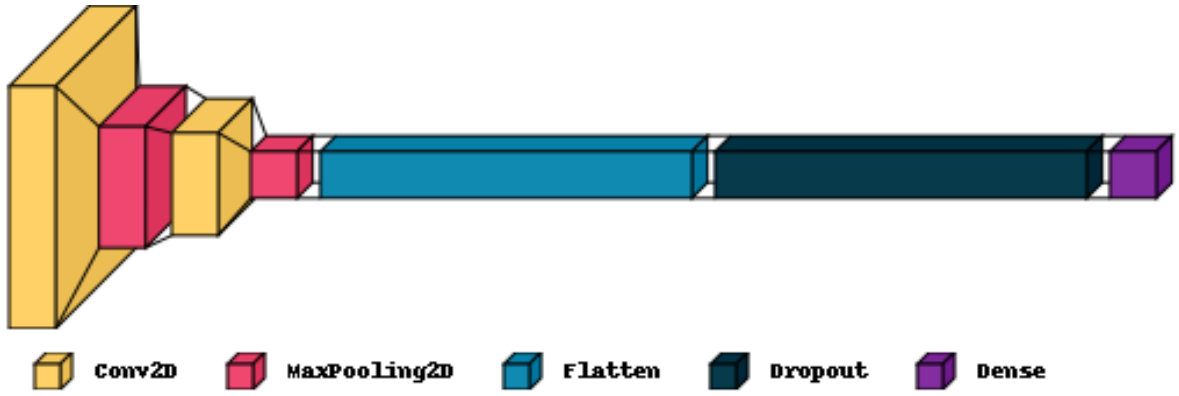
```
# Normalisierung der Pixelwerte
x_train = tf.keras.utils.normalize(x_train, axis=1)
x_test = tf.keras.utils.normalize(x_test, axis=1)
```

mnist-digits.ipynb

Durch die Normalisierung der Pixelwerte werden diese auf einen einheitlichen Wertebereich gebracht, anstatt $[0, 255]$ nun $[0, 1]$. Dies führt zu einer schnelleren und stabileren Konvergenz während des Trainings (vgl. [15]).

```
# Definiere Modellarchitektur
model = tf.keras.models.Sequential([
  # Input-Schicht und erste Faltung mit Max-Pooling
  tf.keras.layers.Conv2D(32, (3, 3), activation='relu',
                                     input_shape=(28, 28, 1)),
  tf.keras.layers.MaxPooling2D((2, 2)),

  # Zweite Faltung mit Max-Pooling
  tf.keras.layers.Conv2D(64, (3, 3), activation='relu'),
  tf.keras.layers.MaxPooling2D((2, 2)),

  # Umwandlung in Vektor und Ausgabeschicht
  tf.keras.layers.Flatten(),
  tf.keras.layers.Dropout(0.5),
  tf.keras.layers.Dense(10, activation='softmax')
])
```

mnist-digits.ipynb



Visualisierung der Modellstruktur (mittels `visualkeras` [17])

Schließlich wird die Modellstruktur definiert. Die spezifische Struktur des Modells kann hierbei variieren und ist eine Designentscheidung. Da der MNIST-Datensatz ein recht beliebter Datensatz ist, gibt es viele vorgefertigte Modelle und Architekturen, die bereits gute Ergebnisse erzielt haben. Die obige Struktur ist jedoch eine recht simple CNN-Architektur mit lediglich zwei Faltungsschichten. Wesentlich ist hier die Angabe der Eingangsgröße des Modells (Z. 5) `input_shape=(28, 28, 1)`, welche der Größe der Bilder des MNIST-Datensatzes entspricht (28×28 Pixel und ein Farbkanal). Dieses Eingabebild wird dann in einer ersten Faltungsschicht durch 32 Filter mit einer Größe von 3×3 Pixeln transformiert, wobei die Aktivierungsfunktion `relu` verwendet wird (Z. 4). Die Anwendung der Filter erlaubt es, Merkmale, wie Kanten, Ecken oder Linien, aus den Bildern zu extrahieren, indem die Filter über das Bild "gefaltet" werden, was in einer sogenannten Feature-Map resultiert, welche die Merkmale enthält. Die Max-Pooling-Schicht reduziert dann anschließend die Dimensionalität der Feature-Map, hier wird aus jedem

$relu(x) = max(x, 0)$

2×2-Fenster lediglich der maximale Wert beibehalten. Die Feature-Map hat somit die räumliche Dimension der Eingabe halbiert, es bleiben nur die wichtigsten Merkmale erhalten. Das hat zur Folge, dass das Modell robuster gegenüber Translationen/Variationen in der Position der Merkmale wird, da Positionsinformationen durch das Max-Pooling verloren gehen. Selbiger Prozess wird in der zweiten Faltungsschicht wiederholt (Z. 9-10), wobei die Anzahl der Filter auf 64 erhöht wird. Die finale Feature-Map wird dann in einen Vektor umgewandelt (Z. 13) und durch eine Dropout-Schicht reguliert (Z. 14), um Overfitting zu vermeiden (vgl. [31]). Die Dropout-Rate von 0.5 bedeutet, dass die Hälfte der Neuronen in der Schicht während des Trainings ausgeschaltet wird, was zu einer besseren Generalisierung des Modells führen kann. Schließlich folgt eine Dense-Schicht mit 10 Neuronen (`Dense(10)`), welche die Ausgabeschicht des Modells darstellt. Die Aktivierungsfunktion `softmax` wird dabei verwendet, um die Ausgabe des Modells in Wahrscheinlichkeiten zu transformieren, sodass die Klasse mit der höchsten Wahrscheinlichkeit als Vorhersage des Modells dient.
Hier sei zu betonen, dass mittels einer reinen `Dense`-Modellstruktur wie:

```
  model = tf.keras.models.Sequential([
    tf.keras.layers.Flatten(input_shape=(28, 28)),
    tf.keras.layers.Dense(128, activation='relu'),
    tf.keras.layers.Dense(10)
  ])
```

das Modell in der Validierung ebenfalls eine hohe Genauigkeit erreichen würde, jedoch lediglich auf den Testdaten des MNIST-Datensatzes. Wenngleich die Vorverarbeitung der Eingaben des Zeichencanvas weitgehend dem des MNIST-Datensatzes entspricht, ist die Wahl eines CNNs für die Handschrifterkennung aufgrund der Fähigkeit von CNNs, räumliche Muster in Bildern zu erkennen und zu generalisieren, zu bevorzugen. Das heißt, dass die vom Zeichencanvas erzeugten Eingaben nicht dem exakten Format (Liniendicke, Schriftart, Schreibstil) des MNIST-Datensatzes entsprechen müssen. Kleine Verschiebungen, Rotationen oder Skalierungen der Zeichnung können durch die Faltungsschichten des CNNs erkannt und trotzdem korrekt klassifiziert werden.
Nach der Definition der Modellstruktur wird das Modell kompiliert:

```
# Kompilierung des Modells
model.compile(optimizer='adam', loss='sparse_categorical_crossentropy',
    metrics=['accuracy'])
```

mnist-digits.ipynb

Der Optimierer und die Verlustfunktion sind dabei übliche Konfigurationen für Klassifikationsprobleme.
Schließlich wird das Modell trainiert und auf den Testdaten evaluiert:

```
# Trainieren des Modells
model.fit(x_train, y_train, epochs=30)

# Evaluierung des Modells
val_loss, val_acc = model.evaluate(x_test, y_test)
print(val_loss, val_acc)
```

mnist-digits.ipynb

Das Modell wird für 30 Epochen trainiert, wobei eine Epoche die Iteration über den gesamten Trainingsdatensatz darstellt. Letztlich wird das trainierte Modell auf den Testdaten evaluiert, um die Qualität des Modells auf neuen, noch nicht gesehenen Daten zu bestimmen. Nach 30 Epochen erreicht das Modell einen Verlustwert (`val_loss`) von $0.0283 = 2.83\%$ und eine Genauigkeit (`val_acc`) von $0.9923 = 99.23\%$, was als gute Genauigkeit für die Ziffernerkennung auf den Testdaten des MNIST-Datensatzes zu bewerten ist.

Konvertierung des Modells

Der letzte Schritt ist die Konvertierung des Modells in ein webfähiges Format, das in die Zeichenkomponente eingebunden werden kann, um die auf dem Zeichencanvas gezeichneten Ziffern zu klassifizieren.

```
# Konvertierung des Modells in ein webfaehiges Format
tfjs.converters.save_keras_model(model, 'model')
```

mnist-digits.ipynb

Die Konvertierung erfolgt mittels der `tfjs.converters`-Funktion der `tensorflowjs`-Bibliothek, die das Modell (im Keras-Format) in ein JSON-Format und die Gewichte in ein Binärformat umwandelt. Das Ergebnis der Umwandlung sind zwei Dateien:
Eine `model.json`-Datei, die die Struktur des Modells enthält, und eine `group1-shard1of1.bin`-Datei, die die Gewichte des Modells enthält. Hier soll darauf hingewiesen werden, dass im Entwicklungsprozess eine korrekte Konvertierung lediglich mit einer Python-Version von 3.8 oder niedriger erreicht wurde. Höhere Versionen führten zu einer fehlerhaften Modellbeschreibung in der `model.json`-Datei. Im Webprojekt werden die beiden Dateien in den Ordner `public/models` verschoben und in den entsprechenden Komponenten eingebunden:

```
import * as tf from "@tensorflow/tfjs";
...
const modelRef = useRef(null);
...
useEffect(() => {
  async function loadModel() {
    try {
      const model = await tf.loadLayersModel("/model/digit-recognition/
          model.json");
      modelRef.current = model;
    } catch (error) {
      console.error("Error loading model:", error);
    }
  }
  loadModel();
}, []);
```

mnist-digit-recognizer.js

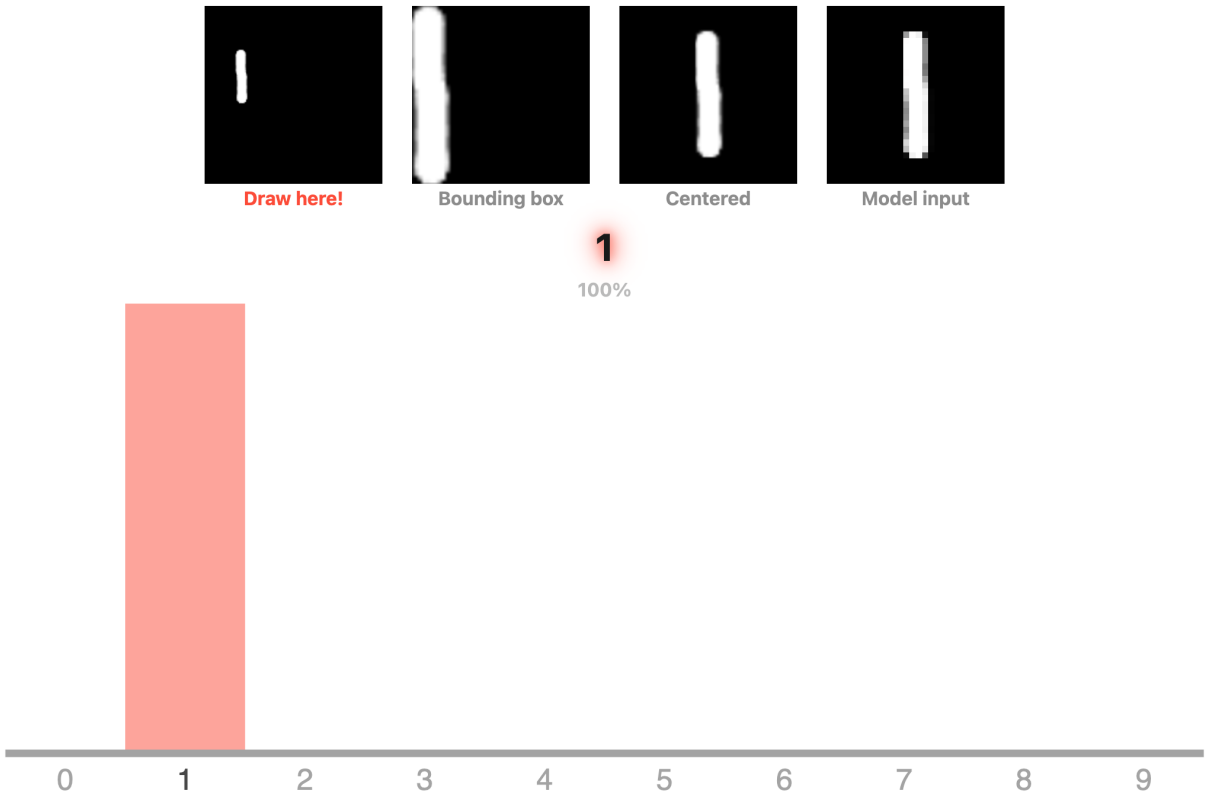
Das in der `model.json`-Datei beschriebene Modell wird mittels der `loadLayersModel`-Funktion geladen und in der `modelRef`-Referenz gespeichert. Das in der Referenz gespeicherte Modell kann dann zur Klassifikation der auf dem Zeichencanvas gezeichneten Ziffern verwendet werden. Diese Ziffern werden durch die Vorverarbeitungsschritte in ein 28x28 Pixel großes Graustufenbild umgewandelt.

```
function predict(grayscaleValues) {
  const input = tf.tensor4d(grayscaleValues, [1, 28, 28, 1]).div(255.0);
  const prediction = modelRef.current.predict(input).dataSync();
  input.dispose();
  return prediction;
}
```

mnist-digit-recognizer.js

Die Funktion `predict` nimmt die Grauwerte der Pixel als Eingabe, wandelt diese in ein Tensor-Objekt um und normalisiert die Werte auf den Bereich $[0, 1]$ (Z.1). Der 4D-Tensor beschreibt dabei, dass es sich um ein Bild handelt, das eine Größe von 28x28 hat und einen Farbkanal (Graustufenbild) besitzt (→ `[1, 28, 28, 1]`).
Das Modell wird dann auf den Eingabetensor angewendet und mittels der `dataSync`-Funktion in ein Array umgewandelt, das die Vorhersagen des Modells auf den Eingabedaten enthält (Z.2).

Der Eingabetensor wird entsorgt (Z.3), um Speicherlecks zu vermeiden, und schließlich werden die Vorhersagen zurückgegeben (Z.4), die dem Nutzer in Form von Wahrscheinlichkeiten visualisiert werden:



Zusammenfassend bilden die hier vorgestellten Methoden, darunter die Erstellung eines Scrollytelling-Gerüsts, Techniken zur schrittweisen Erweiterung und Aktualisierung von Visualisierungen, der Aufbau interaktiver Demos sowie die Integration von ML-Modellen in Webprojekte, die wesentlichen Bausteine mit denen insgesamt 31 Scrollytelling-Abschnitte und 21 interaktiven Demos des Webprojekts realisiert wurden.

6. Erweiterungsmöglichkeiten und Ausblick

Das Webprojekt bietet dabei eine Vielzahl an Erweiterungsmöglichkeiten, die sinnvolle Ergänzungen darstellen können. Die bisherige Aufteilung des Webprojekts ist momentan durch eine interaktive Geschichtserzählung in Form des Scrollytellings und einer anschließenden passenden interaktiven Demonstration von Algorithmen des maschinellen Lernens beschrieben. Dabei ist die interaktive Natur des Scrollytellings primär durch die scrollbasierte und dynamische Anpassung von SVG-Elementen gegeben. Hier wäre es möglich, diese Interaktivität zu erweitern, indem zusätzliche Interaktionen mit den Visualisierungen ermöglicht werden. Die Verwendung von Tooltips, die Einbettung von Drag-and-Drop-Elementen oder das Hinzufügen von interaktiven Schiebereglern sind nur einige Beispiele von vielen Möglichkeiten, die Interaktivität der Scrollytelling-Elemente zu erweitern. Damit würde die momentan strikte Aufteilung von visuell-textueller Erklärung in Form des Scrollytellings und einer anschließenden interaktiven Demonstration aufgebrochen und die Interaktivität des gesamten Webprojekts erhöht werden. Interaktionselemente wie Tooltips oder Drag-and-Drop-Gesten werden dabei oft auf die Interaktion mit der Maus beschränkt, und Touch-Gesten sowie Multi-Touch-Gesten werden aufgrund des damit verbundenen Entwicklungsaufwands vernachlässigt. Der aktuelle Stand des Webprojekts beschränkt sich mit Ausnahme der letzten interaktiven Demonstration eines Zeichencanvas ebenfalls nur auf einfache Touch-"Klick"-Gesten. Interaktive Elemente könnten hier auf mehrere Events wie "touchstart", "touchmove" und "touchend" erweitert werden, um eine erweiterte Interaktion auf mobilen Endgeräten zu ermöglichen. Mausevents könnten ebenfalls öfter durch Events wie "mousedown", "mousemove", "mouseup", aber auch "wheel" und "contextmenu" erweitert werden, um die Interaktion auf Desktop-Endgeräten zu verbessern. Da die Dokumentation von Touch- und Mouse-Events insbesondere in Bezug auf SVG und D3.js stark variiert, soll hier auf den interaktiven Artikel "Multitouch done right" referiert werden, der die Thematik ausführlicher und vor allem interaktiv behandelt (vgl. [29]). Natürlich ist die Fülle der behandelten Themenbereiche des maschinellen Lernens auszuarbeiten. Der aktuelle Stand des Webprojekts beschränkt sich auf die wesentlichen Konzepte der Wahrscheinlichkeitsdichteschätzung, Regressions- und Klassifikationsansätze sowie die Grundlagen von neuronalen Netzen. Im Wesentlichen ist hier die Mitaufnahme der Themen des unüberwachten Lernens (Unsupervised Learning) sowie des bestärkenden Lernens (Reinforcement Learning) zu nennen. Ebenso könnten die bisherigen Themenbereiche um weitere Methoden erweitert werden. Nennenswert sind hier der fortgeschrittene Klassifikationsansatz der "Support Vector Machines" (SVMs), die Deep-Learning-Architektur der "Convolutional Neural Networks" (CNNs), um Bildklassifikationen genauer zu beschreiben, oder der sogenannte EM-Algorithmus, um die Thematik der Wahrscheinlichkeitsdichteschätzung zu vertiefen. Darüber hinaus würde die Einbindung einer 3D-Visualisierungsbibliothek wie Three.js (vgl. [32]) die Möglichkeit bieten, komplexere Visualisierungen, wie sie ständig in den Klassifikations- und Regressionsansätzen vorkommen, besser zu visualisieren. Die momentane Zwischenlösung, die zusätzliche Dimensionen durch Farben darzustellen, könnte so durch eine räumliche Darstellung ersetzt werden, was insgesamt intuitiver und besser verständlich wäre. Ebenso ist zu betonen, dass das Webprojekt momentan nur in englischer Sprache verfügbar ist. Weitere Sprachen könnten die Zugänglichkeit des Projekts erhöhen und somit die Reichweite des Projekts erweitern. Darüber hinaus kann dieses Projekt auch als Anreiz verstanden werden, technische und wissenschaftliche Konzepte etablierter in webbasierten und interaktiven Lernplattformen zu vermitteln. Disziplinen wie die Mathematik, Physik, Informatik und viele andere könnten von einer solchen Plattform profitieren, vorausgesetzt, dass Hürden wie die technische Umsetzung, die didaktische Aufbereitung und vor allem

die barrierearme Gestaltung überwunden werden.

A. Bibliographie

[1] Martin Abadi, Ashish Agarwal, Paul Barham, Eugene Brevdo, and Zhifeng Chen et al. Tensorflow: Large-scale machine learning on heterogeneous systems, 2015. Software available from tensorflow.org. URL: https://www.tensorflow.org/.

[2] Christopher M Bishop and Nasser M Nasrabadi. *Pattern recognition and machine learning*, volume 4. Springer, 2006.

[3] Mike Bostock. What is D3? Zugriff: 09.09.2024. URL: https://d3js.org/what-is-d3.

[4] Mike Bostock and Inc. Observable. D3 Color. Zugriff: 24.06.2024. URL: https://d3js.org/d3-scale-chromatic/sequential.

[5] Daniel Smilkov Carter and Shan. Tensorflow — neural network playground. Zugriff: 06.08.2024. URL: http://playground.tensorflow.org.

[6] Rich Caruana, Yin Lou, Johannes Gehrke, Paul Koch, Marc Sturm, and Noemie Elhadad. Intelligible models for healthcare: Predicting pneumonia risk and hospital 30-day readmission. In *Proceedings of the 21th ACM SIGKDD international conference on knowledge discovery and data mining*, pages 1721–1730, 2015.

[7] Nathan M Castillo, Jinsol Lee, Fatima T Zahra, and Daniel A Wagner. Moocs for development: Trends, challenges, and opportunities. *Information Technologies & International Development*, 11(2):pp–35, 2015.

[8] Leon Chen. Keras.js. Zugriff: 6. August 2024. URL: https://transcranial.github.io/keras-js/#/mnist-cnn.

[9] Gregory Cohen, Saeed Afshar, Jonathan Tapson, and André van Schaik. Emnist: an extension of mnist to handwritten letters, 2017. URL: https://arxiv.org/abs/1702.05373, arXiv:1702.05373.

[10] Peter Cook. D3 Data Joins. Zugriff: 24.06.2024. URL: https://www.d3indepth.com/datajoins/.

[11] Peter Cook. D3 Enter-Update-Exit. Zugriff: 24.06.2024. URL: https://www.d3indepth.com/enterexit/.

[12] Peter Cook. D3 Scales. Zugriff: 24.06.2024. URL: https://www.d3indepth.com/scales/.

[13] Peter Cook. D3 Selections. Zugriff: 24.06.2024. URL: https://www.d3indepth.com/selections/.

[14] Peter Cook. D3 Transitions. Zugriff: 24.06.2024. URL: https://www.d3indepth.com/transitions/.

[15] Google Developers. Numerical data: Normalization. Zugriff: 14.09.2024. URL: https://developers.google.com/machine-learning/crash-course/numerical-data/normalization.

[16] Jiaming Fang, Lufen Tang, Jingjing Yang, and Min Peng. Social interaction in moocs: The mediating effects of immersive experience and psychological needs satisfaction. *Telematics and Informatics*, 39:75–91, 2019.

[17] Paul Gavrikov. visualkeras. https://github.com/paulgavrikov/visualkeras, 2020.

[18] David Gunning and David Aha. Darpa's explainable artificial intelligence (xai) program. *AI magazine*, 40(2):44–58, 2019.

[19] Jason Kao. React Scrollama. Zugriff: 09.09.2024. URL: https://github.com/jsonkao/react-scrollama.

[20] Hanan Khalil and Martin Ebner. Moocs completion rates and possible methods to improve retention-a literature review. *EdMedia+ innovate learning*, 2014.

[21] RWTH Aachen LearnTech. bridgingai - learntech. Zugriff: 06.08.2024. URL: https://learntech.rwth-aachen.de/cms/learntech/forschung/projekte/~slcsa/bridgingai/.

[22] Jakub Majorek (monterail). Vergleich von JS Visualisierungs Bibliotheken. Zugriff: 22.07.2024. URL: https://www.monterail.com/blog/javascript-libraries-data-visualization#first.

[23] Mozilla. SVG Basic Shapes. Zugriff: 24.06.2024. URL: https://developer.mozilla.org/en-US/docs/Web/SVG/Tutorial/Basic_Shapes.

[24] Mozilla. SVG Fills and Strokes. Zugriff: 24.06.2024. URL: https://developer.mozilla.org/en-US/docs/Web/SVG/Tutorial/Fills_and_Strokes.

[25] Mozilla. SVG Paths. Zugriff: 24.06.2024. URL: https://developer.mozilla.org/en-US/docs/Web/SVG/Tutorial/Paths.

[26] Mozilla. SVG Positions. Zugriff: 24.06.2024. URL: https://developer.mozilla.org/en-US/docs/Web/SVG/Tutorial/Positions.

[27] Mozilla. SVG Transformations. Zugriff: 24.06.2024. URL: https://developer.mozilla.org/en-US/docs/Web/SVG/Tutorial/Basic_Transformations.

[28] Numeric.js. Numerical computations in Javascript. Zugriff: 09.09.2024. URL: https://ccc-js.github.io/numeric2/documentation.html.

[29] Oservable. Multitouch done right, 2023. URL: https://observablehq.com/@d3/multitouch.

[30] Marco Tulio Ribeiro, Sameer Singh, and Carlos Guestrin. "why should i trust you?" explaining the predictions of any classifier. In *Proceedings of the 22nd ACM SIGKDD international conference on knowledge discovery and data mining*, pages 1135–1144, 2016.

[31] TensorFlow. Dropout. Zugriff: 14.09.2024. URL: https://www.tensorflow.org/api_docs/python/tf/keras/layers/Dropout.

[32] Three.js. JavaScript 3D Library. Zugriff: 09.09.2024. URL: https://threejs.org/.

[33] W3Schools. SVG Markers. Zugriff: 24.06.2024. URL: https://www.w3schools.com/graphics/svg_marker.asp.

[34] W3Schools. SVG Tutorial. Zugriff: 24.06.2024. URL: https://www.w3schools.com/graphics/svg_intro.asp.

[35] Feiyu Xu, Hans Uszkoreit, Yangzhou Du, Wei Fan, Dongyan Zhao, and Jun Zhu. Explainable ai: A brief survey on history, research areas, approaches and challenges. In *Natural language processing and Chinese computing: 8th cCF international conference, NLPCC 2019, dunhuang, China, October 9–14, 2019, proceedings, part II 8*, pages 563–574. Springer, 2019.

[36] Christopher J.C. Burges Yann LeCun, Corinna Cortes. Mnist datensatz. Zugriff: 22. August 2024. URL: https://yann.lecun.com/exdb/mnist/.